SSW 590 — DevOps

Group 1 DevOps IT Manual

Version 1.1+build.10.f8a6179

Group 1 Submission

by

Group 1: Brahmi Dalvi and Sakshi Deshpande

Stevens Institute of Technology

October 20, 2026

© Stevens Institute of Technology 2026
ALL RIGHTS RESERVED

Template Attribution

This manual is built from the SSW 590 DevOps assignment template (version 1.3, distributed through Canvas), developed by **Darian Muresan, Ph.D.**, in conjunction with **OpenAI Codex** and **Anthropic Claude Code**.

SSW 590 — DevOps

Group 1 DevOps IT Manual

Version 1.1+build.10.f8a6179
Group 1 Submission

Group 1: Brahmi Dalvi and Sakshi Deshpande
Stevens Institute of Technology
October 20, 2026

This is Group 1's living DevOps IT manual for SSW 590. It records what the group actually built, tested, and learned, one chapter per course module. The first chapter documents the startup's Gitea work hub: a student-pack domain, a version-pinned Gitea deployment, HTTPS with automated certificate renewal, a minimal firewall exposure, and an issue-to-reviewed-merge team workflow. Results are stated only where they were observed. The one thing not observed, the first automatic renewal of the production certificate (expected between December 13 and 15, 2026, after the due date), is labeled as such.

Document History

How to Use This Chapter

Add the newest approved change at the top. Retain every approved entry and link each entry to the changed chapter, table, figure, or controlled artifact. Every entry must say whether the Hosts and Passwords tables changed. Author initials: BD = Brahmi Dalvi, SD = Sakshi Deshpande, CC = Claude Code (AI assistant; see the AI Prompt Log).

Table 1: Document Update History

Date / Version	Author Initials	Change	Changed At
October 20, 2026 / 1.1	SD, BD, CC	New Chapter 4 (course Chapter 3): the manual's source moved to `group1-startup/manual` and is built on every push by Gitea Actions on a self-managed runner; the PDF carries a build identifier (run number and commit); evidence 50–66 inline. Version increased from 1.0 to 1.1. **Hosts and Passwords tables: changed** (runner host row; runner registration token hint; SD's Write access to the manual repository).	Chapter 4, Chapter 2, Section 4.2.8

Document Update History (continued)

Date / Version	Author Initials	Change	Changed At
October 15, 2026 / 1.0 (final)	BD, CC	Final fixes before submission: removed the last placeholder markers; set the repository link and the document status; corrected the Introduction link; marked the optional extra credit as not attempted; wrote the human-review lines in the AI Prompt Log; updated the front status table, the abstract, and notes inside captured evidence that had become out of date; made the repository public after a scan of its full history found no secrets. Earlier the same day: second-network firewall test, certificate-renewal dry run on Let's Encrypt staging, redacted AWS console evidence, cost estimate, row 6 setup run, and all evidence embedded inline. **Hosts and Passwords tables: updated** (Sakshi's SSH key recorded as having no passphrase; her Gitea MFA recorded as on, as confirmed by her; Brahmi's MFA and credential-storage hints added).	Chapter 3, Chapter 2, Chapter C
October 15, 2026 / 1.0	SD, CC	Submission version of the Gitea chapter (course Chapter 2): all eight rubric subsections carry real, dated evidence inside the PDF. Final quality pass: version increased from 0.1 to 1.0; secrets and address scan of the chapter sources and printed evidence; every screenshot inspected; failed tests from both members collected (Table 3.19). **Hosts and Passwords tables: updated in this version** (created from placeholders by BD: Gitea host, domain, earlier host, Gitea users, SSH keys, AWS, Name.com, and GitHub accounts; hints only).	Chapter 2, Chapter 3, Section 3.2.8

Document Update History (continued)

Date / Version	Author Initials	Change	Changed At
October 15, 2026 / 0.1 (in progress)	SD, CC	Row 6: wrote the startup team workflow subsection with real evidence (organization, public product repository, member roles, six-issue backlog, two issue-to-merge traces with peer approvals and automatic issue closure; evidence 25–30 inline) and recorded SD's expected HTTP 403 repository-creation failure in the failed-test log. Rows 3 and 7 evidence moved inline as listings and figures. Hosts and Passwords tables: **not changed** by this entry.	Section 3.2.6, Tables 3.14 and 3.15, Table 3.19
October 15, 2026 / 0.1 (in progress)	SD, CC	Row 7: rewrote the architecture and operation subsection for the live AWS deployment (figure labels, request and administration paths, interface table incl. Caddy admin port and IPv6, data locations, runbook, troubleshooting) and ran the read-only runbook commands on the server (evidence 24). Hosts and Passwords tables: **not changed** by this entry.	Section 3.2.7, Figure 3.17
October 15, 2026 / 0.1 (in progress)	BD, CC	Claimed the domain `ssw590-g1.tech` through the Student Pack ($0.00, automatic renewal off, stored card deleted); created the AWS security group, key pair, `t3.micro` instance, and Elastic IP with the AWS command line interface; created the DNS record; deployed Gitea 1.27.3 and Caddy 2.11.7 with Docker Compose and obtained a Let's Encrypt certificate; created the first administrator, the organization `group1-startup`, the team `developers`, and Sakshi's account and SSH key; wrote the domain, deployment, HTTPS, and firewall subsections from captured evidence, with the renewal test, the second-network test, and the open items marked as pending. **Hosts and Passwords tables: updated** (created from placeholders; the Gitea host, domain, earlier host, accounts, and keys are now listed, hints only).	Chapter 3, Chapter 2

Document Update History (continued)

Date / Version	Author Initials	Change	Changed At
October 15, 2026 / 0.1 (in progress)	SD, CC	Row 3: ran the persistence test on the EC2 server (container replaced, repository, issue, and commit unchanged) and replaced the pending results with the real log (evidence 20) and conclusion. Hosts and Passwords tables: **not changed** by this entry.	Section 3.2.3, Table 3.6
October 12, 2026 / 0.1 (SD draft, branch `sd/gitea`)	SD, CC	Drafted the four SD regions of the Gitea chapter: database and persistence (SQLite settings, data locations, scripted persistence-test procedure and a local rehearsal clearly marked as non-graded), startup team workflow (backlog and trace tables, pending G3), architecture and operation (new TikZ figure, request path, interface table, runbook, persistence-versus-backup), and manual quality controls (checklist and failed-test log). Added glossary terms (SQLite, named volume, pull request) and one citation. Hosts and Passwords tables: **not changed**. Version not yet increased; that happens at submission.	Sections 3.2.3, 3.2.6, 3.2.7, 3.2.8; Figure 3.17
October 12, 2026 / 0.1	BD, CC	Initial setup: created the separate IT manual from the course Canvas template (version 1.3, one chapter per file, template guides retained as appendices), the Gitea chapter outline with the eight rubric-aligned subsections and an extra-credit subsection, and the Hosts and Passwords tables. Hosts and Passwords tables created, with no real hosts or secrets recorded yet. No infrastructure exists yet; every evidence item is pending.	Chapter 1, Chapter 2, Chapter 3

Table of Contents

Template Attribution iii

Document History v

1 Introduction 1
- 1.1 Purpose . . . 1
- 1.2 Team and Repository . . . 1
- 1.3 Chapter Map . . . 1
- 1.4 Repository Layout . . . 1

2 Hosts and Passwords 3
- 2.1 Hosts . . . 3
- 2.2 Passwords (hints only) . . . 4

3 Startup Foundation with Gitea 8
- 3.1 Overview and Status . . . 8
- 3.2 Required Subsections . . . 9
  - 3.2.1 Student Pack domain and DNS (10 points) . . . 9
  - 3.2.2 Pinned Gitea deployment (10 points) . . . 14
  - 3.2.3 Database and persistence (10 points) . . . 22
  - 3.2.4 HTTPS and certificate renewal (15 points) . . . 26
  - 3.2.5 Firewall and external exposure (15 points) . . . 33
  - 3.2.6 Startup team workflow (20 points) . . . 38
  - 3.2.7 Architecture and operation (5 points) . . . 43
  - 3.2.8 Manual quality and controls (15 points) . . . 47
- 3.3 Extra Credit . . . 50
  - 3.3.1 Extra credit: verified backup and restore (up to 20 extra-credit points; not attempted) . . . 50

4 Automated Manual Build 51
- 4.1 Overview and Status . . . 51
- 4.2 Required Subsections . . . 51
  - 4.2.1 Source and trigger (15 points) . . . 51

4.2.2 Runner setup (15 points) . . . 52
4.2.3 Reproducible compilation (20 points) . . . 57
4.2.4 PDF version traceability (15 points) . . . 61
4.2.5 Downloadable artifact (10 points) . . . 61
4.2.6 Change, failure, and repair evidence (10 points) . . . 64
4.2.7 Runner and secret boundary (5 points) . . . 67
4.2.8 Manual quality and controls (10 points) . . . 68

Bibliography **71**

A Document Authoring Guide **73**
A.1 How to Use This Appendix . . . 73
A.2 Template Provenance . . . 73
A.3 File Naming and Responsibilities . . . 73
A.4 Synonyms and Controlled Terms . . . 73
A.5 Building the PDF . . . 74
A.5.1 macOS and Linux Build . . . 74
A.5.2 Windows PowerShell Build . . . 74
A.6 Install LaTeX Tools on macOS . . . 74
A.6.1 Recommended Installation . . . 74
A.6.2 Verification and Compilation . . . 74
A.7 Install LaTeX Tools on Windows . . . 75
A.7.1 Recommended Installation . . . 75
A.7.2 Verification and Compilation . . . 75
A.8 Common Build Problems . . . 75
A.9 Authoring and Review Rules . . . 76

B AI Agent Usage Guide **77**
B.1 How to Use This Appendix . . . 77
B.2 Governing Principles . . . 77
B.3 Chapter File Convention . . . 77
B.4 AI Quick Start . . . 78
B.5 Task Routing . . . 78
B.6 DevOps-Specific Rules . . . 78
B.7 Example AI Requests . . . 79
B.7.1 Create an Assignment Brief . . . 79
B.7.2 Review a Student Workflow . . . 79
B.8 Human Decisions and Information Boundaries . . . 79
B.9 Completion Checklist . . . 79

C AI Prompt Log **81**
C.1 How to Use This Appendix . . . 81
C.2 Record: IT Manual Skeleton and Gitea Chapter Outline . . . 81
C.3 Record: SD Regions of the Gitea Chapter (Draft) . . . 82
C.4 Record: BD Deployment, Evidence, and Drafted Regions . . . 82

C.5 Record: SD Rows of the Automated Manual Build Chapter 83

Glossary **85**

Acronyms **87**

Index **88**

List of Tables

1 Document Update History v

2.1 Hosts 3
2.2 Passwords and keys (hints only; no secrets are recorded) 5

3.1 Rubric rows, owners, and status 8
3.2 Domain registration facts 9
3.3 Deployed components and versions (October 15, 2026) 16
3.4 Deployment checks, October 15, 2026 20
3.5 Where Gitea's persistent data lives 23
3.6 Persistence test on the cloud VM 24
3.7 Certificate options 27
3.8 Certificate served by `git.ssw590-g1.tech`, October 15, 2026 29
3.9 HTTPS checks, October 15, 2026 (rows 1 to 4 at 17:35 UTC from the administrator's computer; the renewal rows from the 19:03 to 19:08 UTC test) 33
3.10 Security group 33
3.11 Inbound rules, IPv4 (IPv6 has no rules) 34
3.12 External exposure checks 35
3.13 Startup organization and members 39
3.14 Startup backlog in `group1-startup/twintrack`, October 15, 2026 39
3.15 Issue-to-merge traces, October 15, 2026 40
3.16 Interfaces and exposure 45
3.17 Troubleshooting 46
3.18 Manual quality controls, checked October 15, 2026 47
3.19 Failed tests and corrective actions 48

4.1 Rubric rows, owners, and status 51
4.2 Network path of a push and of a runner job, with public and private ports . 52
4.3 The Gitea Actions runner 54
4.4 Build toolchain 57
4.5 One build traced from the run to the PDF 61
4.6 Successive changes, a controlled failure, and its repair, October 20, 2026 . . 65
4.7 Controlled failure: expected and actual results 65
4.8 Manual quality controls for this chapter, checked October 20, 2026 69

4.9 Failed tests and corrective actions in this chapter 70

B.1 AI Task-Routing Table 78

List of Figures

3.1 Name.com cart before payment, October 15, 2026: `ssw590-g1.tech` free for one year through the partner offer, Domain Safe Pro ($7.99) declined, total $0.00. This is the cart, not a receipt. . . . 10
3.2 Name.com order summary, October 15, 2026: domain and free Domain Safe, total $0.00. . . . 11
3.3 Domain page right after registration, October 15, 2026: "Renews Oct 15, 2027", automatic renewal ON, WHOIS privacy ON, transfer locked, "Renew for $89.99". . . . 11
3.4 Domain page, October 15, 2026: expires October 15, 2027, automatic renewal OFF, renewal price $89.99. . . . 12
3.5 Name.com DNS records for `ssw590-g1.tech`, October 15, 2026: one A record for `git`. . . . 13
3.6 Name.com billing page, October 15, 2026: "No saved payment profiles" after the card was deleted. . . . 15
3.7 Name.com order confirmation email, received Thursday October 15, 2026 at 00:36 (mailbox time): order 28974415, `ssw590-g1.tech`, one year, order total $0.00. The email gives the order date as October 14, 2026. . . . 15
3.8 The deployed Gitea at `git.ssw590-g1.tech`, signed in as the administrator, October 15, 2026. . . . 22
3.9 Repository `sakshi/persistence-check` after the container replacement, October 15, 2026: one commit, `4f15f70000`. . . . 26
3.10 Issue #1 still open after the container replacement, October 15, 2026. . . . 27
3.11 What the security group admits (green) and drops (red, dashed) for the Gitea VM [1, 2]. . . . 35
3.12 AWS console, instance `i-0c68ec1bc1e16a414`, Security tab, October 15, 2026 (redacted). . . . 38
3.13 Pull request #7 before approval, October 15, 2026: SD's review comment, and the merge blocked with "0 of 1 official approvals granted". . . . 41
3.14 Pull request #7 merged, October 15, 2026: "Closes #2", approved by `sakshi`, merged by `brahmi` as commit `0d0c3dac4d`. . . . 41
3.15 Pull request #8 merged, October 15, 2026: "Closes #1", approved by `brahmi`, merged by `sakshi` as commit `15f8a00343`. . . . 42

3.16 Closed issues in twintrack, October 15, 2026: #1 and #2 closed by their merges, #4 closed after the branch-protection test. 42
3.17 Group 1 Gitea deployment on one AWS EC2 instance, October 15, 2026. The security group admits HTTPS (443) and HTTP (80, redirect only) from anyone over IPv4, and SSH (22) only from the approved team address. Only Caddy publishes ports; Gitea's port 3000, its container sshd, Caddy's admin port 2019, and the SQLite file are private. There is no IPv6 path [1, 3, 4]. . 44

4.1 Actions tab of group1-startup/manual, October 19, 2026 (evening, US Eastern): run #1 for commit 1fb4e15 failed, run #2 for commit a5f3446 succeeded; both were started by a push to main. 53
4.2 Run #2 summary: triggered via push, status Success, duration 1 minute 11 seconds, workflow build.yml (on: push), one artifact it-manual-pdf. 53
4.3 Site Administration, Actions, Runners, October 19, 2026: group1-runner-1, version v0.6.1, type Global, label manual-builder, status Idle, last online "now". 55
4.4 Log of run #2, step *Set up job*: the job is assigned to group1-runner-1, runs in a temporary container from group1/texlive-node:2026.09, and the image is not pulled (forcePull=false). The first action fetched is actions/checkout from github.com. 55
4.5 Run #2 on the Gitea runner, October 20, 2026, seen signed out: every step succeeded and the run has one artifact, it-manual-pdf. 60
4.6 Title page of the PDF downloaded from run #2's artifact, October 20, 2026: the version reads 1.0+build.2.a5f3446, matching run #2 and commit a5f3446. 62
4.7 Title page of the PDF downloaded from the artifact of run #2 and opened in a viewer: Version 1.0+build.2.a5f3446. 63
4.8 Run #2 summary with the tooltip of it-manual-pdf: "Expires at Jan 17, 2027, 22:01" and 4.0 MiB. 64
4.9 Run #5, October 20, 2026, seen signed out: the compile step failed, and the verify and upload steps were skipped. 66
4.10 Settings, Collaborators of group1-startup/manual, October 19, 2026: sakshi with Write; teams Owners (Owner) and developers (unit permissions Code, Issues, Pull Requests, Releases, Wiki, Projects, Packages, Actions). 67

Chapter 1

Introduction

1.1 Purpose

This is Group 1's operational IT manual for the SSW 590 startup. It is a living record: one chapter is added per delivered capability, and the compiled PDF is the only artifact submitted for each assignment. It is separate from the earlier assignment report in the `report/` folder of the repository.

1.2 Team and Repository

Item	Value
Group	Group 1
Members	Brahmi Dalvi (BD), Sakshi Deshpande (SD)
Course repository	https://github.com/brahmi9811/ssw-590-group1
Team Gitea	https://git.ssw590-g1.tech; organization group1-startup, product repository twintrack

1.3 Chapter Map

Where	Content
Chapter 2	Hosts and Passwords tables (hints only)
Chapter 3	Startup foundation with Gitea (course Chapter 2)

1.4 Repository Layout

```
assignment-4/manual/   this LaTeX manual (source stays in the team; submit only the PDF)
```

```
assignment-4/gitea/    versioned, non-secret deployment files (compose, proxy config)
assignment-4/*.md      work split and per-person instructions
report/                earlier submitted assignment (do not modify)
```

Chapter 2

Hosts and Passwords

Keep both tables current; a missing or stale table costs 10 points. The Passwords table holds *hints and vault references only*: never a password, derivation key, recovery code, token, or private key. Last updated: October 15, 2026.

2.1 Hosts

Table 2.1: Hosts

Hostname	Role, environment, provider	Address, ports, owner, URL, backup, monitoring
`git.ssw590-g1.tech`	**Role:** Gitea 1.27.3 and Caddy 2.11.7 in Docker; the startup's work hub. **Environment:** the only one (no separate test environment). **Provider:** AWS EC2, `us-east-1c`, `t3.micro`. **Instance:** `i-0c68ec1bc1e16a414`	**Address:** A record to Elastic IP `52.73.254.36`, TTL 300; no IPv6. **Exposed ports:** 443 and 80 from anyone; 22 from the admin address only. **Owner:** Brahmi Dalvi. **HTTPS URL:** `https://git.ssw590-g1.tech` **Backup:** none yet (the optional backup and restore is unassigned). **Monitoring:** Docker health check only

Hostname	Role, environment, provider	Address, ports, owner, URL, backup, monitoring
`gitea-runner` (added October 19, 2026)	**Role:** Gitea Actions runner `group1-runner-1` (`act_runner` 0.6.1) that builds this manual; a Docker container on the same host as Gitea, not a separate machine. **Environment:** the only one. **Provider:** AWS EC2, same instance as above	**Address:** none of its own; it connects out to `https://git.ssw590-g1.tech`. **Exposed ports:** none published by the runner (80 and 443 stay Caddy's, 22 admin only). **Owner:** Brahmi Dalvi. **Local job image:** `group1/texlive-node:2026.09`, about 5.8 GB on the host disk. **Backup:** none; the artifact expires after about 90 days. **Monitoring:** runner status page in Gitea (Site Administration, Actions, Runners)
`ssw590-g1.tech` (domain and DNS zone)	**Role:** domain registration and DNS records. **Provider:** Name.com, via the GitHub Student Pack offer	**DNS:** Name.com nameservers (`ns1bqx`, `ns2kry`, `ns3jmt`, `ns4ksy` at `name.com`). **Ports:** not a server. **Owner:** Brahmi Dalvi. **Expires:** October 15, 2027; automatic renewal OFF (renewal would cost $89.99); first year free
`ssw590-group1-web` (previous assignment)	**Role:** color-buttons website from the earlier assignment, unrelated to Gitea. **Provider:** AWS EC2, `us-east-1`, `t3.micro`. **Instance:** `i-00ee7edd4a5352f05`	**Address:** public DNS name recorded in the earlier report (it changes if the instance is stopped). **Exposed ports:** per the earlier report 80, 443, 22 (not re-checked here). **Owner:** Brahmi Dalvi. **Status:** not touched by this assignment; to be disabled after that assignment is graded

2.2 Passwords (hints only)

Table 2.2: Passwords and keys (hints only; no secrets are recorded)

Service / account	**Owner**	**Vault reference or non-revealing hint**	**Privilege / MFA**
Gitea user `brahmi`	Brahmi	Generated once by Gitea when the account was created and saved directly into Brahmi's password manager; never typed into chat, the repository, or this document. Entry name in the password manager: "Gitea admin"	Site administrator and organization owner. MFA: on (reported by Brahmi)
Gitea user `sakshi`	Sakshi	A temporary password was shared outside the chat; Sakshi changed it at first sign-in and keeps the new one herself (where is not recorded here)	Member of team `developers` (Write) and, since October 19, 2026, collaborator with Write on `group1-startup/manual`; not an administrator. MFA: on (confirmed by Sakshi to Brahmi on October 15, 2026)
Server login, key `ssw590-g1-gitea-key`	Brahmi	Private key file in the `.ssh` folder of Brahmi's computer, outside the repository, readable only by her Windows account. Created by AWS; no passphrase. Public half is on the server	Linux user `ec2-user`: `sudo` and the `docker` group (root-equivalent). Key login only. MFA: none
Server login, key `sakshi-ssw590-g1`	Sakshi	Private key stays on Sakshi's computer and was never shared; only the public half is on the server. No passphrase is set on the key (reported by Sakshi)	Same `ec2-user` account and privileges. Key login only. MFA: none

Service / account	Owner	Vault reference or non-revealing hint	Privilege / MFA
AWS IAM user `ssw-590`	Brahmi	Access key kept in the AWS command line configuration on Brahmi's computer, not in the repository or this document	Can create and manage EC2 resources in `us-east-1`; observed to be denied AWS Budgets and public SSM parameters. MFA: not applicable here: the key is used only through the command line, and commands ran without an MFA prompt
AWS account sign-in (console, billing)	Brahmi	Written down offline by Brahmi; the location is not recorded here	Account owner (root sign-in). MFA: on (reported by Brahmi)
Name.com account	Brahmi	Password kept by Brahmi (not in the repository or this document). No payment card is stored (deleted October 15, 2026)	Owner of the domain and its DNS records. MFA: on (reported by Brahmi)
GitHub account `brahmi9811`	Brahmi	Written down offline by Brahmi; the location is not recorded here	Owns the course repository and the Student Pack. MFA: on (reported by Brahmi)

Gitea runner registration token	Brahmi	Not stored anywhere. It is shown in Gitea's Site Administration (Actions, Runners), was used once on October 19, 2026 through the clipboard and an SSH pipe, and the server file that held it was emptied after registration. If lost or leaked, reset it on the same page	Lets whoever holds it register a runner for this Gitea. Rotation: on demand. MFA: not applicable

Changes in this update (October 15, 2026): both tables were created from placeholders. They record the Gitea host, the domain, the earlier assignment's host, the two Gitea users, the two SSH keys, and the AWS, Name.com, and GitHub accounts. The hints and the MFA status were supplied by the account owners on October 15, 2026; MFA status is as reported and was not independently checked.

Chapter 3

Startup Foundation with Gitea

3.1 Overview and Status

This chapter answers the course manual's Chapter 2 [1]: the startup's Gitea work hub with a student domain, HTTPS, minimal public exposure, and persistent data. The eight numbered subsections below match the eight rubric rows (100 points) in the same order. Optional backup and restore evidence is a separate extra-credit subsection and is not part of the eight; it is optional, and the group did not attempt it.

Provider status

The assignment specifies a DigitalOcean Droplet. Group 1 asked the instructor whether AWS could be used instead. On October 12, 2026 (8:36 pm, Canvas message "Gitea Assignment — Question on Cloud Provider (Group 1)") the instructor replied that any hosting service is allowed, including AWS or the group's own servers, **as long as they are available online and can be used by the TA and the instructor to test the assignments.** Group 1 therefore uses AWS. The deployment must stay reachable and testable by the TA and instructor until grading is complete.

Table 3.1: Rubric rows, owners, and status

Subsection	Points	Owner	Status
Student Pack domain and DNS	10	BD	**PASS**
Pinned Gitea deployment	10	BD	**PASS**
Database and persistence	10	SD	**PASS**
HTTPS and certificate renewal	15	BD	**PASS**
Firewall and external exposure	15	BD	**PASS**
Startup team workflow	20	SD	**PASS**

Subsection	Points	Owner	Status
Architecture and operation	5	SD	**PASS**
Manual quality and controls	15	SD	**PASS**
Extra credit: backup and restore (optional)	up to 20	none	Not attempted

3.2 Required Subsections

3.2.1 Student Pack domain and DNS (10 points)

Owner: BD. Domain registration: **PASS** (order confirmed, $0.00). Stored card removed: **PASS**. DNS record, public-resolver lookup, and HTTPS link: **PASS**. A teammate's separate-network lookup was not done (see below).

Offer and eligibility. The domain comes from the GitHub Student Developer Pack, which lists domain offers from Namecheap, Name.com, and .TECH Domains [5]. The group's Student Pack is approved on Brahmi Dalvi's GitHub account. The Namecheap offer was started first but its GitHub authorization page requested the `repo` scope (read and write access to all public and private repositories), which the group judged too broad for a domain claim, so it was cancelled. The Name.com Education Pack authorization requested "public data only" and was accepted.

Domain and costs.

Table 3.2: Domain registration facts

Item	Value
Domain	`ssw590-g1.tech`, registered at Name.com, order #28974415. The confirmation email arrived Thursday, October 15, 2026 at 00:36 (mailbox local time) and gives the order date as October 14, 2026; the one-day difference is consistent with Name.com stamping the date in a different time zone, which we did not confirm
Planned Gitea hostname	`git.ssw590-g1.tech` (A record created, see below)
First-year cost	$0.00 (cart total $0.00; the regular price shown struck through in the cart was $89.99)
Renewal cost	$89.99 ("Renew for $89.99" on the domain page)
Expiration	October 15, 2027
Automatic renewal	OFF (turned off after registration)

Item	Value
Extras declined	Domain Safe Pro ($7.99/year); the free Domain Safe plan (domain lock, WHOIS privacy, free SSL certificate) is included

The order summary after checkout also shows a struck-through $7.99 next to the domain; the group did not investigate that figure and relies on the $0.00 total and the $89.99 renewal price shown on the domain page. Figure 3.1 shows the cart before payment: the domain is free and the paid Domain Safe Pro is declined, so the total is $0.00. Figure 3.2 shows the completed order, Figure 3.3 shows the domain page right after registration (automatic renewal still on, renewal price $89.99), and Figure 3.4 shows the same page after automatic renewal was turned off.

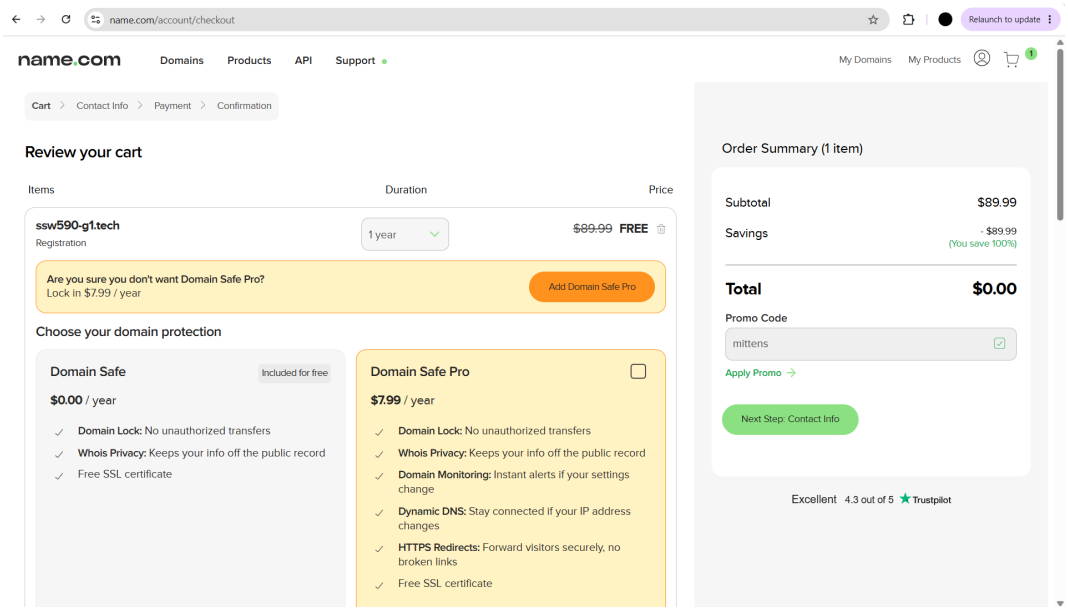


Figure 3.1: Name.com cart before payment, October 15, 2026: `ssw590-g1.tech` free for one year through the partner offer, Domain Safe Pro ($7.99) declined, total $0.00. This is the cart, not a receipt.

Renewal tracking. The group does not plan to renew: the domain is needed only through grading, and the TA and instructor must be able to reach the service until then. Expiry is October 15, 2027. Brahmi Dalvi owns this date. It is recorded in the Hosts table (Table 2.1), and issue #6 in the product repository, assigned to her, tracks adding a calendar reminder; that issue was still open on October 15, 2026, so no calendar event is claimed here.

DNS record. In Name.com's DNS manager the group created one record, `A git.ssw590-g1.tech 52.73.254.36` with TTL 300 (Figure 3.5). The address is the Elastic IP of the EC2 instance described in the deployment subsection. No AAAA record

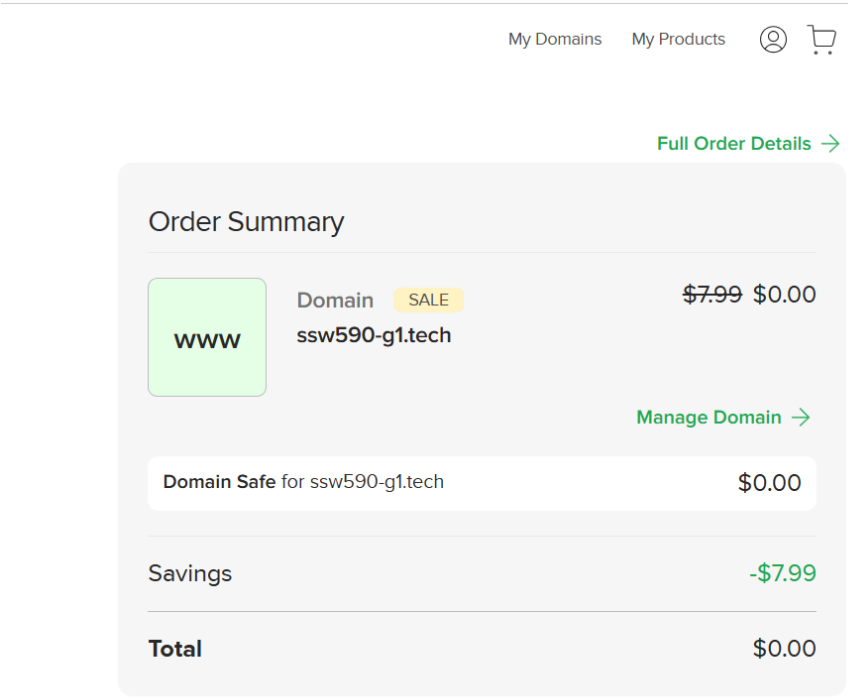


Figure 3.2: Name.com order summary, October 15, 2026: domain and free Domain Safe, total $0.00.

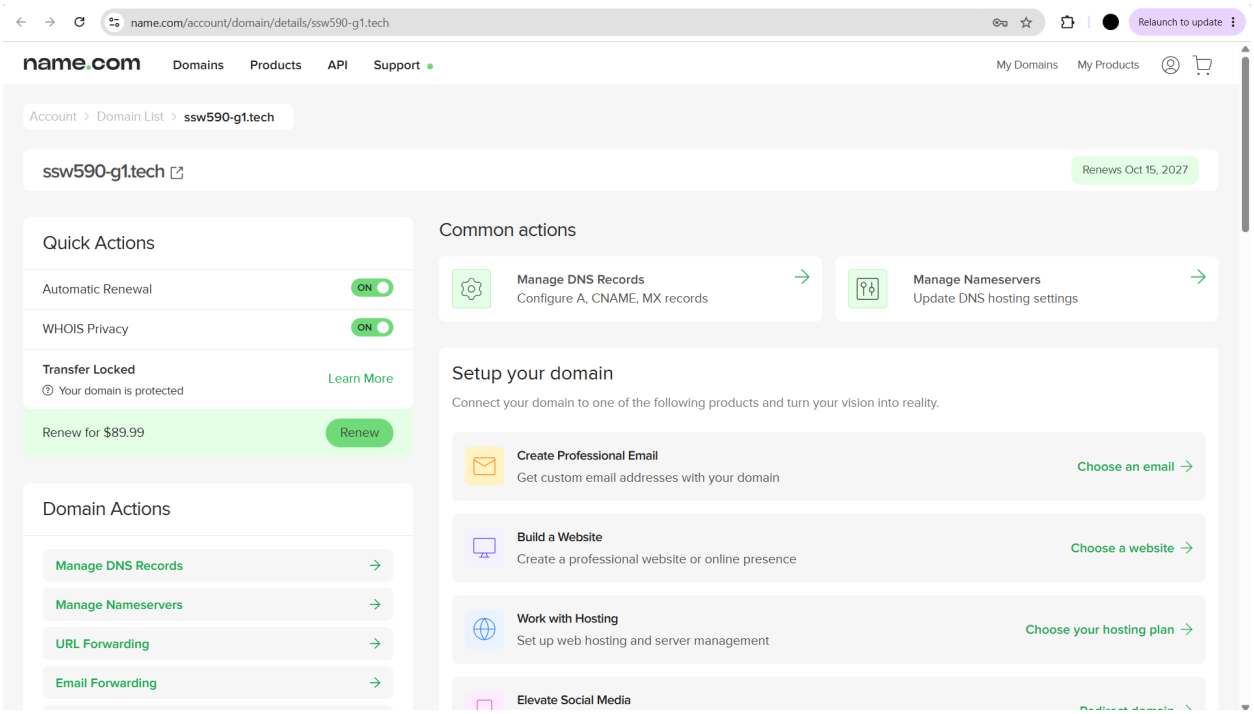


Figure 3.3: Domain page right after registration, October 15, 2026: “Renews Oct 15, 2027”, automatic renewal ON, WHOIS privacy ON, transfer locked, “Renew for $89.99”.

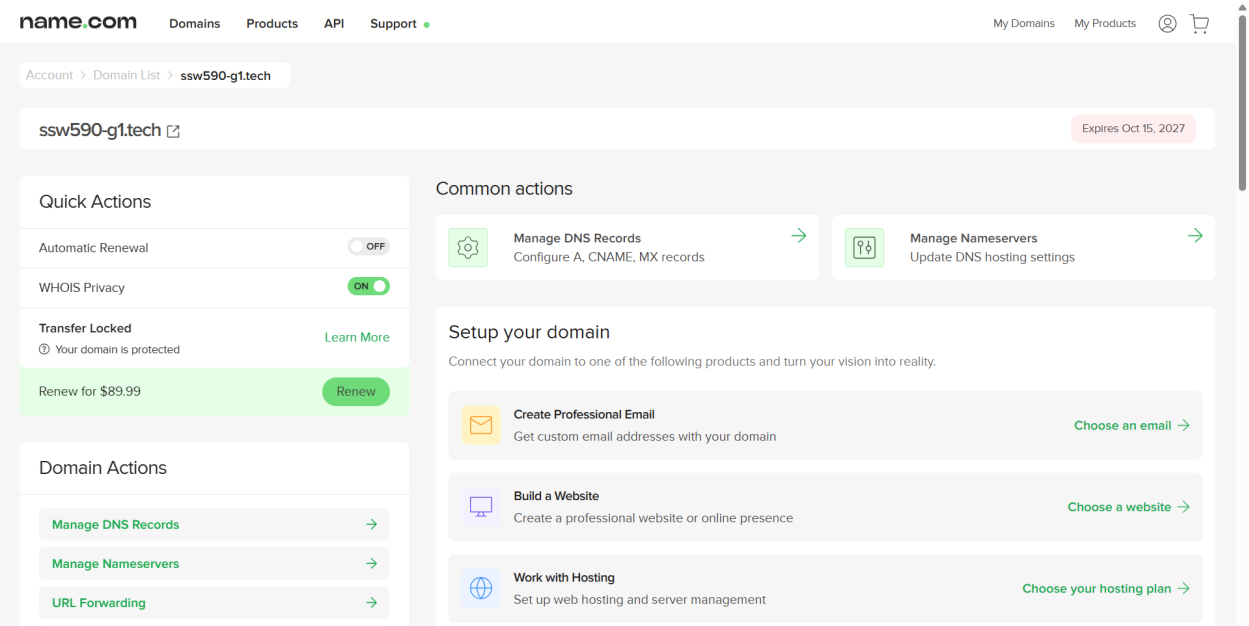


Figure 3.4: Domain page, October 15, 2026: expires October 15, 2027, automatic renewal OFF, renewal price $89.99.

was created, because the deployment is IPv4 only. The domain uses Name.com's own nameservers (`ns1bqx`, `ns2kry`, `ns3jmt`, `ns4ksy .name.com`).

Lookup result. At 17:30:34 UTC on October 15, 2026 the name resolved to the expected address through three public resolvers (expected: `52.73.254.36`; actual: the same).

Resolver	Result for `git.ssw590-g1.tech`	TTL
8.8.8.8 (Google)	A 52.73.254.36	300
1.1.1.1 (Cloudflare)	A 52.73.254.36	300
9.9.9.9 (Quad9)	A 52.73.254.36	300

These lookups were run from Brahmi Dalvi's computer through public resolvers; they were not run by a teammate on a separate network, so the independent-lookup wording of the assignment is met only in that sense. Listing 1 is the real output. A teammate has not repeated the lookup from a separate network; the three public resolvers above are the independent check. The Gitea site is live at the hostname: https://git.ssw590-g1.tech.

```
Real command output, Oct 15, 2026, run from Brahmi's machine (Windows PowerShell) by Claude Code on Brahmi's
↪  go-ahead.
Time of the DNS lookups: 2026-10-15 17:30:34 UTC. The admin's own public IP does not appear in this file.
These lookups were made from the admin's machine through three PUBLIC resolvers. They were not run by a
↪  teammate
on a separate network (the manual's wording for "independent lookup"); a teammate lookup can still be added.

=== Authoritative nameservers for ssw590-g1.tech (queried via 8.8.8.8) ===
ns4ksy.name.com
```

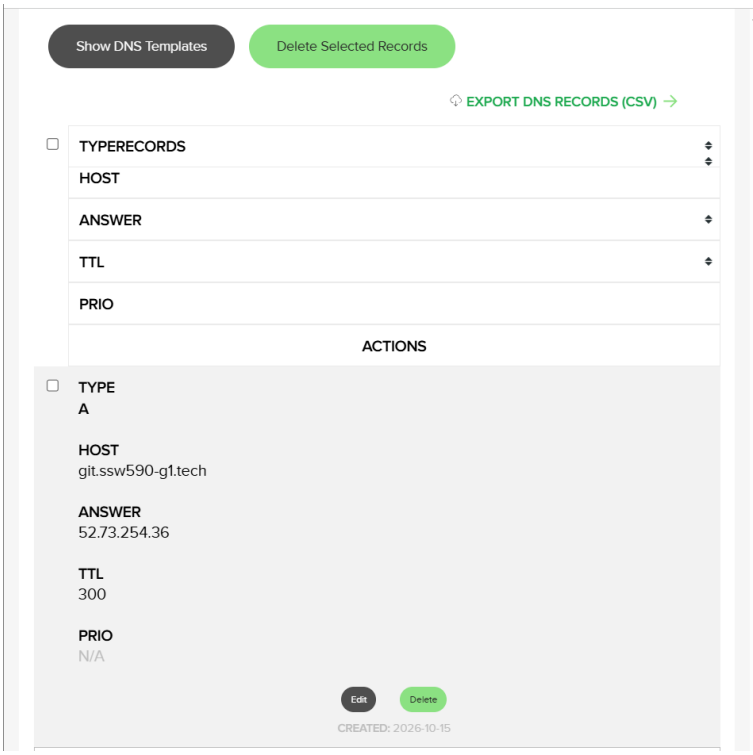


Figure 3.5: Name.com DNS records for `ssw590-g1.tech`, October 15, 2026: one A record for `git`.

```
ns2kry.name.com
ns1bqx.name.com
ns3jmt.name.com

=== A record lookups for git.ssw590-g1.tech ===
via 8.8.8.8 (Google):     git.ssw590-g1.tech  A  TTL 300  52.73.254.36
via 1.1.1.1 (Cloudflare): git.ssw590-g1.tech  A  TTL 300  52.73.254.36
via 9.9.9.9 (Quad9):      git.ssw590-g1.tech  A  TTL 300  52.73.254.36
AAAA via 8.8.8.8: no record returned (expected: IPv4 only by design)

=== SSH host-key verification before the first connection ===
Host key printed in the EC2 console log (ed25519):
↪  AAAAC3NzaC1lZDI1NTE5AAAAIP7bthP0bP6lOQ24+z2zzul/kB+xh4COxobDdf2gLDgY
Host key presented by 52.73.254.36 via ssh-keyscan: same key -> MATCH
(Connected with StrictHostKeyChecking=yes against a known_hosts file holding that verified key; key auth only,
↪  BatchMode.)

=== Read-only SSH check: ssh ec2-user@git.ssw590-g1.tech (connected by hostname) ===
ec2-user
ip-172-31-29-240.ec2.internal
PRETTY_NAME="Amazon Linux 2023.12.20261014"
6.18.54-121.166.amzn2023.x86_64
Docker version 25.0.14, build 0bab007
Docker Compose version v5.6.0
git version 2.50.1
active                     (systemctl is-active docker)
               total        used        free      shared  buff/cache   available
Mem:             913         259          77           0         575         492
NAME      TYPE SIZE USED PRIO
/swapfile file   2G 352K   -2
/dev/nvme0n1p1   20G  4.1G   16G  21% /
ec2-user adm wheel systemd-journal docker
exit code: 0

Notes:
- Proves: the A record is live on public resolvers, SSH by hostname works from the admin IP, the instance
↪  matches the planned setup.
- Does NOT prove: external exposure (ports 80/443 closed/open from other networks), HTTPS, or anything about
↪  Gitea (not deployed yet).
```

Listing 1: DNS lookups through three public resolvers, SSH host-key check, and a read-only SSH check of the server, October 15, 2026 (real output; the administrator's address is not part of it).

Payment safeguards. Name.com required a card even for the $0.00 order. To avoid an unintended renewal charge, the group turned automatic renewal off and deleted the stored payment profile; the billing page then reads "No saved payment profiles" (Figure 3.6). The confirmation email shows order #28974415 with an order total of $0.00 (Figure 3.7). The group has not yet compared a card statement; because the order total was $0.00 and the card was removed, no charge is expected.

3.2.2 Pinned Gitea deployment (10 points)

Owner: BD. Deployed and checked on October 15, 2026: **PASS**. Cost verification and the repository link are still open (see the end of this subsection).

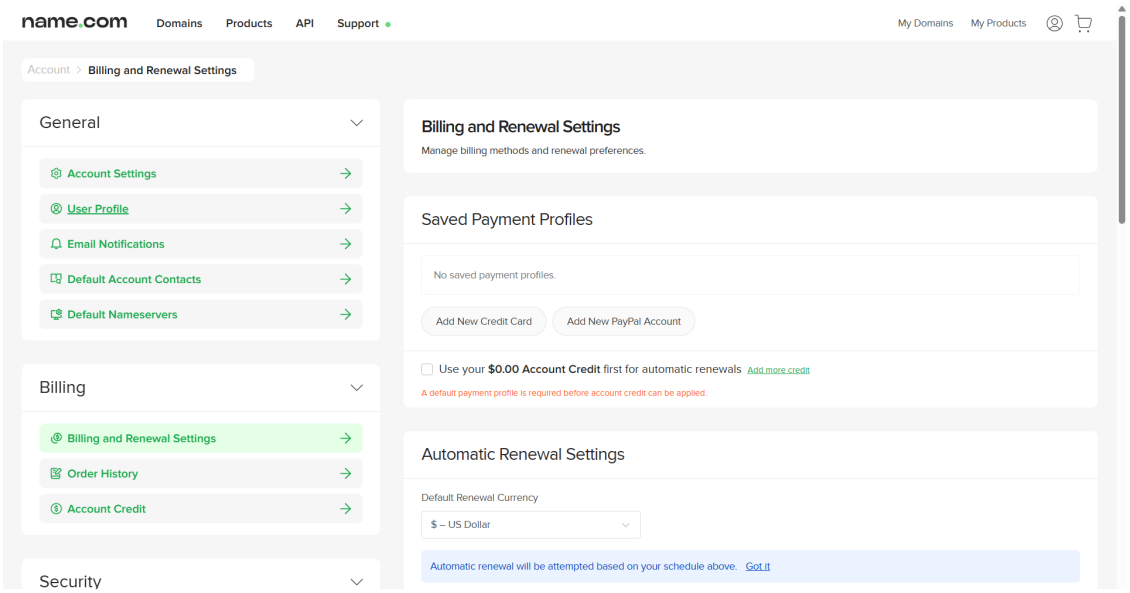


Figure 3.6: Name.com billing page, October 15, 2026: “No saved payment profiles” after the card was deleted.

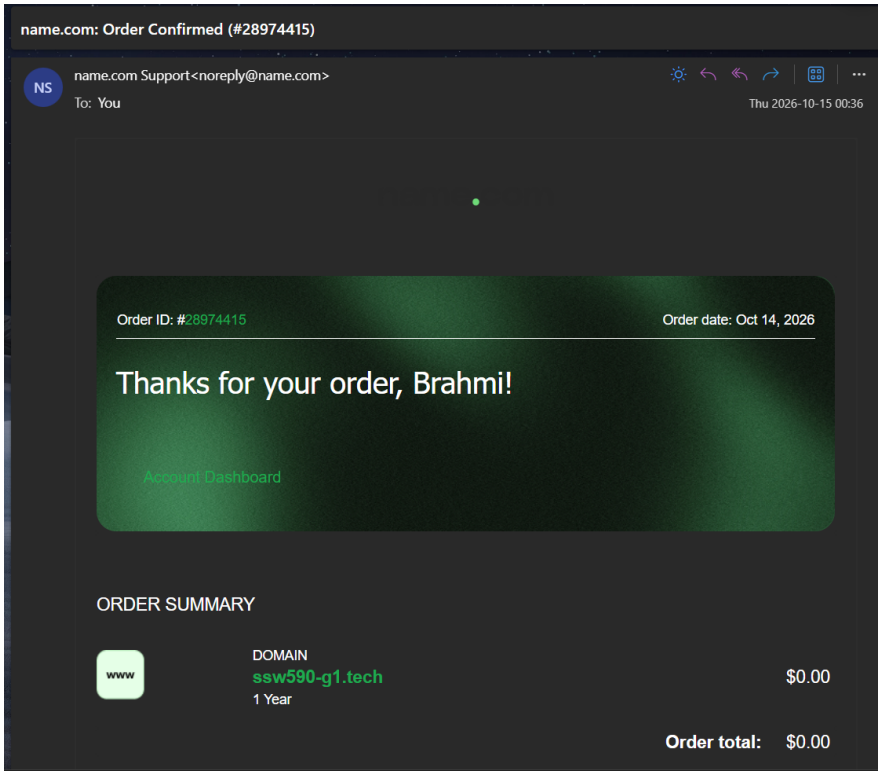


Figure 3.7: Name.com order confirmation email, received Thursday October 15, 2026 at 00:36 (mailbox time): order 28974415, `ssw590-g1.tech`, one year, order total $0.00. The email gives the order date as October 14, 2026.

Provider and versions. Gitea runs on one AWS EC2 instance instead of a DigitalOcean Droplet; the instructor approved this substitution (see the provider note at the start of this chapter). Table 3.3 records what was actually created. The instance was created with the AWS command line interface, not the web console, so every setting is repeatable.

Table 3.3: Deployed components and versions (October 15, 2026)

Item	Value
Cloud and region	AWS EC2, `us-east-1`, availability zone `us-east-1c`
Instance	`i-0c68ec1bc1e16a414`, type `t3.micro` (2 vCPU, 1 GiB RAM), Name tag `ssw590-g1-gitea`
Operating system	Amazon Linux 2023 (`al2023-ami-2023.12.20261014.0-kernel-6.18-x86_64`), image `ami-0ed59cbb87dfa8dba`, kernel `6.18.54-121.166.amzn2023.x86_64`
Disk and memory	20 GB gp3 root volume (deleted with the instance); 2 GB swap file added at first boot
Instance metadata	IMDSv2 only (`HttpTokens=required`)
Public address	Elastic IP `52.73.254.36`, named `git.ssw590-g1.tech` in DNS
Docker Engine	25.0.14 (Amazon Linux repository)
Docker Compose	v5.6.0 plugin, installed from the project's GitHub release and verified against a pinned SHA-256 (`40343e21...e76a`)
git	2.50.1
Gitea image	`gitea/gitea:1.27.3` from Docker Hub, image ID `79814cfc668d`, 191 MB [3]
Proxy image	`caddy:2.11.7` from Docker Hub, image ID `f77f856a30f0`, 66.3 MB
SSH key pair	`ssw590-g1-gitea-key` (ed25519); the private key stays on the administrator's computer, outside the repository

Both image tags were confirmed to exist on Docker Hub on October 12, 2026. `1.27.3` was the newest 1.27 release that day; a 28.1 release existed but was six days old and was not chosen. No image uses the `latest` tag.

```
AWS CLI output and EC2 console log excerpts, Oct 15, 2026 (UTC times from the console). Region us-east-1.
Commands were run by Claude Code on Brahmi's explicit go-ahead from Brahmi's machine (PowerShell, aws-cli
↪  2.37.9).
No credentials or private keys appear below. The admin's public IP is not part of this file.

=== Launch (aws ec2 run-instances, summarized) ===
image:         ami-0ed59cbb87dfa8dba  (Amazon Linux 2023, al2023-ami-2023.12.20261014.0-kernel-6.18-x86_64)
type:          t3.micro (2 vCPU, 1 GiB RAM)
disk:          20 GB gp3 root volume, DeleteOnTermination=true
key pair:      ssw590-g1-gitea-key (ed25519; private key kept on the admin's machine, outside the repo)
security grp:  sg-0415c75429368eb85 (ssw590-g1-gitea)
IMDS:          HttpTokens=required (IMDSv2 only)
user-data:     assignment-4/gitea/aws/ec2-user-data.sh (swap, Docker, git, pinned Compose)
tags:          Name=ssw590-g1-gitea, Project=ssw590-group1-assignment4
```

```
Instance: i-0c68ec1bc1e16a414
Elastic IP allocation: eipalloc-0dee5296b5382f367  (association eipassoc-0ea58688cb7153571)

=== Summary (describe-instances / describe-addresses) ===
i-0c68ec1bc1e16a414  running  t3.micro  ami-0ed59cbb87dfa8dba  us-east-1c  52.73.254.36  sg-0415c75429368eb85
↪  ssw590-g1-gitea-key  IMDS tokens: required
52.73.254.36  eipalloc-0dee5296b5382f367  i-0c68ec1bc1e16a414

=== Status checks (describe-instance-status) ===
running  instance-status: ok  system-status: ok

=== Boot script result (EC2 console output, snapshot 2026-10-15T17:22:54Z) ===
Oct 15 17:17:37 user-data: Complete!
Oct 15 17:17:40 user-data: Docker version 25.0.14, build 0bab007
Oct 15 17:17:40 user-data: Docker Compose version v5.6.0
Oct 15 17:17:40 user-data: git version 2.50.1
Oct 15 17:17:40 user-data: Mem:             913         254         121           0         537         498
Oct 15 17:17:40 user-data: Swap:           2047          16        2031
Oct 15 17:17:40 user-data: user-data finished OK
cloud-init v. 22.2.2 finished at Thu, 15 Oct 2026 17:17:40 +0000 (up 89.67 seconds)

Notes:
- Compose binary checksum: the script runs "sha256sum -c" with set -e; the script reached "finished OK", so
↪  the check passed.
  Expected SHA-256 40343e21ca777173e69cff5dbafeb37c6f81f3b0d57d9e597f036e95eb63e76a (GitHub asset digest and
  ↪  .sha256 file agreed).
- Gitea is NOT deployed yet; DNS is not created yet; no external exposure test has been run yet.
- Problems met: the AWS CLI on Windows crashed printing a non-ASCII character from the console log (charmap
↪  codec);
  an earlier poll looked like "still waiting" because that error was suppressed. Fixed by forcing UTF-8
  ↪  (PYTHONIOENCODING/PYTHONUTF8).
- The IAM user could not read the public SSM AMI parameter (AccessDenied); the AMI was found with
↪  describe-images instead.
```

Listing 2: The instance as created and the result of its first-boot script, October 15, 2026 (real AWS command line and console-log output).

How it is configured.

- **Restart behavior:** both containers use `restart: unless-stopped`, so they return after a reboot or a Docker restart.
- **Persistent data:** the named volume `gitea_gitea_data` holds Gitea's repositories and SQLite database at `/data`; `gitea_caddy_data` keeps Caddy's certificates and ACME account; `gitea_caddy_config` holds Caddy's saved configuration. The `Caddyfile` is mounted read-only from the project directory.
- **Proxy connection:** Caddy and Gitea share the private Compose network `gitea_web`; Caddy reaches Gitea at `gitea:3000` by service name. Gitea has no `ports:` entry, so it is not published on the host; only Caddy publishes 80 and 443.
- **Non-secret configuration (environment variables):** SQLite database at `/data/gitea/gitea.db`; `DOMAIN` and `ROOT_URL` set to the HTTPS hostname; Gitea's own SSH server disabled (Git over HTTPS only); web installer skipped (`INSTALL_LOCK`); self-registration disabled.
- **Secrets:** none appear in the Compose file, the `.env` file (hostname and image versions only), or this document. The first administrator password was generated once in the administrator's own terminal and stored in a password manager.

Compose and proxy configuration. The files below are exactly what runs on the server (their SHA-256 prefixes were compared after copying, Listing 3). They contain no secrets, so no redaction was needed. The first comment line of the Compose file still says it is a draft that has not been run; it was written before the first deployment and was left unchanged so that the file shown is exactly the one on the server. They are versioned in the team's course repository on GitHub, which was made public on October 15, 2026 so that graders can open it: https://github.com/brahmi9811/ssw-590-group1/tree/main/assignment-4/gitea. A second copy of the same files is published in the team's own Gitea at https://git.ssw590-g1.tech/group1-startup/infra. It was created on October 15, 2026 and is public; a signed-out check of Gitea's API listed the four files (`docker-compose.yml`, `Caddyfile`, `.env.example`, and `aws/ec2-user-data.sh`) plus a README, and their checksums match the files shown below.

```
# Gitea (SQLite) behind a Caddy TLS reverse proxy. Cloud-agnostic: works on a
# DigitalOcean Droplet or an AWS EC2 instance. DRAFT: not yet run on a server.
# Copy .env.example to .env (untracked) and set the values before `docker compose up -d`.
services:
  gitea:
    image: gitea/gitea:${GITEA_VERSION}          # pinned in .env
    container_name: gitea
    restart: unless-stopped
    environment:
      USER_UID: "1000"
      USER_GID: "1000"
      GITEA__database__DB_TYPE: sqlite3           # FINAL team decision (Oct 12, 2026):
      ↪  SQLite
      GITEA__database__PATH: /data/gitea/gitea.db # inside the persistent /data volume
      GITEA__server__DOMAIN: ${GITEA_DOMAIN}
      GITEA__server__ROOT_URL: https://${GITEA_DOMAIN}/
      GITEA__server__HTTP_PORT: "3000"
      GITEA__server__DISABLE_SSH: "true"          # HTTPS-only Git; no SSH listener
      GITEA__security__INSTALL_LOCK: "true"       # skip web installer; create admin via
      ↪  CLI
      GITEA__service__DISABLE_REGISTRATION: "true"
    volumes:
      - gitea_data:/data                          # repos, attachments, SQLite file
    networks:
      - web
    # No `ports:` on purpose: Gitea (3000) is reachable only from the proxy.
    healthcheck:
      test: ["CMD", "curl", "-fsS", "http://localhost:3000/api/healthz"]
      interval: 30s
      timeout: 5s
      retries: 5

  caddy:
    image: caddy:${CADDY_VERSION}                 # pinned in .env
    container_name: caddy
    restart: unless-stopped
    depends_on:
      - gitea
```

```
    ports:
      - "80:80"     # HTTP-to-HTTPS redirect and ACME HTTP-01 validation only
      - "443:443"
    environment:
      GITEA_DOMAIN: ${GITEA_DOMAIN}
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data                          # certificates + ACME account: persist!
      - caddy_config:/config
    networks:
      - web

networks:
  web:

volumes:
  gitea_data:
  caddy_data:
  caddy_config:
```

```
# Caddy obtains and renews a Let's Encrypt certificate for this hostname and
# redirects HTTP to HTTPS automatically. Gitea is reached by Compose service name.
{$GITEA_DOMAIN} {
        reverse_proxy gitea:3000
}
```

Steps another team could repeat. These are the commands as they were run (abridged to the settings that matter). Placeholders in angle brackets were filled with the values in Table 3.3.

```
# 1. Firewall: security group with 443 and 80 open, 22 limited to the admin /32
aws ec2 create-security-group --group-name ssw590-g1-gitea \
  --description "SSW590 Group1 Gitea: 443/80 public, 22 admin only" --vpc-id <default
  ↪  VPC>
aws ec2 authorize-security-group-ingress --group-id <sg> --ip-permissions <443, 80, 22
↪  rules>
# 2. SSH key pair; the private key is saved to a file outside the repository
aws ec2 create-key-pair --key-name ssw590-g1-gitea-key --key-type ed25519 \
  --query KeyMaterial --output text
# 3. Instance with the versioned first-boot script (swap, Docker, git, pinned Compose)
aws ec2 run-instances --image-id ami-0ed59cbb87dfa8dba --instance-type t3.micro \
  --key-name ssw590-g1-gitea-key --security-group-ids <sg> \
  --block-device-mappings DeviceName=/dev/xvda,Ebs={VolumeSize=20,VolumeType=gp3} \
  --metadata-options HttpTokens=required \
  --user-data file://assignment-4/gitea/aws/ec2-user-data.sh
# 4. Fixed public address
aws ec2 allocate-address --domain vpc
aws ec2 associate-address --instance-id <instance> --allocation-id <allocation>
# 5. DNS (Name.com web console): A record  git  ->  <Elastic IP>, TTL 300
```

```
# 6. Copy the three non-secret files, create .env, and start the stack
scp docker-compose.yml Caddyfile .env.example ec2-user@git.ssw590-g1.tech:~/gitea/
sed 's/^GITEA_DOMAIN=.*/GITEA_DOMAIN=git.ssw590-g1.tech/' .env.example > .env
chmod 600 .env
docker compose config -q && docker compose pull && docker compose up -d
# 7. First administrator, run interactively by the administrator (password printed once)
docker compose exec -u git gitea gitea admin user create --admin \
  --username brahmi --email <email> --random-password
```

Expected versus actual.

Table 3.4: Deployment checks, October 15, 2026

Check	Expected	Actual	Evidence
Instance state	running, both status checks ok	running; instance and system checks ok	Listing 2
First-boot script	finishes without error, Compose checksum matches	"user-data finished OK" at 17:17:40 UTC	Listing 2
Installed tools	Docker, Compose, git, swap	Docker 25.0.14, Compose v5.6.0, git 2.50.1, 2 GB swap	Listing 2, Listing 1
Files on the server	identical to the repository copies	SHA-256 prefixes match for all three files	Listing 3
Containers	both up, Gitea healthy	`gitea` Up (healthy), `caddy` Up	Listing 3
Gitea settings	HTTPS `ROOT_URL`, SQLite	AppURL `https://git.ssw590-g1.tech/`; database ping `sqlite3`	Listing 3
Site and login	page loads, admin can sign in	HTTP 200 over HTTPS; signed in as the administrator	Listing 5, Figure 3.8

```
Real output from the server (ssh ec2-user@git.ssw590-g1.tech), Oct 15, 2026 (UTC). Run by Claude Code on
↪  Brahmi's go-ahead.
SANITIZED: log lines trimmed to the relevant ones; remote client IPs in the Caddy request/ACME lines are
↪  omitted. No secrets involved:
the server .env holds only non-secret values and no Gitea admin exists yet.

=== Files copied to ~/gitea on the server (scp), SHA-256 prefix identical to the repo copies ===
ed23b4136370d08a  docker-compose.yml
7705e125f84a2c82  Caddyfile
f42b0bd6bee72df6  .env.example
(source in the repo: assignment-4/gitea/)

=== ~/gitea/.env on the server (created from .env.example, chmod 600; no secrets) ===
GITEA_VERSION=1.27.3
CADDY_VERSION=2.11.7
```

```
GITEA_DOMAIN=git.ssw590-g1.tech

=== docker compose config -q  ->  "compose config OK" ===

=== docker compose pull (pinned images) ===
caddy:2.11.7        f77f856a30f0   66.3MB
gitea/gitea:1.27.3  79814cfc668d   191MB

=== docker compose up -d  (start: 2026-10-15 17:34:23 UTC) ===
Volume gitea_caddy_config Created, Network gitea_web Created
Container gitea Created/Started; Container caddy Created/Started
(volumes gitea_gitea_data, gitea_caddy_data, gitea_caddy_config; project directory name "gitea")

=== docker compose ps (about 40 s after start) ===
caddy  caddy:2.11.7          Up 40 seconds             0.0.0.0:80->80/tcp, [::]:80->80/tcp,
↪  0.0.0.0:443->443/tcp, [::]:443->443/tcp, 443/udp, 2019/tcp
gitea  gitea/gitea:1.27.3    Up 40 seconds (healthy)   22/tcp, 3000/tcp          (not published to the host)

=== Caddy log excerpts (JSON, trimmed) ===
acme_client: trying to solve challenge, identifier git.ssw590-g1.tech, challenge_type tls-alpn-01, ca
↪  https://acme-v02.api.letsencrypt.org/directory
tls: served key authentication certificate (tls-alpn-01) x5   (Let's Encrypt validation from several vantage
↪  points)
acme_client: authorization finalized, authz_status valid
acme_client: validations succeeded; finalizing order
acme_client: got renewal info (ACME Renewal Information), window_start 1797189600, window_end 1797345050
tls.obtain: certificate obtained successfully, issuer acme-v02.api.letsencrypt.org-directory
http.log.error: dial tcp 172.18.0.2:3000: connect: connection refused ... status 502
   -> one early 502: an internet scanner hit the site about a second before Gitea was listening. Transient;
   ↪  Gitea was up moments later.

=== Gitea log excerpts ===
cmd/web.go: Listen: http://0.0.0.0:3000
cmd/web.go: AppURL(ROOT_URL): https://git.ssw590-g1.tech/
HTTPRequest: GET / -> 200 OK (via the proxy)
HTTPRequest: GET /wp-json/ -> 404, GET /xmlrpc.php?rsd -> 404   (automated scanners probing the new site
↪  within seconds)
PING DATABASE sqlite3
HTTPRequest: GET /api/healthz -> 200 OK   (container healthcheck)

Notes:
- Certificate challenge used: TLS-ALPN-01 (port 443). Port 80 is still open for the HTTP-to-HTTPS redirect.
- Caddy's published ports show IPv6 bindings ([::]) on the host, but the security group has no IPv6 rules and
↪  no AAAA record exists, so IPv6 is not reachable.
- Port 2019/tcp in "ps" is Caddy's admin API inside the container; it is not published to the host.
- Gitea port 3000 and 22 are only exposed inside the Compose network.
- At the time of this capture (about 17:35 UTC) not yet done: admin account, persistence test, renewal test,
↪  external test from a second network. All were done later the same day (see the later listings).
```

Listing 3: Files copied to the server, image pull, `docker compose up -d`, container status, and the Caddy and Gitea logs, October 15, 2026 (real server output, trimmed, remote addresses removed).

Problems met and corrections.

- The AWS command line tool on Windows crashed while printing a non-ASCII character from the instance console log. An early check looked like “still waiting” because that error was hidden; the log was read again with UTF-8 forced and showed the script had finished.
- The IAM user was not allowed to read the public image-lookup parameter (access denied), so the Amazon Linux image was found with `describe-images` instead.

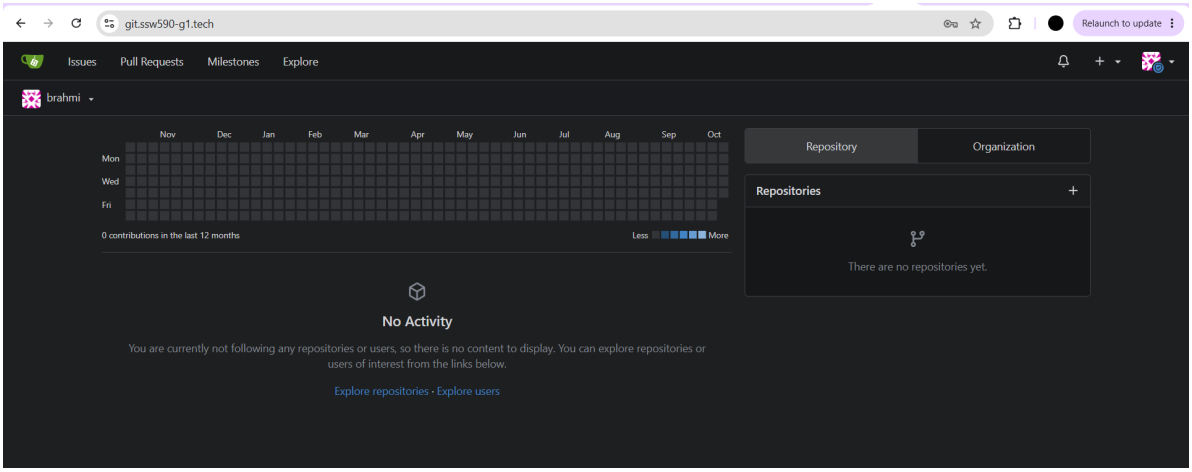


Figure 3.8: The deployed Gitea at `git.ssw590-g1.tech`, signed in as the administrator, October 15, 2026.

- Passing a multi-line script directly to `ssh` from Windows PowerShell 5.1 stripped quotes and failed with a shell syntax error; scripts are now copied to the server and run there.
- One 502 error appeared in the Caddy log about a second after start: an internet scanner reached the site before Gitea was listening. It did not recur.

Cost and cleanup. The running costs are the instance, the 20 GB volume, and the Elastic IP. The group has not confirmed the exact prices on AWS's own pricing pages. An estimate at on-demand list prices in `us-east-1`, for about 730 hours a month: the `t3.micro` instance at $0.0104 per hour is about $7.59; the public IPv4 address at $0.005 per hour is about $3.65 [6]; the 20 GB gp3 volume at $0.08 per GB-month is about $1.60 [7]. The total is **about $12.84 per month**. The instance rate comes from a third-party price calculator [8], and the gp3 rate from AWS blog and third-party guides; neither was confirmed on AWS's own pricing pages, so the figure is an estimate, not a bill. The group uses AWS credits and may qualify for free-tier hours (not checked), so the real charge may be lower. The account has a monthly cost budget of $15, created for the earlier assignment; its alert thresholds were not reviewed. The earlier assignment's instance, which keeps running until that assignment is graded, is a separate cost. The service must stay online until grading is finished because the instructor and TA test it. Teardown, after grading and only then: terminate the instance, release the Elastic IP, delete the security group and key pair, delete the DNS record, and let the domain expire (automatic renewal is already off, see the domain subsection).

3.2.3 Database and persistence (10 points)

Owner: SD. **Status:** persistence test on the cloud VM **PASS** (October 15, 2026, 18:29 UTC).

Decision: SQLite

Group 1 uses Gitea's built-in SQLite database [9]. This was a team decision recorded on October 12, 2026. Reasons:

- The startup has two members; Gitea's write load is a few pushes, issues, and reviews per day, well within SQLite's capability.
- No second container, database port, or database credential exists, so there is nothing extra to secure, patch, or expose.
- The whole service state (database file and repositories) lives in one volume, which keeps a future backup simple.

Limitation. SQLite serialises writes in a single file. A larger team with many concurrent pushes, or a deployment that runs several Gitea instances behind a load balancer, should move to PostgreSQL in its own container [1].

Settings

The database is configured with environment variables in the versioned Compose file (`gitea/docker-compose.yml`), which Gitea writes into `/data/gitea/conf/app.ini` at start-up [10]:

```yaml
services:
  gitea:
    environment:
      GITEA__database__DB_TYPE: sqlite3
      GITEA__database__PATH: /data/gitea/gitea.db
    volumes:
      - gitea_data:/data        # named volume, outside the container layer
    # no ports: Gitea is reached only by Caddy on the private network
volumes:
  gitea_data:
```

Table 3.5: Where Gitea's persistent data lives

Item	Location / value
Database type	`sqlite3` (file, no server)
Database file	`/data/gitea/gitea.db`
Git repositories	`/data/git/repositories/<owner>/<repo>.git`
Attachments, avatars, indexes	`/data/gitea/attachments`, `avatars`, `indexers`
Generated configuration	`/data/gitea/conf/app.ini` (contains generated secrets; never copied into this manual)
Docker volume	`gitea_gitea_data` (Compose project `gitea`) mounted at `/data`
Database network port	none: SQLite is a file, so there is no port to publish or firewall
Database credential	none: SQLite has no user or password

Because the volume is mounted from Docker's volume store, it is *outside* the container's disposable writable layer: removing or recreating the container does not touch it. Only an explicit `docker compose down -v` or `docker volume rm` deletes it, and both are forbidden in the team's runbook (Section 3.2.7).

Persistence test procedure

The test is scripted in `assignment-4/sd/persistence-test.sh` so it can be repeated exactly. Run on the Docker host from `assignment-4/gitea`, it:

1. creates repository `persistence-check` (with an initial README commit) and issue #1 through the Gitea API, unless they already exist;
2. records the container ID and creation time, the volume mount, the database file, and the repository, issue, and `main` commit IDs;
3. replaces *only* the Gitea container, keeping volumes:

```
docker compose up -d --force-recreate gitea
```

4. waits for the health check, then records the same items again.

The password is read from the terminal and never printed.

Evidence

Local rehearsal, October 12, 2026: not graded evidence

The procedure was rehearsed on SD's workstation (Docker Desktop, Gitea 1.27.3, Caddy 2.11.7, `https://localhost`). The container ID changed from `fdca1b83aa58` to `c1e3fb1ba256`; repository id 1, issue #1 (open) and commit `19b82109db90` were unchanged afterwards. This only shows the procedure works; the graded result is the run on the cloud VM below.

Table 3.6: Persistence test on the cloud VM

Check	Expected	Actual (VM)
Container ID before / after	different	`858a02ae8412` (created 17:34:23) → `834924d4bc0a` (created 18:29:21) **PASS**
Volume mounted at `/data`	`gitea_gitea_data` both times	`gitea_gitea_data` before and after **PASS**
Database file	same file, still present	`/data/gitea/gitea.db`, 2 187 264 bytes before and after **PASS**
Repository `persistence-check`	same id before and after	id 1, `sakshi/persistence-check`, created 18:29:20 both times **PASS**
Issue #1	present, same id and state	id 1, open, same title **PASS**

Check	Expected	Actual (VM)
`main` commit	same commit ID	`4f15f700003e` both times **PASS**

The test was run by SD on October 15, 2026 from her workstation over SSH, as `ec2-user` on the EC2 instance, from the Compose directory `~/gitea`:

```
ssh -t -i ~/.ssh/ssw590-g1-sakshi ec2-user@git.ssw590-g1.tech \
  'cd ~/gitea && GITEA_URL=https://git.ssw590-g1.tech GITEA_USER=sakshi \
   bash ~/persistence-test.sh | tee ~/persistence.log'
```

Listing 4 is the complete, unedited log. It contains no credential: the password was typed at a hidden prompt. The `host:` line is the instance's AWS-internal name, which is not reachable from the internet.

```
=== Gitea persistence test, started 2026-10-15T18:29:19Z ===
host: ip-172-31-29-240.ec2.internal  url: https://git.ssw590-g1.tech  user: sakshi
$ POST https://git.ssw590-g1.tech/api/v1/user/repos  (name=persistence-check, auto_init=true)
$ POST https://git.ssw590-g1.tech/api/v1/repos/sakshi/persistence-check/issues
--- BEFORE container replacement (2026-10-15T18:29:20Z) ---
$ docker inspect gitea --format {{.Id}} created={{.Created}}
858a02ae8412 created=2026-10-15T17:34:23.469902499Z
$ docker inspect gitea --format (mounts: type name -> destination)
volume gitea_gitea_data -> /data

$ docker compose exec gitea ls -l /data/gitea/gitea.db
-rw-r--r--    1 git      git        2187264 Oct 15 18:29 /data/gitea/gitea.db
$ GET https://git.ssw590-g1.tech/api/v1/repos/sakshi/persistence-check
repo id=1 full_name=sakshi/persistence-check created=2026-10-15T18:29:20Z
$ GET https://git.ssw590-g1.tech/api/v1/repos/sakshi/persistence-check/issues/1
issue #1 id=1 state=open title='Persistence check: this issue must survive container replacement'
$ GET https://git.ssw590-g1.tech/api/v1/repos/sakshi/persistence-check/branches/main
branch main commit=4f15f700003e
--- replacing container (2026-10-15T18:29:21Z) ---
$ docker compose up -d --force-recreate gitea
 Container gitea Recreate
 Container gitea Recreated
 Container gitea Starting
 Container gitea Started
--- AFTER container replacement (2026-10-15T18:29:52Z) ---
$ docker inspect gitea --format {{.Id}} created={{.Created}}
834924d4bc0a created=2026-10-15T18:29:21.291713393Z
$ docker inspect gitea --format (mounts: type name -> destination)
volume gitea_gitea_data -> /data

$ docker compose exec gitea ls -l /data/gitea/gitea.db
-rw-r--r--    1 git      git        2187264 Oct 15 18:29 /data/gitea/gitea.db
$ GET https://git.ssw590-g1.tech/api/v1/repos/sakshi/persistence-check
repo id=1 full_name=sakshi/persistence-check created=2026-10-15T18:29:20Z
$ GET https://git.ssw590-g1.tech/api/v1/repos/sakshi/persistence-check/issues/1
issue #1 id=1 state=open title='Persistence check: this issue must survive container replacement'
$ GET https://git.ssw590-g1.tech/api/v1/repos/sakshi/persistence-check/branches/main
branch main commit=4f15f700003e
=== finished 2026-10-15T18:29:52Z ===
```

Listing 4: Persistence test on the EC2 instance, October 15, 2026, 18:29:19–18:29:52 UTC (real, unedited output of `persistence-test.sh`).

Figure 3.9 and Figure 3.10 show the repository and the issue in the browser on October 15, 2026, a few minutes after the test finished (Gitea displays relative times such as "2 minutes ago"; the absolute times are in Listing 4). The commit shown, `4f15f70000`, is the `main` commit recorded before and after the replacement.

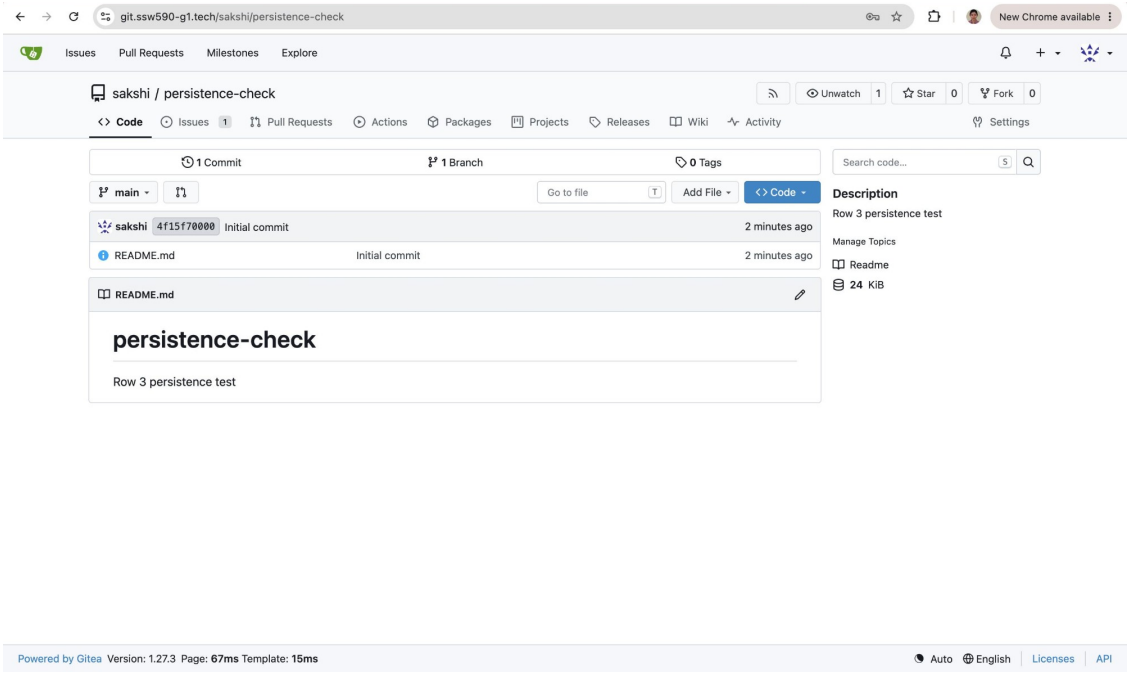


Figure 3.9: Repository `sakshi/persistence-check` after the container replacement, October 15, 2026: one commit, `4f15f70000`.

Conclusion

The Gitea container was replaced (new container ID and creation time) while the named volume was kept, and the repository, its `main` commit, and issue #1 were unchanged afterwards. They persisted because SQLite and the Git repositories live in the `gitea_gitea_data` volume, outside the container's disposable layer; the new container mounted the same volume. Gitea was unavailable for about 30 seconds during the replacement. This test shows that data survives replacement of the container. It is *not* a backup or restore test: no copy of the data exists outside the volume, so losing the volume or the VM disk would still lose the data (see Section 3.2.7).

3.2.4 HTTPS and certificate renewal (15 points)

Owner: BD. Certificate issued, redirect and TLS checked on October 15, 2026: **PASS**. Second-option research is partly done (open questions remain). The staging dry run passed as an issuance test; an actual renewal of the production certificate has not yet happened, so it is **PENDING**.

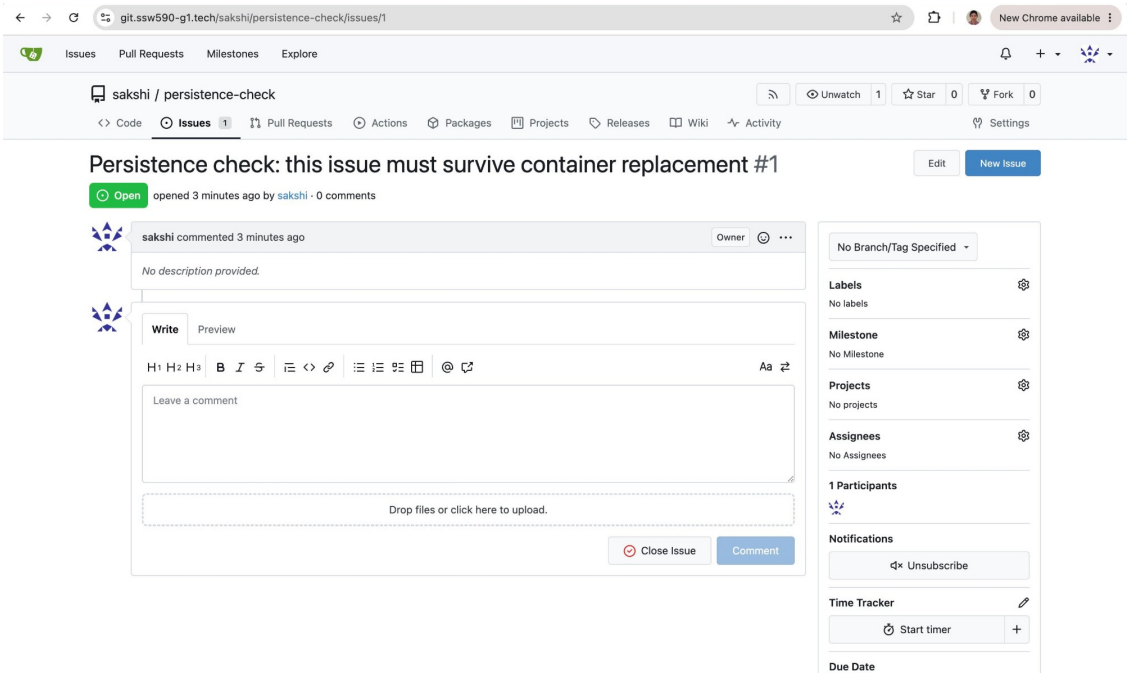


Figure 3.10: Issue #1 still open after the container replacement, October 15, 2026.

Certificate options compared. Two ways to get a certificate for `git.ssw590-g1.tech` were considered.

Table 3.7: Certificate options

Aspect	**A. Let's Encrypt through Caddy**	**B. Free certificate offered with the Student Pack or domain bundle**
Cost	Free [4]	Included free with Name.com's Domain Safe tier (visible in the cart); the Student Pack page also lists a free one-year Namecheap certificate [11, 5]
Issuance	Automatic: Caddy proves control of the name to Let's Encrypt with an ACME challenge (TLS-ALPN-01 was used) [12]	A domain-validated certificate from Name.com's free-certificate program. The support pages we could read do not say how it is requested, or whether it can be installed on a server hosted elsewhere [11]

Aspect	A. Let's Encrypt through Caddy	B. Free certificate offered with the Student Pack or domain bundle
Renewal	Automatic inside Caddy; certificates live in a persistent volume	Manual: in the Name.com account (My Products, then Renew; the checkout shows $0), and only within 90 days of expiry [13]. A missed window removes the renewal option
Fit for this project	Fits Docker Compose: no extra tool, no manual step, nothing to remember in a year	Not confirmed to work on our AWS-hosted server; adds a yearly manual step and an account dependency. For the Namecheap certificate, one student reports being asked to pay after claiming the free domain [14]

Option B's facts come from Name.com's support pages as summarized by a search engine, because the article bodies could not be retrieved in full; they have not been confirmed with the vendors. Whether the Domain Safe certificate can be installed on an external server, and whether its private key can be exported, is an open question that the group did not pursue, because option A already met the need.

Choice: option A. It is free, fully automatic, needs no extra software because Caddy is already the reverse proxy, and removes a manual renewal step. This rests on the documentation and on what the deployment actually did (a certificate was issued within seconds of the first start, see below). Option B was researched only through public support pages, so its limits are stated as open questions above, not as proven facts.

Where TLS terminates. TLS ends at Caddy, which is the only container that publishes ports 80 and 443. Caddy then forwards the request over plain HTTP on the private Compose network `gitea_web` to `gitea:3000`. That traffic never leaves the instance. Gitea's `ROOT_URL` is `https://git.ssw590-g1.tech/`, so the links and clone URLs it generates use HTTPS.

Certificate issued. On the first `docker compose up -d` (17:34 UTC) Caddy requested a certificate, completed the TLS-ALPN-01 challenge from several Let's Encrypt vantage points, and logged "certificate obtained successfully" within seconds of the stack starting (Listing 3). The certificate served at 17:35 UTC was read directly from the live connection (Listing 5).

Table 3.8: Certificate served by `git.ssw590-g1.tech`, October 15, 2026

Property	**Value**
Subject and name	`CN=git.ssw590-g1.tech`; subject alternative name `git.ssw590-g1.tech` only
Issuer	Let's Encrypt, intermediate `YE2`
Valid from / to	2026-10-15 16:35:56 UTC to 2027-01-13 16:35:55 UTC (90 days)
Key type and protocol	ECC key; TLS 1.3
Chain	`YE2`, `Root YE`, `ISRG Root X2`; Windows built the chain successfully (valid)
Client check	`curl` accepted the certificate without the insecure flag and received HTTP 200

```
Real output, Oct 15, 2026, 17:35 UTC, from Brahmi's Windows machine (outside the server) using curl.exe,
↪  Test-NetConnection, and .NET SslStream.
Run by Claude Code on Brahmi's go-ahead. The admin's own public IP is not in this file.
Limitation: this machine's IP is the ADMIN IP allowed to use SSH, so it cannot show that SSH is blocked for
↪  others.
A test from a second network (for example a phone hotspot) is still needed for the firewall row.

=== HTTP (port 80), curl -I http://git.ssw590-g1.tech/ ===
HTTP/1.1 308 Permanent Redirect
Location: https://git.ssw590-g1.tech/
Server: Caddy
Date: Thu, 15 Oct 2026 17:35:19 GMT

=== HTTPS (port 443), curl -I https://git.ssw590-g1.tech/ (no -k flag; certificate accepted) ===
HTTP/1.1 200 OK
Alt-Svc: h3=":443"; ma=2592000

=== Served certificate (read with .NET SslStream) ===
TLS protocol  : Tls13
Subject       : CN=git.ssw590-g1.tech
Issuer        : CN=YE2, O=Let's Encrypt, C=US
Not before    : 2026-10-15 16:35:56 UTC
Not after     : 2027-01-13 16:35:55 UTC      (90-day certificate)
SANs          : DNS Name=git.ssw590-g1.tech
Key algorithm : ECC
Chain valid   : True
  chain: CN=git.ssw590-g1.tech
  chain: CN=YE2, O=Let's Encrypt, C=US
  chain: CN=Root YE, O=ISRG, C=US
  chain: CN=ISRG Root X2, O=Internet Security Research Group, C=US

=== TCP probes from this machine (Test-NetConnection) ===
port 443  : OPEN
port 80   : OPEN
port 3000 : closed/filtered   (Gitea backend, expected not reachable)
port 2019 : closed/filtered   (Caddy admin API, expected not reachable)
port 5432 : closed/filtered   (PostgreSQL is not used; expected not reachable)
port 22   : OPEN from this machine only because its IP is the approved admin IP

Notes:
- Expected vs actual: 443 open, 80 open (redirect only), 3000/2019/5432 closed -> as expected.
- "closed/filtered" does not distinguish closed from firewall-dropped; both mean "not reachable".
```

```
- Renewal: not tested at the time of this capture (17:35 UTC). Caddy logged ACME renewal information (renewal
↪  window starts about 2026-12-14). A staging dry run was done later the same day (see the renewal-test
↪  listing); the real renewal of this certificate is expected between December 13 and 15, 2026 and has not
↪  been observed.
```

Listing 5: HTTP redirect, HTTPS response, the served certificate, and port probes from the administrator's computer, October 15, 2026, 17:35 UTC (real output).

HTTP to HTTPS redirect. Requesting `http://git.ssw590-g1.tech/` returned `HTTP/1.1 308 Permanent Redirect` with `Location: https://git.ssw590-g1.tech/` from Caddy (Listing 5, 17:35 UTC). Port 80 therefore serves only this redirect and certificate validation; it exposes no application.

Renewal. Caddy renews its certificates automatically before they expire; Let's Encrypt certificates last 90 days, so the first renewal is due about two months after issue [4]. Caddy's log already recorded the renewal window reported by the certificate authority (ACME Renewal Information), and the certificate and account are stored in the persistent volume `gitea_caddy_data`, so a container replacement does not lose them.

Renewal test (staging dry run). Caddy has no renew-now or dry-run command that we know of, so the test used the idea behind a certbot dry run: run the same ACME process against Let's Encrypt's *staging* server, which issues untrusted test certificates and does not touch the production certificate (Listing 6, October 15, 2026, 19:03 to 19:08 UTC). Three attempts were needed, and all are reported.

1. **Staging CA for the real hostname: inconclusive.** Adding a global staging CA and reloading Caddy produced no new order. Caddy already held a valid certificate for the name, kept serving it (same thumbprint), and logged only that automatic TLS was enabled. This proved nothing about renewal, and the production configuration was restored at once.
2. **Wrong directive: rejected.** The first version of the test block used `ca` inside the ACME issuer, which Caddy rejected ("unrecognized ACME issuer property: ca") and kept its previous working configuration, so the live site was unaffected. The correct directive is `dir`.
3. **Throwaway hostname with a staging-only issuer: succeeded.** A second site block was added for `52-73-254-36.sslip.io` (a public wildcard-DNS name that resolves to the server's address), configured with the staging directory, and the production block was left unchanged. Within about five seconds of the reload Caddy created an ACME account, solved the TLS-ALPN-01 challenge, finalized the order, downloaded the chain, and logged "certificate obtained successfully" with the staging issuer. The server then presented that certificate for the test name (issuer "(STAGING) Artificial Amaranth YE1", not trusted by Windows, as expected) while the production certificate stayed identical (same thumbprint).

The production Caddyfile was then restored (its SHA-256 matches the repository copy), the test certificate and backup file were removed, and normal checks confirmed the redirect, an HTTP 200 response, and the original production certificate (Listing 6).

```
Real output from the server (ssh ec2-user@git.ssw590-g1.tech) and from Brahmi's computer, Oct 15, 2026, 19:03
↪  to 19:08 UTC.
Run by Claude Code on Brahmi's go-ahead ("do the renewal test, she is not working right now").
SANITIZED: remote addresses in Caddy log lines replaced by <masked>; long JSON lines trimmed.

PURPOSE: show that Caddy, with this setup, can complete the full ACME certificate process. Caddy has no "renew
↪  now" or "dry run" command that we know of.
Method: run the same ACME process against Let's Encrypt's STAGING server (the idea behind "certbot renew
↪  --dry-run").
IMPORTANT LIMIT: this is a certificate ISSUANCE test for a different, throwaway hostname. It is NOT a renewal
↪  of the production certificate.

=== 0. Baseline (19:03:58 UTC): production certificate before the test ===
Issuer CN=YE2, O=Let's Encrypt | NotBefore 2026-10-15 16:35:56 | NotAfter 2027-01-13 16:35:55
Thumbprint 62AC575CAC34B6E3D61F581D5DCFFB321EB7C4AF | chain trusted: True

=== 1. ATTEMPT 1: global staging CA for the real hostname (19:04:08 UTC) ===  RESULT: INCONCLUSIVE
Caddyfile: added a global block { acme_ca https://acme-staging-v02.api.letsencrypt.org/directory }, then
↪  "caddy reload".
After 30 s and again at 19:04:54: the only certificate-related log line was "enabling automatic TLS
↪  certificate management";
no staging certificate was requested, storage held only the production certificate directory,
and the served certificate was still the production one (same thumbprint 62AC575C...C4AF).
Reading: Caddy already held a valid certificate for the name, so a reload with a different CA did not start a
↪  new order.
Nothing about renewal was demonstrated. Production config restored at 19:05:28 UTC (Caddyfile sha256 prefix
↪  7705e125f84a2c82 = repo copy).

=== 2. ATTEMPT 2a: invalid syntax (19:05:52 UTC) ===
Test block used "ca <url>" inside "issuer acme { }".  caddy reload -> error:
  adapting config using caddyfile: parsing caddyfile tokens for 'tls': unrecognized ACME issuer property: ca
Caddy refused the reload and kept the previous working configuration, so the live site was not affected.
↪  Correct directive is "dir <url>".

=== 3. ATTEMPT 2b: throwaway hostname with a staging-only issuer (19:06:44 UTC) ===  RESULT: SUCCESS
Hostname: 52-73-254-36.sslip.io (public wildcard DNS service; resolves to the server's Elastic IP, checked via
↪  8.8.8.8).
Caddyfile = production block UNCHANGED + this block:
  52-73-254-36.sslip.io {
      tls { issuer acme { dir https://acme-staging-v02.api.letsencrypt.org/directory } }
      respond "renewal test" 200
  }
caddy validate -> "Valid configuration"; caddy reload -> no error.
Caddy log (ACME lines, trimmed):
  using ACME account  https://acme-staging-v02.api.letsencrypt.org/acme/acct/353835223
  trying to solve challenge  identifier 52-73-254-36.sslip.io  challenge_type tls-alpn-01  ca
  ↪  .../acme-staging-v02.api.letsencrypt.org/directory
  served key authentication certificate (tls-alpn-01) x5   (remote <masked>)
  authorization finalized  authz_status valid
  validations succeeded; finalizing order
  ↪  https://acme-staging-v02.api.letsencrypt.org/acme/order/353835223/51532536033
  got renewal info (ACME Renewal Information) x3   names 52-73-254-36.sslip.io  window_start 1795720004
  ↪  window_end 1795830526
  successfully downloaded available certificate chains  count 3
  tls.obtain: certificate obtained successfully  issuer acme-staging-v02.api.letsencrypt.org-directory
Time from reload (ts 1792091205) to "certificate obtained successfully" (ts 1792091210): about 5 seconds.
Storage after: .../acme-v02.api.letsencrypt.org-directory/git.ssw590-g1.tech  (production)  and
               .../acme-staging-v02.api.letsencrypt.org-directory/52-73-254-36.sslip.io  (test)

=== 4. Certificates served at 19:07:40 UTC (read with .NET SslStream, validation bypass ON for this inspection
↪  only) ===
```

```
SNI 52-73-254-36.sslip.io : Issuer CN=(STAGING) Artificial Amaranth YE1, O=Let's Encrypt | NotBefore
↪  2026-10-15 18:08:16 | NotAfter 2026-12-18 18:08:15
                           Thumbprint 44202E6BCB0E5097A10AAD60D95B695E1DD9153F | trusted by Windows: False
                           ↪  (expected: staging CA is not trusted)
SNI git.ssw590-g1.tech    : Issuer CN=YE2, O=Let's Encrypt | Thumbprint
↪  62AC575CAC34B6E3D61F581D5DCFFB321EB7C4AF (IDENTICAL to baseline) | trusted: True
Production site during the test: https://git.ssw590-g1.tech/ -> HTTP/1.1 200 OK

=== 5. Restore and cleanup (19:07:56 UTC) ===
cat Caddyfile.prod > Caddyfile ; sha256 prefix 7705e125f84a2c82 (equals the repo copy)
caddy validate -> "Valid configuration" ; caddy reload ; throwaway staging certificate directory removed from
↪  storage;
backup file Caddyfile.prod removed. ~/gitea contains only .env, .env.example, Caddyfile, docker-compose.yml.
containers: caddy Up 2 hours, gitea Up 38 minutes (healthy).

=== 6. Final public checks (19:08:02 UTC, normal certificate validation, no bypass) ===
http://git.ssw590-g1.tech/  -> HTTP/1.1 308 Permanent Redirect
https://git.ssw590-g1.tech/ -> HTTP/1.1 200 OK
production certificate: issuer CN=YE2, O=Let's Encrypt; thumbprint 62AC575CAC34B6E3D61F581D5DCFFB321EB7C4AF
↪  (unchanged)

Notes:
- What it shows: Caddy as deployed (same image, same ports, TLS-ALPN-01 challenge reachable from the internet,
↪  writable certificate storage) completes a full ACME order, a fresh certificate is stored and served, and
↪  the process takes seconds.
- What it does NOT show: that the production certificate has been renewed. That certificate was issued
↪  2026-10-15 17:34 UTC; Caddy logged a renewal window starting at Unix time 1797189600 (2026-12-13 19:20:00
↪  UTC) and ending 1797345050 (2026-12-15 14:30:50 UTC), so the first automatic renewal is expected between
↪  December 13 and 15, 2026. That is a prediction from Caddy's log, not a test result.
- The test used a third-party service (sslip.io) for DNS and Let's Encrypt's staging server; no production
↪  certificate was requested and no rate limit on the real hostname was used.
- Live-site impact: no downtime, no change to the production certificate; Caddy reloaded its configuration
↪  four times (reload at 19:04:08 and 19:05:28 and 19:06:44 and 19:07:56).
```

Listing 6: The staging dry run, with all three attempts, the restore, and the final checks, October 15, 2026, 19:03 to 19:08 UTC (real output; remote addresses masked).

What this shows and what it does not. It shows that Caddy as deployed (same image, same ports, a reachable challenge port, writable certificate storage) completes a full certificate order in seconds and stores the result. It is an *issuance* test for a different hostname; it is **not** an observed renewal of the production certificate. That certificate was issued at 17:34 UTC on October 15, 2026, and Caddy's log gives it a renewal window from December 13, 2026 (19:20 UTC) to December 15, 2026. The first automatic renewal is therefore expected in that window; this is a prediction from the log, not a result. That window falls after this document is due, so the real renewal could not be observed before submission.

Proxy configuration. The proxy configuration is the `Caddyfile` shown in the deployment subsection: one site block for `git.ssw590-g1.tech` with a `reverse_proxy` to `gitea:3000`. No certificate settings are needed because Caddy's automatic HTTPS is the default. The file contains no secrets.

Expected versus actual.

Table 3.9: HTTPS checks, October 15, 2026 (rows 1 to 4 at 17:35 UTC from the administrator's computer; the renewal rows from the 19:03 to 19:08 UTC test)

Check	Expected	Actual	Evidence
HTTP on port 80	redirect to HTTPS	308 to `https://git.ssw590-g1.tech/`	Listing 5
HTTPS on port 443	valid certificate, page served	HTTP 200, certificate accepted	Listing 5
Certificate identity	issued for the hostname by a public CA	Let's Encrypt, subject and SAN match the hostname	Listing 5
Validity	not expired, about 90 days	expires 2027-01-13	Listing 5
Renewal process (staging dry run)	Caddy completes a full ACME order and serves the new certificate	staging order for a throwaway name succeeded in about 5 seconds; production certificate unchanged	Listing 6; **PASS** (issuance only)
Renewal of the production certificate	renews automatically before expiry	window predicted Dec 13 to 15, 2026; not yet observed	**PENDING**

3.2.5 Firewall and external exposure (15 points)

Owner: BD. Security group created and first outside checks run on October 15, 2026: **PASS**. Second-network (hotspot) test: **PASS**. Console screenshot of the rules: **PASS**.

The firewall. The public boundary is an AWS security group, a stateful firewall attached to the instance's network interface. Docker-published ports can bypass a host firewall such as `ufw` [2], so the group did not rely on any host firewall; none was configured on the instance.

Table 3.10: Security group

Property	Value
Name and ID	`ssw590-g1-gitea`, `sg-0415c75429368eb85`
Network	default VPC (`vpc-08a308ecc88120f9f`) in `us-east-1`
Attached to	instance `i-0c68ec1bc1e16a414`; this is the only security group passed at launch
Created	October 15, 2026, with the AWS command line interface (Listing 7)
Default network ACL	left unchanged and not examined

Rules.

Table 3.11: Inbound rules, IPv4 (IPv6 has no rules)

Port	Source	Purpose	Notes
443/TCP	`0.0.0.0/0`	HTTPS to Caddy	the one public application entry point
80/TCP	`0.0.0.0/0`	redirect and certificate validation	only a 308 redirect is served
22/TCP	the administrator's public IPv4 address, one /32 (redacted)	administrator SSH	key login only; the two administrators share one home network address

No other inbound rule exists, so everything else is dropped: Gitea's port 3000, Caddy's administration port 2019, and port 5432 (no database service exists because SQLite is used). **IPv6:** the security group has no IPv6 rules and no AAAA DNS record was created, so the service is IPv4 only. Caddy's published ports show IPv6 bindings (`[::]`) on the host, but they are not reachable because nothing admits IPv6 traffic. **UDP:** Caddy advertises HTTP/3 on UDP 443, but the group admits only TCP, so clients fall back to HTTPS over TCP. **Outbound:** the AWS default (all traffic allowed) was kept, because the instance must reach Docker Hub, GitHub releases, package mirrors, DNS, time servers, and Let's Encrypt.

```
AWS CLI output, run from Brahmi's machine (PowerShell, aws-cli 2.37.9), region us-east-1, Oct 15, 2026.
SANITIZED: the admin's public IPv4 address was replaced by <ADMIN-IP>. Nothing else was edited.
The commands were run by Claude Code on Brahmi's go-ahead; no credentials appear in this output.

=== Step 1: default VPC (read-only) ===
VPC: vpc-08a308ecc88120f9f
=== Step 2: create security group ===
  aws ec2 create-security-group --group-name ssw590-g1-gitea \
     --description "SSW590 Group1 Gitea: 443/80 public, 22 admin only" --vpc-id <default VPC> \
     --tag-specifications Name=ssw590-g1-gitea, Project=ssw590-group1-assignment4
SG: sg-0415c75429368eb85
=== Step 3: ingress rules (authorize-security-group-ingress, one call per rule) ===
sgr-07ffad0f8d05c8f97        443
sgr-09bfd2a236c9aa42a        80
sgr-071a77a76f43108d2        22
=== Step 4: verify (describe-security-groups) ===
80        80        tcp        0.0.0.0/0        HTTP redirect and ACME validation only
22        22        tcp        <ADMIN-IP>/32        Admin SSH from Brahmi only
443        443        tcp        0.0.0.0/0        HTTPS to Caddy
--- egress ---
-1        0.0.0.0/0

Notes:
- Inbound IPv4 only; no IPv6 rules and no AAAA DNS record are planned.
- Egress is the AWS default (all outbound allowed).
- This security group is not attached to any instance yet (no instance exists).
- This output proves the rules exist on the group. It is NOT an external exposure test.
```

Listing 7: Creation of the security group and its rules with the AWS command line interface, October 15, 2026 (real output; the administrator's address is replaced by a placeholder).

Host and Compose port mappings. `docker compose ps` on the server (Listing 3) shows:

- `caddy`: publishes `0.0.0.0:80`, `0.0.0.0:443`, their `[::]` equivalents, and `443/udp`; port `2019/tcp` (Caddy's administration interface) is only exposed inside the container and is not published.
- `gitea`: `22/tcp` and `3000/tcp` are exposed only to the Compose network; nothing is published on the host.

Network diagram. Figure 3.11 shows what the security group admits and drops. The request path through the proxy is in Figure 3.17.

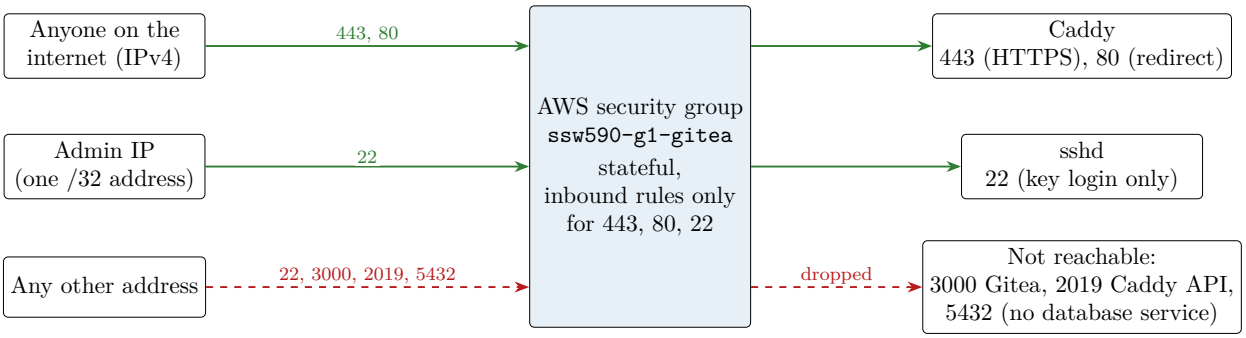


Figure 3.11: What the security group admits (green) and drops (red, dashed) for the Gitea VM [1, 2].

External tests. Two sets of outside checks were run from the administrator's computer, which is not the server (Listing 5 and Listing 8). The first was at 17:35 UTC on October 15, 2026 on the administrator's home network, which uses the approved administrator address, so it could not show that SSH is blocked for anyone else. The second was at 18:57:54 UTC the same day with the computer connected to a phone hotspot: a different network and a different public IPv4 address (the program confirmed the address differed from the approved one before probing; the addresses are not recorded here).

Table 3.12: External exposure checks

Port	Expected	Admin network, 17:35	Hotspot network, 18:57	Result
443	open, HTTPS	open, HTTP 200	open, HTTP 200	**PASS**

Port	Expected	Admin network, 17:35	Hotspot network, 18:57	Result
80	open, redirect only	open, 308 redirect	open, 308 redirect	**PASS**
3000 (Gitea)	not reachable	closed or filtered	closed or filtered	**PASS**
2019 (Caddy API)	not reachable	closed or filtered	closed or filtered	**PASS**
5432 (database)	not reachable	closed or filtered	closed or filtered	**PASS**
22 (SSH)	open from the admin address only	open	blocked: connection timed out	**PASS**

"Closed or filtered" means the connection attempt failed; the test cannot tell a refused connection from a dropped one, and both mean the port is not usable from that network. For SSH, the hotspot attempt used the *valid* private key and still failed after about 20 seconds with "Connection timed out". That is a network-level drop, not an authentication failure, which confirms that the security group, not the key, keeps other addresses out. **Limits:** this is one mobile network at one moment, and it does not test IPv6 (no AAAA record exists).

```
Real output, Oct 15, 2026, 18:57:54 UTC, from Brahmi's Windows laptop connected to her PHONE HOTSPOT (a
↪  different network and public IPv4 address from the approved admin address).
Run by Claude Code on Brahmi's go-ahead. Checked before probing: the laptop's public IPv4 differs from the
↪  approved admin address (neither address is printed here).
Target: git.ssw590-g1.tech (Elastic IP of the instance). Tools: Test-NetConnection, curl.exe, OpenSSH client.

=== TCP probes (Test-NetConnection) ===
port 443  : OPEN
port 80   : OPEN
port 22   : closed/filtered (no connection)
port 3000 : closed/filtered (no connection)
port 2019 : closed/filtered (no connection)
port 5432 : closed/filtered (no connection)

=== HTTP and HTTPS ===
curl -I http://git.ssw590-g1.tech/   ->  HTTP/1.1 308 Permanent Redirect   Location:
↪  https://git.ssw590-g1.tech/
curl -I https://git.ssw590-g1.tech/  ->  HTTP/1.1 200 OK

=== SSH with the VALID private key from this unapproved address ===
ssh -i <valid key> -o StrictHostKeyChecking=yes -o BatchMode=yes -o ConnectTimeout=10
↪  ec2-user@git.ssw590-g1.tech "echo connected"
ssh: connect to host git.ssw590-g1.tech port 22: Connection timed out      (exit 255, about 20 s)

Notes:
- Expected vs actual: 443 open, 80 open (redirect only), 22 blocked, 3000/2019/5432 not reachable -> all as
↪  expected.
- SSH failed at the network level (timeout), not at authentication, even though the key is valid: the security
↪  group drops the packets from addresses other than the approved admin /32.
- "closed/filtered" cannot tell a refused connection from a dropped one; both mean the port is not usable from
↪  this network.
```

```
- This closes the second-network row in the firewall section. It is one test, from one mobile network, at one
↪  moment; it does not test IPv6 (no AAAA record exists).
```

Listing 8: Probes from the phone-hotspot network, October 15, 2026, 18:57 UTC (real output; neither network address is printed).

The server's own SSH log gives a second view. It was read at 18:34 UTC, about an hour after the instance started. It lists only successful key logins from the two administrators' keys and **zero** failed or invalid attempts (Listing 9), which is consistent with port 22 being reachable only from the approved address. One hour of log is a weak measure, not proof.

```
Real output from the server (journalctl for sshd), Oct 15, 2026, read at 18:34:18 UTC. Run by Claude Code on
↪  Brahmi's go-ahead.
SANITIZED: source addresses replaced by <masked>; remote port numbers replaced by <n>. Only public key
↪  fingerprints are shown.

--- successful key logins, newest 12 ---
2026-10-15T17:55:28+0000 ... Accepted publickey for ec2-user from <masked> port <n> ssh2: ED25519
↪  SHA256:Wb244wRaDU7Dd+q7qlh7Lr2g2838XDRUGSS+4zGYX34   (Brahmi's key)
2026-10-15T17:55:29+0000 ... same key
2026-10-15T18:15:23+0000 ... same key
2026-10-15T18:15:24+0000 ... same key (two lines)
2026-10-15T18:25:52+0000 ... Accepted publickey for ec2-user from <masked> port <n> ssh2: ED25519
↪  SHA256:1oYCcSuCBcdksbkTTJ91uPmkJ8MQmvWDUJuENqtQfPo   (Sakshi's key: FIRST LOGIN)
2026-10-15T18:28:39+0000 ... Sakshi's key
2026-10-15T18:28:40+0000 ... Sakshi's key
2026-10-15T18:29:10+0000 ... Sakshi's key
2026-10-15T18:30:33+0000 ... Sakshi's key
2026-10-15T18:34:17+0000 ... Brahmi's key (this check)
2026-10-15T18:34:18+0000 ... Brahmi's key (this check)

--- count of successful logins by key fingerprint (whole log since boot) ---
      5 SHA256:1oYCcSuCBcdksbkTTJ91uPmkJ8MQmvWDUJuENqtQfPo     (Sakshi)
     18 SHA256:Wb244wRaDU7Dd+q7qlh7Lr2g2838XDRUGSS+4zGYX34     (Brahmi, including the AI assistant's checks
     ↪  run with her key)

--- failed or invalid SSH attempts in the same log (count) ---
0

Notes:
- Proves: Sakshi's key authenticates to the server (first success 18:25:52 UTC); the security group admitted
↪  her connection.
- The zero count means no failed, invalid-user, or pre-authentication-close lines appeared since boot (about 1
↪  hour). It is consistent with port 22 being reachable only from the approved address, but it is one hour of
↪  logs, not a long-term measure.
- Does NOT prove Sakshi's Gitea sign-in (she reported it works; not independently checked).
- Both administrators use the same Linux account (ec2-user); the key fingerprint in the log is what tells them
↪  apart.
```

Listing 9: SSH journal on the server, read at 18:34 UTC on October 15, 2026 (source addresses masked; only public-key fingerprints shown).

Figure 3.12 shows the same rules in the AWS console, on the instance's Security tab, with the account number and the administrator's address blacked out. It confirms that the instance is running with the Elastic IP, has no IPv6 address, requires IMDSv2, and has the security group `ssw590-g1-gitea` attached (launched October 15, 2026, 13:16 Eastern time). The rule list is scrolled to its end; the three rules are 80 and 443 from anyone and 22 from the redacted administrator address, with the descriptions given when they were created. A command-line listing of the same rules is in Listing 7.

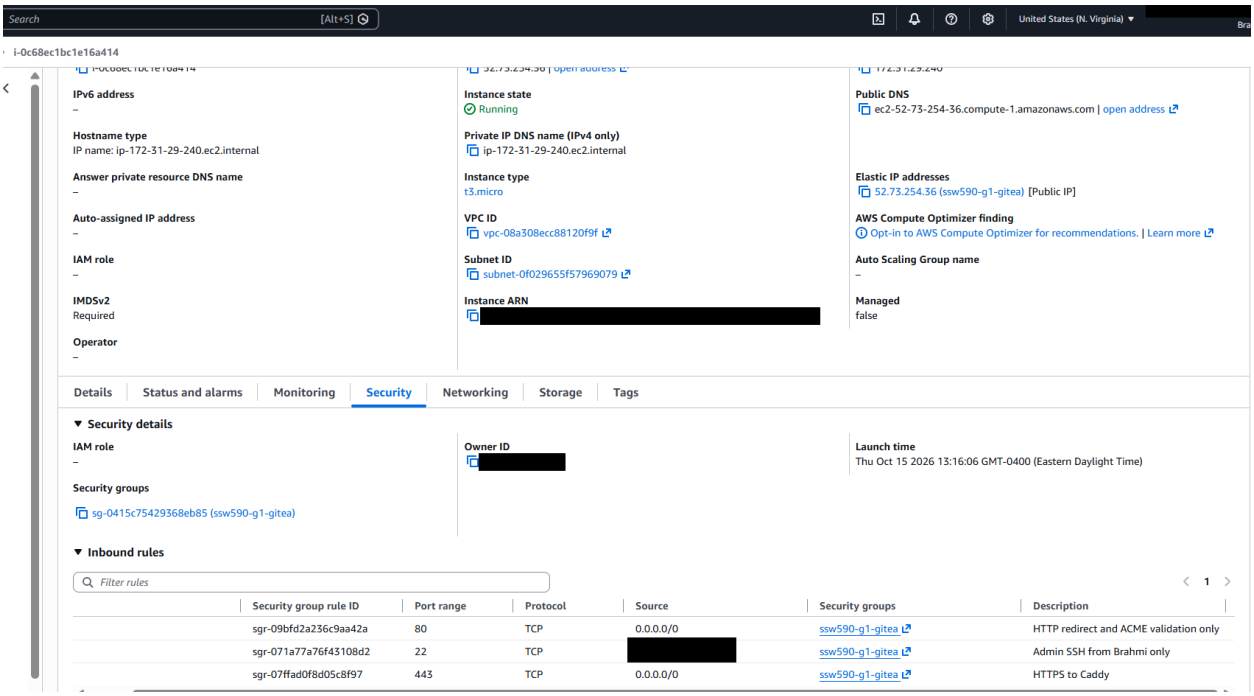


Figure 3.12: AWS console, instance `i-0c68ec1bc1e16a414`, Security tab, October 15, 2026 (redacted).

Corrections. None have been needed so far. The only exposure found beyond the plan is informational: internet scanners reached the HTTPS port within seconds of the site coming online (Listing 3) and probed for unrelated software, which returned 404 responses. Nobody could create an account, because registration is disabled.

Why HTTPS does not replace a firewall. HTTPS protects the content of a connection between a browser and Caddy; it says nothing about which ports exist or who may connect to them. Without the security group, ports such as 22, 3000, or 2019 would accept connections from the whole internet, and encryption on port 443 would not help. The firewall decides which services can be reached at all; TLS only protects the ones that are reachable. The two address different risks, so both are needed [1].

3.2.6 Startup team workflow (20 points)

Owner: SD (BD is the second member and reviewer). **Status:** organization, repository, backlog, and two reviewed, merged, issue-closing pull requests on October 15, 2026 **PASS**.

Organization, repository, and members

Table 3.13: Startup organization and members

Item	Value
Organization	group1-startup
Product repository	group1-startup/twintrack (public, so the TA and instructor can inspect it signed out); default branch main
README	README.md on main, added by pull request #8
Member: Brahmi Dalvi (brahmi)	organization owner and Gitea site administrator (infrastructure); also in team developers
Member: Sakshi Deshpande (sakshi)	team developers: write access to code, issues, and pull requests; cannot create repositories in the organization or change settings (product)
Branch rule on main	no direct pushes, one approving review required; a direct commit by the administrator was refused with HTTP 403 (issue #4)

The least-privilege split was confirmed by a real failure: SD's first attempt to create the repository returned HTTP 403 "User is not allowed to create repositories in this organization", so BD, as owner, created it (Table 3.19).

Backlog

Six actionable issues mix product and operations work (Table 3.14). They were created through the Gitea API by the setup script assignment-4/gitea/row6-setup.py, run by each member with a personal token typed at a hidden prompt and deleted afterwards (Listing 10).

Table 3.14: Startup backlog in group1-startup/twintrack, October 15, 2026

#	Issue	Type	Assignee	State
1	Write the product README	product	SD	closed by PR #8
2	Add CONTRIBUTING.md for the issue, branch, PR, review flow	operations	BD	closed by PR #7
3	Define the TwinTrack task data model	product	SD	open
4	Protect main: require PR + 1 approval	operations	BD	closed after the refusal test
5	Document the Gitea restart and troubleshooting runbook	operations	SD	open
6	Add the domain expiry date (October 15, 2027) to the shared calendar and name its owner	operations	BD	open

Issue-to-merge traces

Each trace has an issue, a branch named after it, a pull request whose description contains `Closes #N`, a comment and an approving review by the *other* member, a merge, and automatic closure of the issue. Gitea numbers pull requests in the same sequence as issues, so the first pull request is #7. All times are UTC from the Gitea API (Listing 11).

Table 3.15: Issue-to-merge traces, October 15, 2026

Step	Trace 1 (author SD)	Trace 2 (author BD)
Issue	#1 Write the product README	#2 Add CONTRIBUTING.md
Branch	`issue-1-readme`	`issue-2-contributing`
Pull request	#8 “Add product README”, body “Closes #1”, opened 19:40:12	#7 “Add CONTRIBUTING.md”, body “Closes #2”, opened 19:31:18
Reviewer comment	BD, 19:47:09	SD, 19:39:04 (asks that authors reply to or resolve each comment before approval)
Approval	BD, APPROVED, 19:47:21	SD, APPROVED, 19:46:27
Merge	by SD, 19:47:45, commit `15f8a00343`	by BD, 19:49:04, commit `0d0c3dac4d`
Issue closed	#1 closed 19:47:45 by the merge	#2 closed 19:49:04 by the merge

Review quality, stated honestly. Both reviews were real approvals by the other member, enforced by the branch rule (“0 of 1 official approvals” blocked the merge until then; Figure 3.13). They were light: BD’s comment on #8 was a placeholder (“comment from brahmi”), and SD’s request on #7 was approved and merged without a reply or a change to `CONTRIBUTING.md`. SD’s suggestion remains open as a follow-up for the next contribution.

```
Real output of assignment-4/gitea/row6-setup.py run by Sakshi in her own terminal (macOS), Oct 15, 2026 (about
↪  19:39 UTC).
The access token was typed at a hidden prompt and is not in this output. Followed by an independent check made
↪  with NO token (anonymous Gitea API).

=== Script output (Sakshi's stages) ===
Gitea access token (hidden, not saved):
Signed in as: sakshi  (site admin=False)
[repo] exists already: https://git.ssw590-g1.tech/group1-startup/twintrack  (private=False)
[issues] exists: #1 Write the product README
[issues] exists: #2 Add CONTRIBUTING.md for the issue, branch, PR, review flow
[issues] exists: #3 Define the TwinTrack task data model
[issues] exists: #4 Protect `main`: require PR + 1 approval
[issues] exists: #5 Document the Gitea restart and troubleshooting runbook
[issues] exists: #6 Add the domain expiry date (Oct 15, 2027) to the shared calendar and name its owner
[readme-pr] branch issue-1-readme created with the product README (from sd/backlog-draft.md)
[pr] opened: https://git.ssw590-g1.tech/group1-startup/twintrack/pulls/8
    NEXT: Brahmi reviews it (at least one comment), Sakshi addresses it, Brahmi approves, then merge.

Your stages are done. Brahmi runs her part next (protection and the CONTRIBUTING.md PR).

Now DELETE the token in Gitea > Settings > Applications.
```

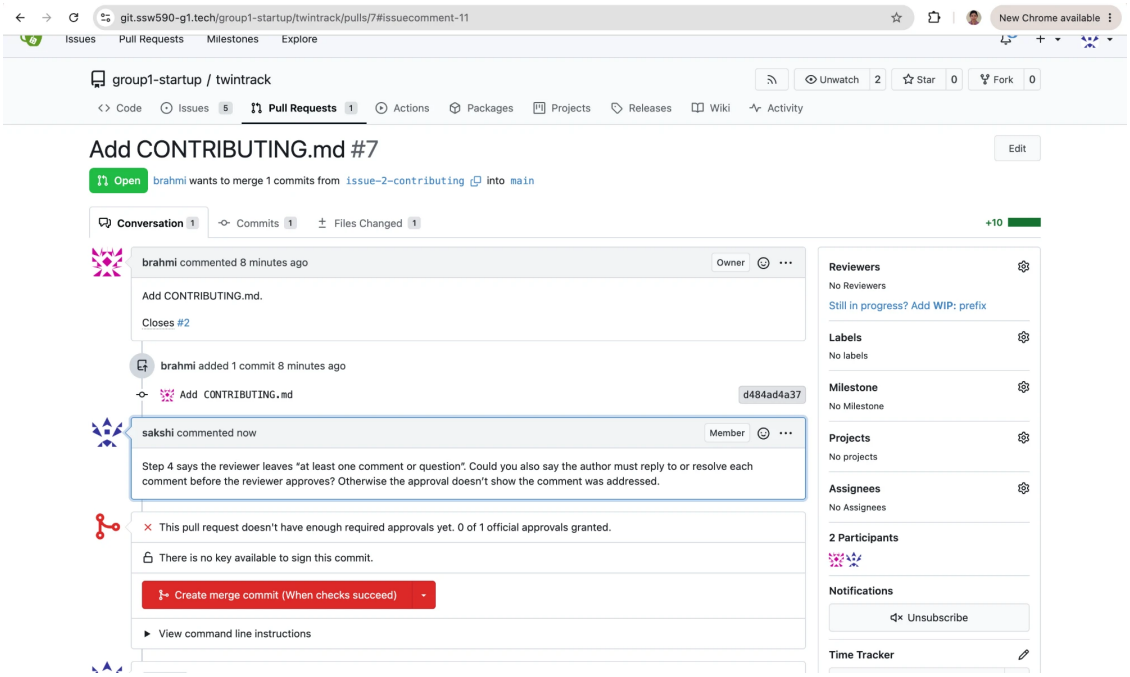


Figure 3.13: Pull request #7 before approval, October 15, 2026: SD's review comment, and the merge blocked with "0 of 1 official approvals granted".

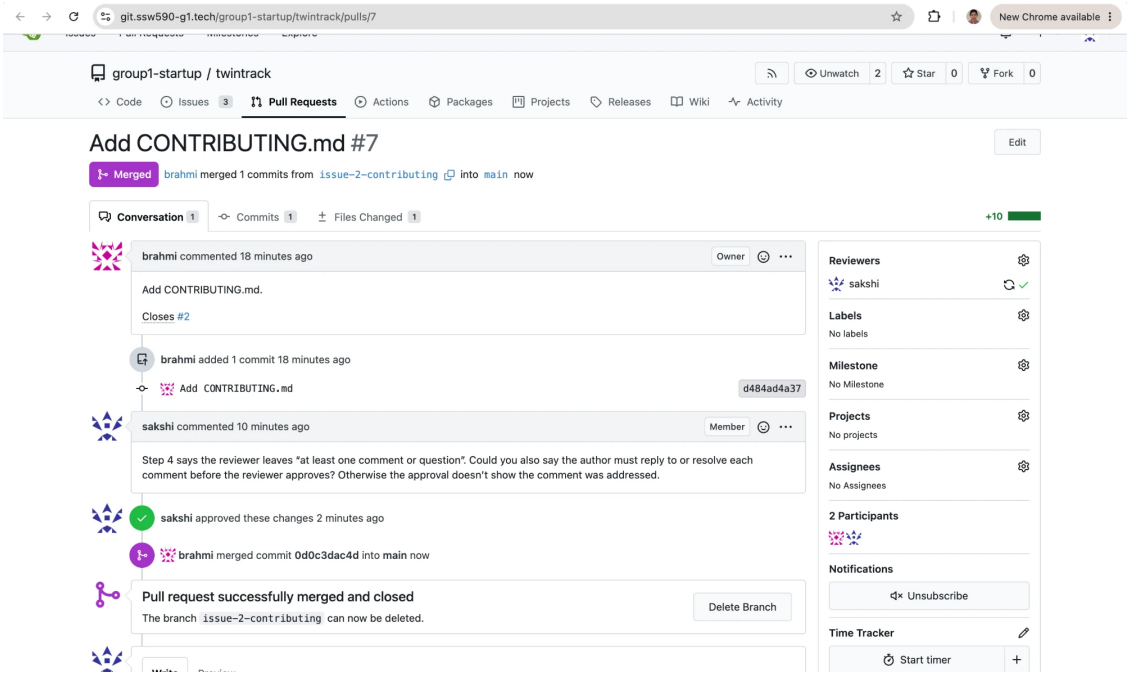


Figure 3.14: Pull request #7 merged, October 15, 2026: "Closes #2", approved by `sakshi`, merged by `brahmi` as commit `0d0c3dac4d`.

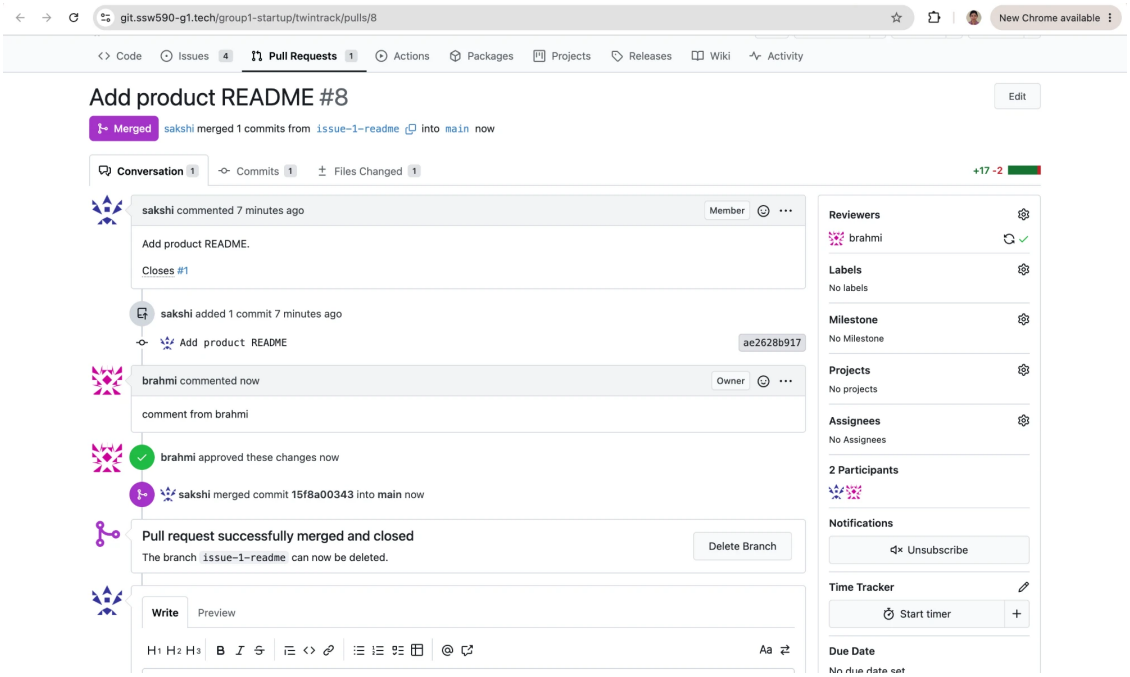


Figure 3.15: Pull request #8 merged, October 15, 2026: "Closes #1", approved by `brahmi`, merged by `sakshi` as commit `15f8a00343`.

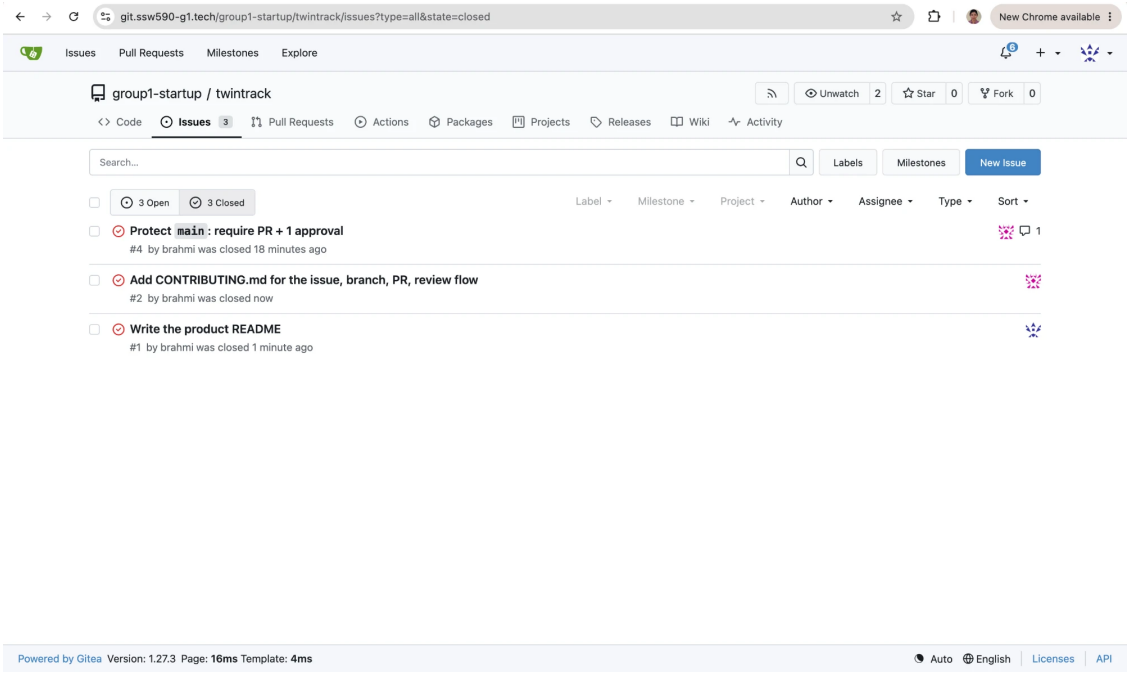


Figure 3.16: Closed issues in `twintrack`, October 15, 2026: #1 and #2 closed by their merges, #4 closed after the branch-protection test.

```
Repository: https://git.ssw590-g1.tech/group1-startup/twintrack

=== Independent check, anonymous API, 2026-10-15 19:40:30 UTC ===
#8 'Add product README' state=open head=issue-1-readme base=main by=sakshi body='Add product README.\n\nCloses
↪  #1'

Notes:
- Sakshi's first run (before Brahmi's) stopped with HTTP 403 "User is not allowed to create repositories in
↪  this organization";
  nothing was created. Expected: the team `developers` cannot create repositories. Brahmi then ran her stages
  ↪  first.
```

Listing 10: SD's run of the setup script, which opened pull request #8, October 15, 2026 (real output; no token in it).

```
Anonymous Gitea API listing (no token), group1-startup/twintrack, read 2026-10-15 19:49:50 UTC
Captured by Claude Code from Sakshi's machine. Public data only; nothing below is secret. Curly quotes and
↪  apostrophes in comment text are written as straight ones.

repo: private=False default_branch=main
branch main: protected=True required_approvals=1

=== Issues ===
#1  closed assignee=sakshi  closed_at=2026-10-15T19:47:45Z  Write the product README
#2  closed assignee=brahmi  closed_at=2026-10-15T19:49:04Z  Add CONTRIBUTING.md for the issue, branch, PR,
↪  review flow
#3  open   assignee=sakshi  closed_at=None  Define the TwinTrack task data model
#4  closed assignee=brahmi  closed_at=2026-10-15T19:31:18Z  Protect `main`: require PR + 1 approval
#5  open   assignee=sakshi  closed_at=None  Document the Gitea restart and troubleshooting runbook
#6  open   assignee=brahmi  closed_at=None  Add the domain expiry date (Oct 15, 2027) to the shared calendar
↪  and name its owner

=== Pull request #7 ===
title='Add CONTRIBUTING.md' author=brahmi head=issue-2-contributing base=main
body='Add CONTRIBUTING.md.\n\nCloses #2'
created_at=2026-10-15T19:31:18Z state=closed merged=True merged_at=2026-10-15T19:49:04Z merged_by=brahmi
↪  merge_commit=0d0c3dac4d
comment  2026-10-15T19:39:04Z by sakshi  'Step 4 says the reviewer leaves "at least one comment or question".
↪  Could you also say the author must reply to or resolve each comment before the reviewer approves?
↪  Otherwise the approval doesn't show the comment was addressed.'
review   2026-10-15T19:46:27Z by sakshi  state=APPROVED body=''

=== Pull request #8 ===
title='Add product README' author=sakshi head=issue-1-readme base=main
body='Add product README.\n\nCloses #1'
created_at=2026-10-15T19:40:12Z state=closed merged=True merged_at=2026-10-15T19:47:45Z merged_by=sakshi
↪  merge_commit=15f8a00343
comment  2026-10-15T19:47:09Z by brahmi  'comment from brahmi'
review   2026-10-15T19:47:21Z by brahmi  state=APPROVED body=''
```

Listing 11: The whole workflow read from the public Gitea API without a token, October 15, 2026, 19:49:50 UTC: issues, branch rule, both pull requests with comments, approvals, merges, and merge commits.

3.2.7 Architecture and operation (5 points)

Owner: SD. **Status:** figure and text describe the live deployment of October 15, 2026 **PASS**; runbook commands run on the server (Listing 12) **PASS**.

Architecture

Figure 3.17 shows the deployment, adapted from the course reference design [1] for the SQLite path only. Black arrows cross the public internet; blue arrows stay inside the instance. The facts come from the deployment evidence (Sections 3.2.2 to 3.2.5): one AWS EC2 instance `ssw590-g1-gitea` (`t3.micro`, Amazon Linux 2023, region `us-east-1`) with an Elastic IP, security group `ssw590-g1-gitea`, and Docker Compose project `gitea` in `~/gitea`.

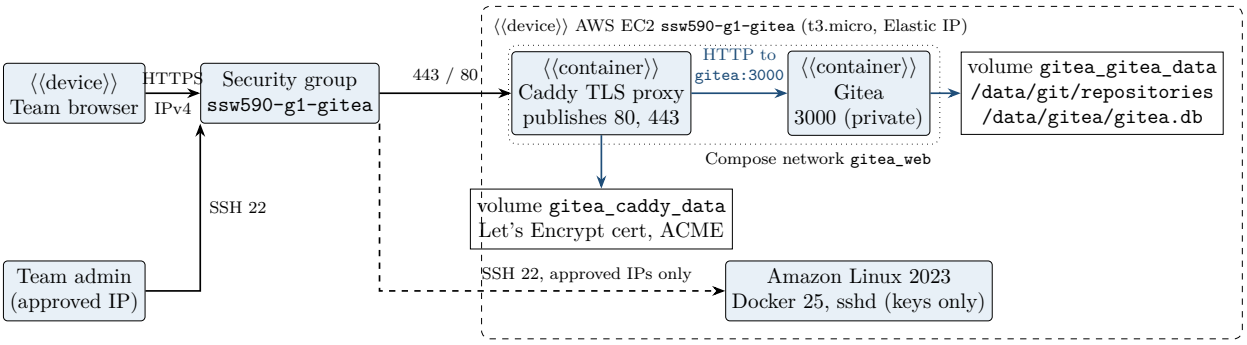


Figure 3.17: Group 1 Gitea deployment on one AWS EC2 instance, October 15, 2026. The security group admits HTTPS (443) and HTTP (80, redirect only) from anyone over IPv4, and SSH (22) only from the approved team address. Only Caddy publishes ports; Gitea's port 3000, its container `sshd`, Caddy's admin port 2019, and the SQLite file are private. There is no IPv6 path [1, 3, 4].

Request path

1. The browser resolves git.ssw590-g1.tech through public DNS (an `A` record at Name.com, TTL 300; no `AAAA` record) to the instance's Elastic IP and connects on 443.
2. The security group admits 443 and 80 from anywhere and 22 only from the approved address; everything else is dropped.
3. Caddy terminates TLS with a Let's Encrypt certificate for git.ssw590-g1.tech, obtained with the TLS-ALPN-01 challenge on port 443 and stored in the `gitea_caddy_data` volume. A request on port 80 receives `308 Permanent Redirect` to HTTPS.
4. Caddy forwards the request as plain HTTP over the private Compose network `gitea_web` to `gitea:3000`, found by service name.
5. Gitea reads and writes the SQLite file and the bare Git repositories in the `gitea_gitea_data` volume and answers through Caddy.

Git clone and push use the same HTTPS path; Gitea's own SSH clone is disabled (`DISABLE_SSH=true`). The image's `sshd` still listens on port 22 *inside* the container, but that port is not published.

Administration path

Administrators connect with SSH keys only, as `ec2-user`, from the approved address. Each member has a separate key, and the server's `sshd` log tells them apart by key fingerprint.

Before the first connection, each member compared the server's host key with the one published by the administrator (`SHA256:sJSqc/LAVTJ...buvgosM`). Docker is run by `ec2-user`, which is in the `docker` group.

Public and private interfaces

Table 3.16: Interfaces and exposure

Interface	Exposure	Notes
Caddy 443/TCP	public	HTTPS for the web UI, API, and Git
Caddy 80/TCP	public	`308` redirect to HTTPS only
Host SSH 22/TCP	restricted	approved address only (Section 3.2.5)
Gitea 3000/TCP	private	Compose network only, no host port mapping
Container `sshd` 22	private	not published
Caddy admin 2019/TCP	private	inside the Caddy container, not published
SQLite	none	a file, not a network service
IPv6	none	no security-group rule and no `AAAA` record

Where data lives

All Gitea state is in the named volume `gitea_gitea_data` (Table 3.5); TLS certificates and the ACME account are in `gitea_caddy_data`, and Caddy's runtime configuration in `gitea_caddy_config`. The volumes are on the instance's 20 GB `gp3` root disk. Containers hold no state and can be replaced at any time; row 3 did exactly that.

Operating runbook

All commands run on the instance from `~/gitea`.

```
docker compose ps                       # both services Up; gitea (healthy)
docker compose logs --tail=100 gitea    # application errors
docker compose logs --tail=100 caddy    # TLS issuance/renewal, proxy errors
docker compose restart gitea            # restart without recreating
docker compose up -d --force-recreate gitea   # replace the container, keep volumes
docker compose up -d                    # re-apply the Compose file after a change
curl -sS -o /dev/null -w "%{http_code}\n" https://git.ssw590-g1.tech/api/healthz
```

The read-only commands were run on October 15, 2026 at 18:50 UTC (Listing 12). Container replacement was exercised by the row 3 test (Table 3.6): Gitea was back and healthy about 30 seconds later. A plain `docker compose restart` was not run separately, because it does strictly less than that replacement.

```
Real output from the server, run by Claude Code from Sakshi's machine (ssh ec2-user@git.ssw590-g1.tech,
↪  Sakshi's key), Oct 15, 2026.
Purpose: the read-only commands of the row 7 runbook, run for real. Restart command not run here (see the
↪  manual).
SANITIZED: Caddy log lines reduced to level, logger, message and upstream, so client IP addresses are not
↪  shown. The only addresses left are 0.0.0.0 (listen on all), 127.0.0.1 (loopback) and 172.18.0.x (the
↪  private Docker network). The single-character ellipsis (U+2026) that docker prints in the COMMAND column
↪  is written as "...".

$ date -u
2026-10-15T18:50:22Z
$ docker compose ps
NAME      IMAGE                COMMAND                  SERVICE   CREATED             STATUS
↪  PORTS
caddy     caddy:2.11.7         "caddy run --config ..."   caddy     About an hour ago   Up About an hour
↪  0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp, 443/udp, 2019/tcp
gitea     gitea/gitea:1.27.3   "/usr/bin/entrypoint..."   gitea     21 minutes ago      Up 21 minutes
↪  (healthy)   22/tcp, 3000/tcp
$ docker compose logs --tail=3 gitea
gitea  | 2026/10/15 18:49:54 HTTPRequest [I] router: completed GET /api/healthz for 127.0.0.1:55688, 200 OK in
↪  0.4ms @ healthcheck/check.go:76(healthcheck.Check)
gitea  | 2026/10/15 18:50:10 .../src/net/http/server.go:2286:HandlerFunc.ServeHTTP() [I] PING DATABASE sqlite3
gitea  | 2026/10/15 18:50:10 HTTPRequest [I] router: completed GET /api/healthz for 172.18.0.3:32964, 200 OK
↪  in 0.5ms @ healthcheck/check.go:76(healthcheck.Check)
$ docker compose logs --tail=3 caddy   (fields: level, logger, msg, upstream)
warn http.handlers.reverse_proxy aborting with incomplete response upstream=gitea:3000
warn http.handlers.reverse_proxy aborting with incomplete response upstream=gitea:3000
warn http.handlers.reverse_proxy aborting with incomplete response upstream=gitea:3000
$ curl -sS -o /dev/null -w "%{http_code}" https://git.ssw590-g1.tech/api/healthz
healthz HTTP 200
$ docker volume ls --filter name=gitea_
DRIVER    VOLUME NAME
local     gitea_caddy_config
local     gitea_caddy_data
local     gitea_gitea_data
$ docker network ls --filter name=gitea_
NETWORK ID     NAME        DRIVER    SCOPE
ea58c8a1e342   gitea_web   bridge    local
```

Listing 12: Read-only runbook commands on the EC2 instance, October 15, 2026, 18:50 UTC (real output; client addresses removed from the Caddy lines).

The Caddy warnings "aborting with incomplete response" appear when a response stream to a client is cut off (for example, a browser closing the connection); users did not see errors, and the health check returned 200.

Table 3.17: Troubleshooting

Symptom	**Check**
Browser cannot connect	DNS still resolves to the Elastic IP; security group allows 443; `docker compose ps` shows Caddy Up
502 Bad Gateway	Gitea is down or still starting: `docker compose logs gitea`; one transient 502 was seen at first start (Section 3.2.2)
Certificate error	`docker compose logs caddy`; port 443 reachable for TLS-ALPN-01; `gitea_caddy_data` still mounted
SSH refused or times out	the member's public address has changed; ask the administrator to update the security group

Symptom	Check
Data appears missing	`docker volume ls` still lists `gitea_gitea_data`; Compose is run from `~/gitea` so the project name is still `gitea`

Never run `docker compose down -v` or `docker volume rm` on the instance: they delete the volumes and all data.

Persistence is not backup

The persistence test (Section 3.2.3) proves data survives a *container* replacement because it lives on a volume. A backup is a separate copy of that data, stored elsewhere, that has been restored successfully. Without one, deleting the volume, terminating the instance (its root disk has `DeleteOnTermination=true`), or corrupting the database still loses everything. A verified backup and restore is the optional extra credit (Section 3.3.1).

3.2.8 Manual quality and controls (15 points)

Owner: SD. **Status:** final review of the whole manual on October 15, 2026, before submission **PASS**.

Table 3.18: Manual quality controls, checked October 15, 2026

Control	Status	Where / how checked
Manual version increased	**PASS**	0.1 → 1.0 (title page and page footers), dated October 15, 2026
History: initial setup entry	**PASS**	Version 0.1, October 12, 2026, Table 1
History: this update, with links	**PASS**	Version 1.0 entry and the dated entries for each row, each linking its section, table, or figure
Hosts and Passwords tables current, hints only, named in history	**PASS**	Chapter 2; every row holds a vault reference or hint, never a secret; the version 1.0 history entry names both tables as updated
Headings, captions, labels	**PASS**	One subsection per rubric row in rubric order; every figure, table, and listing has a caption and label; the final build has no undefined or multiply defined references
Evidence inside the PDF	**PASS**	Command output as listings and screenshots as figures, referenced by number rather than by file name
Citations for external facts	**PASS**	Course manual, Gitea, Caddy, Let's Encrypt, Docker, and GitHub Education sources in the Bibliography

Control	Status	Where / how checked
Glossary and index entries for new terms	**PASS**	Glossary, Acronyms, Index
Secrets and addresses redacted	**PASS**	Scripted scan of all chapter sources and of every command-output file printed in the PDF for private keys, access tokens, AWS keys and account numbers, passwords, e-mail addresses, and IPv4 addresses. Only expected addresses remain: the server's public Elastic IP (published in DNS anyway), public DNS resolvers, loopback, and private Docker and AWS addresses. Every screenshot was inspected by eye; the AWS console screenshot is redacted, and browser tab bars with unrelated personal tabs were cropped
Hyperlinks to the Gitea service, organization, repositories, issues, and pull requests	**PASS**	Repositories are public, so the links work for the TA and instructor without an account
No `\Todo`, undefined references, or build errors	**PASS**	Final build of October 15, 2026: no undefined or multiply defined references, no errors, and no `\Todo` marker in the sources or in the text of the PDF (both searched). Five markers that remained outside SD's regions (domain calendar owner, extra-credit placeholder, Introduction link, and two AI-log review lines) were removed by BD

Failed tests and corrective actions

All failed or inconclusive attempts made for this chapter are listed here, with the section where each is described in full.

Table 3.19: Failed tests and corrective actions

Date	What failed	Correction
October 12, 2026	SD's first local rehearsal of `persistence-test.sh` stopped with a Python syntax error in the script's output formatting.	Fixed the quoting; the rerun completed. Rehearsal only; no server involved.

Date	What failed	Correction
October 15, 2026	The AWS command line tool on Windows crashed printing a non-ASCII character from the instance console log, so an early check looked like "still waiting" (Section 3.2.2).	Read the log again with UTF-8 forced.
October 15, 2026	The IAM user was denied the public image-lookup parameter (Section 3.2.2).	Found the Amazon Linux image with `describe-images`.
October 15, 2026	A multi-line script passed to `ssh` from Windows PowerShell 5.1 lost its quotes and failed (Section 3.2.2).	Scripts are copied to the server and run there.
October 15, 2026	One transient 502 at first start: a scanner reached Caddy before Gitea was listening (Section 3.2.2).	None needed; it did not recur.
October 15, 2026	Renewal test, attempt 1 (staging CA for the real hostname): inconclusive; attempt 2: Caddy rejected the directive `ca` (Section 3.2.4).	Attempt 3 used a throwaway hostname with a staging-only issuer and succeeded; the production Caddyfile was restored.
October 15, 2026	SD's first run of `row6-setup.py` could not create the product repository: HTTP 403 "User is not allowed to create repositories in this organization" (Section 3.2.6).	Expected under least privilege. BD, as owner, ran her stages first; SD's rerun opened pull request #8.
October 15, 2026	A manual build failed: a single-character ellipsis in captured `docker compose ps` output could not be line-broken by `minted`.	The character is written as three dots in that evidence file, and the file's header says so.

Accepted limitations

- No backup exists; the persistence test is not a backup (Section 3.2.7). The extra credit was not attempted.
- The peer reviews in row 6 were light (Section 3.2.6).
- MFA statuses in the Passwords table are as reported by each account owner and were not independently checked.

3.3 Extra Credit

3.3.1 Extra credit: verified backup and restore (up to 20 extra-credit points; not attempted)

Status: not attempted. The group did not carry out the optional backup and restore, and no backup, restore, or result is claimed here. The persistence test in the database subsection (a repository and an issue surviving the replacement of the Gitea container) is a different test and is not a backup.

Chapter 4

Automated Manual Build

4.1 Overview and Status

This chapter answers the course manual's Chapter 3 [1]: every push of this manual's LaTeX source to the team's Gitea repository builds the PDF on a self-managed Gitea runner and publishes it as a downloadable artifact whose title page and footers carry the commit and run that produced it. The eight numbered subsections match the eight rubric rows in order.

Table 4.1: Rubric rows, owners, and status

Subsection	Points	Owner	Status
Source and trigger	15	BD	**PASS**
Runner setup	15	BD	**PASS**
Reproducible compilation	20	SD	**PASS**
PDF version traceability	15	SD	**PASS**
Downloadable artifact	10	BD	**PASS**
Change, failure, and repair evidence	10	SD	**PASS**
Runner and secret boundary	5	BD	**PASS**
Manual quality and controls	10	SD	**PASS**

4.2 Required Subsections

4.2.1 Source and trigger (15 points)

Owner: BD. **Status:** two push-triggered runs observed on October 19–20, 2026 **PASS**.

What is where. The source of this manual lives in the Gitea repository `group1-startup/manual` (public, created by BD on October 19, 2026). The main file is `devopsITManual.tex`; the compile command is the `latexmk` command in the workflow `.gitea/workflows/build.yml`.

The workflow runs on a `push` to the branch `main` (and on a manual start). The repository's two members, BD (organization owner) and SD (collaborator with write access), push over HTTPS; the course's Chapter 2 repository `twintrack` is the product, and this repository holds the manual (Table 4.2).

Table 4.2: Network path of a push and of a runner job, with public and private ports

Step	Port	Detail
`git push` from a laptop	**443/TCP, public**	HTTPS remote `https://git.ssw590-g1.tech/group1-startup/manual.git`; the AWS security group admits 443 from anyone.
TLS termination	443 (Caddy)	Caddy terminates TLS with the Let's Encrypt certificate and forwards the request to Gitea.
Caddy to Gitea	**3000/TCP, private**	Only on the private Docker network `gitea_web`; not published to the host and not reachable from the internet (Chapter 3).
Gitea Git over SSH	none	Disabled (`DISABLE_SSH`); HTTPS-only Git.
Runner to Gitea	443 (outbound)	The runner calls `https://git.ssw590-g1.tech` like a browser does. **No inbound port exists for the runner.**

Why the runner needs no public submission port. The runner works by pulling: it connects out to Gitea over HTTPS, asks for waiting jobs, and reports the log and result back over the same connection. Gitea never opens a connection to the runner, and a push goes to Gitea, not to the runner. The runner container therefore publishes no port at all (the `Ports` column of `docker ps` is empty for it, and the only published ports on the host remain Caddy's 80 and 443).

A run started by a push. SD's first push, commit `1fb4e15`, started run #1, and her second push, commit `a5f3446`, started run #2; both appear in the repository's Actions list (Figure 4.1). Run #2's summary shows "Triggered via push" on `main` and `build.yml (on: push)` (Figure 4.2). Run #1 compiled the manual but failed at the upload step, which is explained with the other failed attempts in Section 4.2.3.

4.2.2 Runner setup (15 points)

Owner: BD. **Status:** runner online and two jobs run on it, October 19–20, 2026 **PASS**.

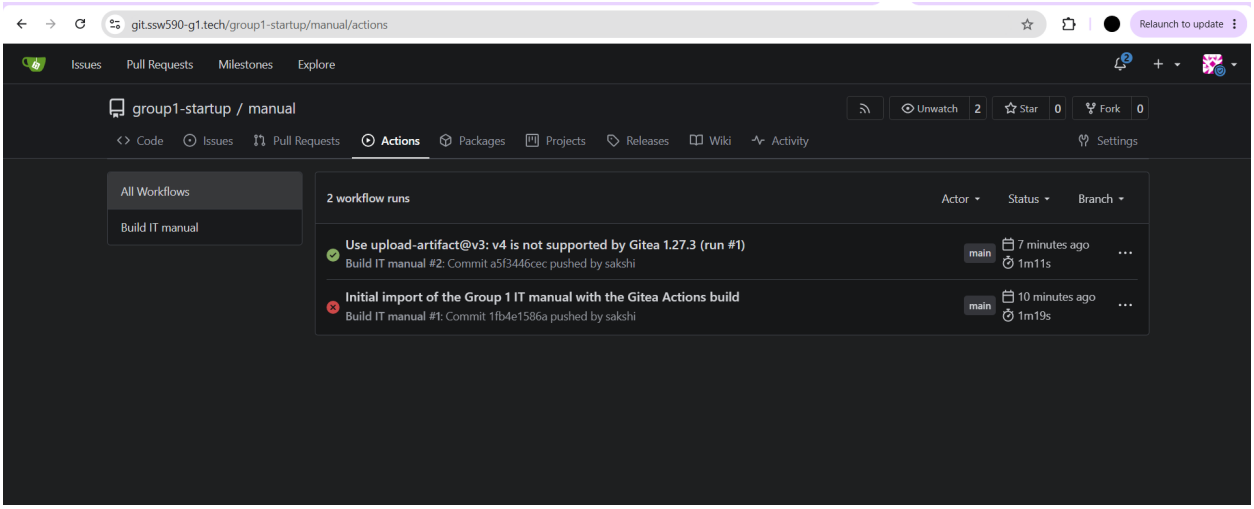


Figure 4.1: Actions tab of `group1-startup/manual`, October 19, 2026 (evening, US Eastern): run #1 for commit `1fb4e15` failed, run #2 for commit `a5f3446` succeeded; both were started by a push to `main`.

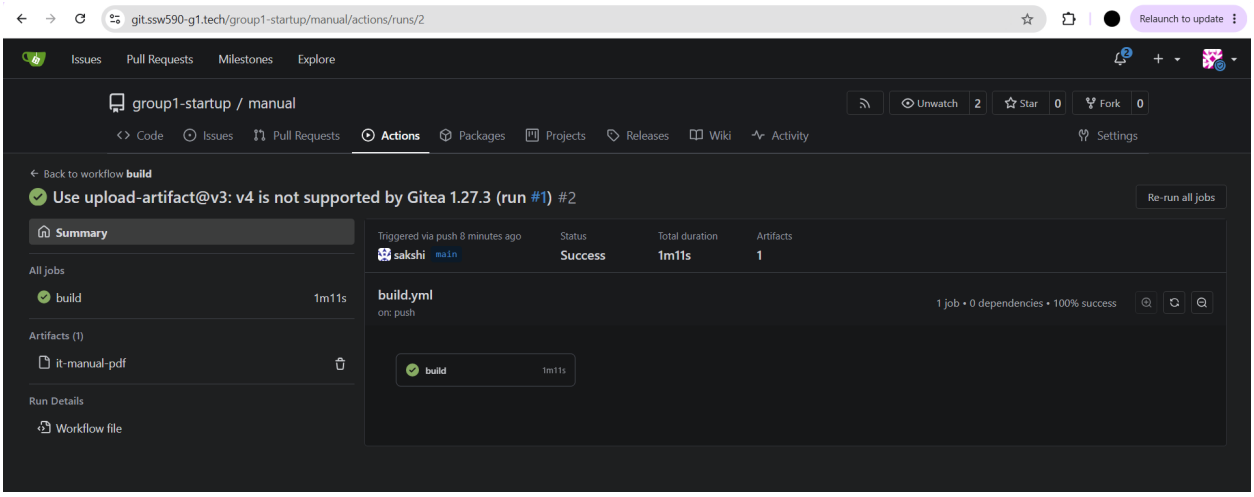


Figure 4.2: Run #2 summary: triggered via push, status Success, duration 1 minute 11 seconds, workflow `build.yml` (`on: push`), one artifact `it-manual-pdf`.

Table 4.3: The Gitea Actions runner

Property	Value
Name and version	`group1-runner-1`, `act_runner` v0.6.1 (Docker image `gitea/act_runner:0.6.1`, pinned) [15]
Location	A Docker container named `gitea-runner` on the same EC2 `t3.micro` host as Gitea (single-host layout of the course's Figure 3.2)
Scope	Global: registered with the instance-level registration token from Site Administration, so it can serve any repository of this Gitea (Figure 4.3); only `group1-startup/manual` uses it
Label	`manual-builder:docker://group1/texlive-node:2026.09` [16]; the workflow selects it with `runs-on: manual-builder`
Execution mode	`docker://`: every job runs in a temporary job container created from the label's image, not directly in the runner
Job image	Built on the host from `ci/Dockerfile` in the manual repository (TeX Live 2026 pinned by digest plus Node.js), 5.82 GB; it is local only, so the runner is configured with `force_pull: false` and never downloads it
Network path	Runner and job container → `https://git.ssw590-g1.tech` (public name, port 443). The host reaches its own public address; this was checked before installing (HTTP 200 on `/api/healthz`)
Capacity	One job at a time (`capacity: 1`) because the host has 2 vCPUs, about 1 GB of RAM and 2 GB of swap; 20 GB disk with about 9.4 GB free after the image
Restart behavior	`restart: unless-stopped`; the registration is kept in the Docker volume `runner_runner_data`
Published ports	None

Decision: same host. Three options were compared on October 19, 2026: the runner on the Gitea host (chosen), a larger instance, or a laptop. The Gitea host costs nothing extra and stays online for the instructor and TA; the risks are the small memory and the shared disk. Observed: run #2 took 1 minute 11 seconds, Gitea stayed healthy, and swap use was about 110 MB afterwards.

Registered and online, with a job assigned. The runner shows as *Idle* with its label in the administration page (Figure 4.3). After SD's push, the first lines of run #2's *Set up job* log show the runner receiving the job and starting the pinned image without pulling it (Figure 4.4): `group1-runner-1(version:v0.6.1) received task 2 of job build, be triggered by event: push`, `forcePull=false`, `Image exists? true`, then a new container on its own network. The container is removed after the job.

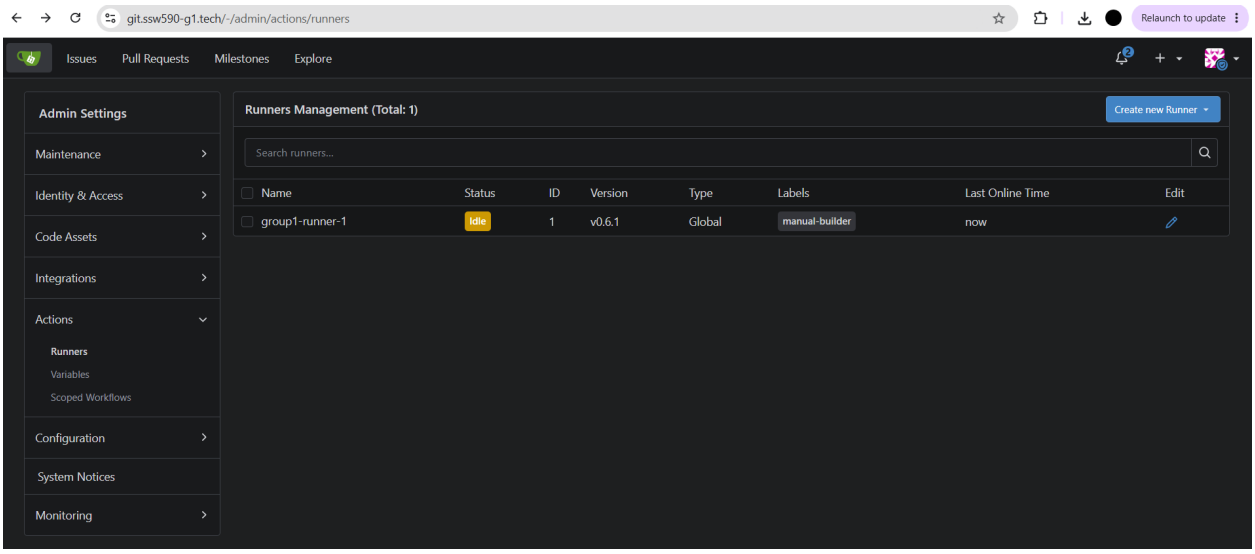


Figure 4.3: Site Administration, Actions, Runners, October 19, 2026: `group1-runner-1`, version v0.6.1, type Global, label `manual-builder`, status Idle, last online "now".

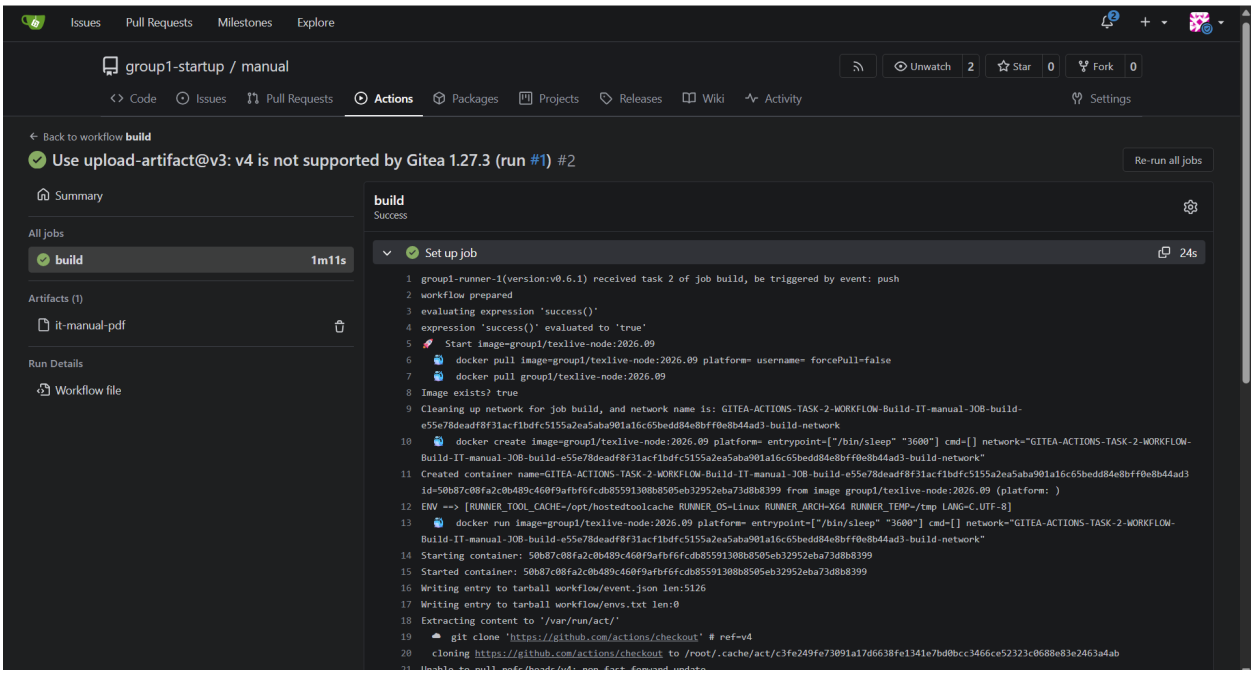


Figure 4.4: Log of run #2, step *Set up job*: the job is assigned to `group1-runner-1`, runs in a temporary container from `group1/texlive-node:2026.09`, and the image is not pulled (`forcePull=false`). The first action fetched is `actions/checkout` from github.com.

Isolation and ports. The runner container publishes no port; the only host ports open to the internet are still 80 and 443 on Caddy. Jobs run in their own containers on a fresh network per job; the runner is not run in privileged mode, and no host volumes are allowed for jobs (the configuration sets no `valid_volumes`). Whether a workflow can still reach the Docker socket was *not* tested; the risk is discussed in Section 4.2.7.

```yaml
# Gitea Actions runner (act_runner) for git.ssw590-g1.tech, same EC2 host as Gitea.
# Non-secret. The registration token lives only in runner.env (chmod 600, emptied after registration).
# The runner publishes NO ports; it only makes outbound HTTPS calls to Gitea.
# It mounts the Docker socket to start job containers: root-equivalent on this host, so only
# BD and SD may edit workflows (see the runner and secret boundary section of the manual).
services:
  runner:
    image: gitea/act_runner:0.6.1
    container_name: gitea-runner
    restart: unless-stopped
    env_file: runner.env
    environment:
      CONFIG_FILE: /config.yaml
      GITEA_INSTANCE_URL: https://git.ssw590-g1.tech
      GITEA_RUNNER_NAME: group1-runner-1
      GITEA_RUNNER_LABELS: "manual-builder:docker://group1/texlive-node:2026.09"
    volumes:
      - runner_data:/data
      - ./config.yaml:/config.yaml:ro
      - /var/run/docker.sock:/var/run/docker.sock

volumes:
  runner_data:
```

Listing 13: Runner Compose file (non-secret; also in the repository's `config/runner/` folder). The registration token is read from a root-only file that is emptied after registration.

```yaml
# act_runner configuration (non-secret).
log:
  level: info

runner:
  file: /data/.runner
  capacity: 1          # one job at a time: the host has about 1 GB RAM
  timeout: 1h

cache:
  enabled: false       # artifacts go through Gitea, not the runner cache

container:
  network: ""          # default: a fresh network per job
  privileged: false
  force_pull: false    # the job image is local-only (built on this host); never re-download it
  force_rebuild: false
```

Listing 14: Runner configuration (non-secret): one job at a time, no cache server, no privileged jobs, never re-pull the local image.

Failed attempts and corrections. Registration took four attempts on October 19, 2026; the first three failed. Attempt 1 was rejected: Gitea answered "runner registration token not

found" and the runner retried in a loop; the container was stopped and the token file on the server was emptied. The script was changed to report how many characters it received, and attempt 2 showed the cause: only *one* character of the 40-character token arrived, because pasting into a hidden terminal prompt kept only the first character. The script was changed again to read the token one key at a time, but attempt 3 still read one character; the script stopped on the administrator's machine before sending anything. The third change reads the token from the clipboard (which holds the full 40 characters, as checked with a length-only command) and refuses to send anything shorter than 20 characters. Attempt 4 printed `Runner registered successfully`. Afterwards the token file was emptied again (0 bytes, mode 600) and the runner restarted without it. The token itself is never written in this manual.

4.2.3 Reproducible compilation (20 points)

Owner: SD. **Status:** successful build on the Gitea runner, run #2, October 20, 2026, 03:00 UTC **PASS**.

Toolchain

Table 4.4: Build toolchain

Item	Value
Job image	`group1/texlive-node:2026.09`, built from `ci/Dockerfile`
Base image	`texlive/texlive`, pinned by digest `sha256:7334b00bf8e7...3083ff` (image created September 20, 2026)
TeX	TeX Live 2026, pdfTeX 3.141592653-2.6-1.40.29, `latexmk` 4.88
Other tools in the image	Python 3 with Pygments (`minted`), `bibtex`, `makeglossaries`, `makeindex`, `git`
Added for Actions	Node.js 24 (Debian package), needed by `actions/checkout` and `actions/upload-artifact`
Main file	`devopsITManual.tex` at the repository root
Build command	`latexmk -pdf -shell-escape -interaction=nonstopmode -halt-on-error -usepretex=... devopsITManual.tex`
Expected output	`devopsITManual.pdf`, copied to `out/IT-manual+build.<run>.<sha>.pdf`

TeX is installed only in the job image, which is stored in the server's Docker engine; it is not in the Gitea container and not in the server's own operating system. The runner starts a fresh container from that image for each job and removes it afterwards.

Workflow

```yaml
# Builds the Group 1 DevOps IT manual on every push to main and publishes the PDF as an
↪  artifact.
# Runs on the team's self-managed Gitea runner; the label below must map to the job image
↪  built
# from ci/Dockerfile (TeX Live pinned by digest + Node.js for the JavaScript actions).
name: Build IT manual

on:
  push:
    branches: [main]
  workflow_dispatch:

jobs:
  build:
    runs-on: manual-builder
    timeout-minutes: 30
    defaults:
      run:
        shell: bash
    steps:
      - name: Check out the triggering commit
        uses: actions/checkout@v4

      - name: Record the toolchain
        run: |
          set -euo pipefail
          pdflatex --version | head -n 1
          latexmk --version | tail -n 1
          pygmentize -V
          node --version

      - name: Compute the build identifier
        id: buildid
        run: |
          set -euo pipefail
          short="$(printf '%s' "${{ gitea.sha }}" | cut -c1-7)"
          echo "short=${short}" >> "$GITHUB_OUTPUT"
          echo "suffix=+build.${{ gitea.run_number }}.${short}" >> "$GITHUB_OUTPUT"
          echo "Build identifier: +build.${{ gitea.run_number }}.${short} (attempt ${{
          ↪  gitea.run_attempt }})"

      - name: Compile the manual (fails on any LaTeX error)
        run: |
          set -euo pipefail
          latexmk -pdf -shell-escape -interaction=nonstopmode -halt-on-error \
            -usepretex='\def\BuildSuffix{${{ steps.buildid.outputs.suffix
            ↪  }}}\def\BuildRun{${{ gitea.run_number }}}\def\BuildCommit{${{
            ↪  steps.buildid.outputs.short }}}' \
            devopsITManual.tex

      - name: Verify the PDF exists and carries the build identifier
```

```
      run: |
        set -euo pipefail
        test -s devopsITManual.pdf || { echo "::error::devopsITManual.pdf is missing or
        ↪  empty"; exit 1; }
        if grep -E "undefined references|multiply defined" devopsITManual.log; then
          echo "::error::unresolved or duplicate references in the final pass"; exit 1
        fi
        grep -F "Build identifier: Version" devopsITManual.log
        grep -qF "${{ steps.buildid.outputs.suffix }}" devopsITManual.log \
          || { echo "::error::build identifier not found in the LaTeX log"; exit 1; }
        mkdir -p out
        cp devopsITManual.pdf "out/IT-manual${{ steps.buildid.outputs.suffix }}.pdf"
        ls -l out

    # upload-artifact@v4 is rejected by Gitea 1.27.3 ("GHESNotSupportedError", run #1);
    ↪  v3 is supported.
    - name: Upload the PDF artifact
      uses: actions/upload-artifact@v3
      with:
        name: it-manual-pdf
        path: out/*.pdf
        if-no-files-found: error
```

Failure detection

The build fails, and no artifact is uploaded, when:

- `pdflatex` reports any error: `-halt-on-error` stops it and `latexmk` exits non-zero;
- `devopsITManual.pdf` is missing or empty;
- the final pass still reports undefined or multiply defined references;
- the build identifier is not in the LaTeX log (row 4);
- no PDF matches the upload path (`if-no-files-found: error`).

Before pushing, the same commands were run in the same image on SD's workstation (`ci/local-build.sh`): a clean build took about 18 seconds, and a copy with an undefined command inserted exited with code 1, reported "Undefined control sequence" with the line number, and produced no PDF.

Checkout and artifact path

The runner polls Gitea over HTTPS for work. In the job container, `actions/checkout` fetches the triggering commit from `https://git.ssw590-g1.tech`, and `actions/upload-artifact` sends the PDF back to the same run over HTTPS. Gitea and the job share no filesystem.

Runs on the server

The first push (commit `1fb4e15`, run #1) compiled the manual but failed at the upload step: Gitea 1.27.3 rejects `actions/upload-artifact@v4` with "GHESNotSupportedError". Version 3 of the action is supported, so commit `a5f3446` changed only that line, and run #2

succeeded (Figure 4.5, Listing 15). The compile step took 36 seconds on the `t3.micro` server, and the whole job 1 minute 11 seconds, including 24 seconds to start the job container. The run pages and the full job logs are public, so they can be opened without an account.

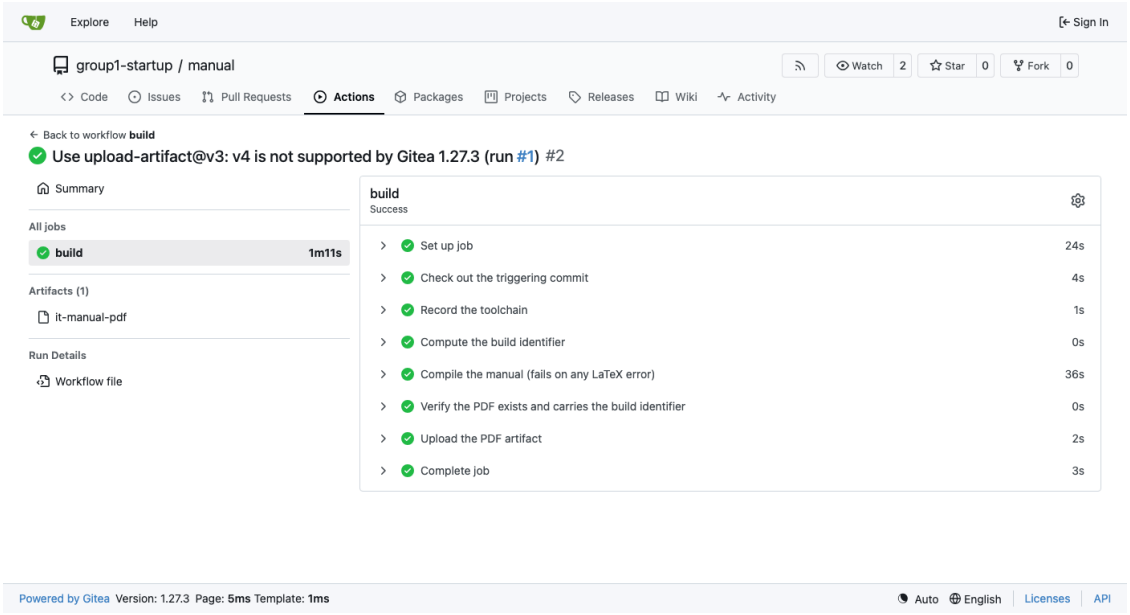


Figure 4.5: Run #2 on the Gitea runner, October 20, 2026, seen signed out: every step succeeded and the run has one artifact, `it-manual-pdf`.

```
Gitea Actions run #2, read anonymously 2026-10-20 03:09:31 UTC
commit a5f3446  event push  runner group1-runner-1  label manual-builder
started 2026-10-20T02:59:52Z  finished 2026-10-20T03:01:03Z  conclusion success
steps:
  Check out the triggering commit                        success  03:00:16 -> 03:00:20
  Record the toolchain                                   success  03:00:20 -> 03:00:21
  Compute the build identifier                           success  03:00:21 -> 03:00:21
  Compile the manual (fails on any LaTeX error)          success  03:00:21 -> 03:00:57
  Verify the PDF exists and carries the build identifier success  03:00:58 -> 03:00:58
  Upload the PDF artifact                                success  03:00:58 -> 03:01:00

Key lines from the job log (timestamps UTC):
  03:00:20 pdfTeX 3.141592653-2.6-1.40.29 (TeX Live 2026)
  03:00:20 Latexmk, John Collins, 9 March 2026. Version 4.88
  03:00:21 Pygments version 2.20.0, (c) 2006-present by Georg Brandl, Matthaus Chajdas and contributors.
  03:00:21 v24.21.0
  03:00:21 Build identifier: +build.2.a5f3446 (attempt 1)
  03:00:21 Latexmk: This is Latexmk, John Collins, 9 March 2026. Version 4.88.
  03:00:23 Build identifier: Version 1.0+build.2.a5f3446
  03:00:29 Output written on devopsITManual.pdf (65 pages, 4095419 bytes).
  03:00:39 Output written on devopsITManual.pdf (86 pages, 4226429 bytes).
  03:00:48 Output written on devopsITManual.pdf (87 pages, 4228163 bytes).
  03:00:57 Output written on devopsITManual.pdf (87 pages, 4228180 bytes).
  03:00:58 -rw-r--r--. 1 root root 4228180 Oct 20 03:00 IT-manual+build.2.a5f3446.pdf
  03:00:59 With the provided path, there will be 1 file uploaded
  03:00:59 Artifact name is valid!
  03:01:00 Total size of all the files uploaded is 4076563 bytes
  03:01:00 Artifact has been finalized. All files have been successfully uploaded!
  03:01:00 The size of all the files that were uploaded is 4076563 bytes. This takes into account any gzip
  ↪  compression use
```

```
03:01:00 Artifact it-manual-pdf has been successfully uploaded!
03:01:03 Job succeeded
```

Listing 15: Run #2: step results from the Gitea API and key lines of the job log (toolchain versions, build identifier, compile passes, artifact upload), October 20, 2026.

4.2.4 PDF version traceability (15 points)

Owner: SD. **Status:** identifier verified on the PDF downloaded from run #2 **PASS**.

The human-maintained version stays `\DocumentVersion` in `devopsITManual.tex`, changed only by a person with a history entry. The workflow appends a build suffix `+build.<run_number>.<first 7 characters of the commit>`, taken from Gitea's `gitea.run_number` and `gitea.sha` [17]. It reaches LaTeX through `latexmk -usepretex`, which defines `\BuildSuffix`, `\BuildRun`, and `\BuildCommit` before the main file is read. The main file provides `+local` as the default, so a local build is clearly unofficial. The suffix appears on the title page and in every page footer, and is written to the LaTeX log, where the workflow checks it.

Nothing is written back to the repository: the identifier exists only inside the generated PDF and the run's log, so a build cannot trigger another push. This PDF itself shows **Version 1.1+build.10.f8a6179**.

Evidence

Table 4.5: One build traced from the run to the PDF

Item	Value (run #2)
Human-maintained version	`1.0` (`\DocumentVersion`)
Gitea run	#2, attempt 1 (Figure 4.5)
Triggering commit	`a5f3446`
Identifier in the job log	`Build identifier: Version 1.0+build.2.a5f3446` (Listing 15)
Artifact file name	`IT-manual+build.2.a5f3446.pdf`
Identifier printed in the PDF	`Version 1.0+build.2.a5f3446` on the title page (Figure 4.6) and in every page footer

A re-run of the same run keeps its run number; the job log also records the attempt number ("attempt 1" in Listing 15), which tells re-runs apart.

4.2.5 Downloadable artifact (10 points)

Owner: BD. **Status:** artifact of run #2 downloaded without signing in and opened, October 19–20, 2026 **PASS**.

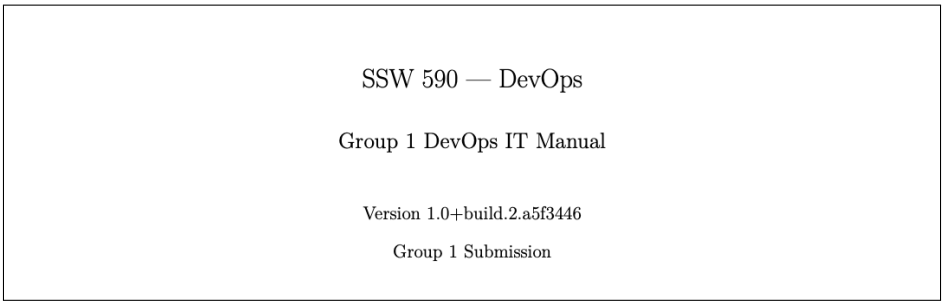


Figure 4.6: Title page of the PDF downloaded from run #2's artifact, October 20, 2026: the version reads `1.0+build.2.a5f3446`, matching run #2 and commit `a5f3446`.

Where it is. The workflow's last step uploads `out/*.pdf` as the artifact `it-manual-pdf` (the step fails if no file matches, so a missing PDF cannot pass silently). On the successful run #2 the artifact is listed under *Artifacts* in the left column of the run summary (Figure 4.2). A team member opens the `group1-startup/manual` repository, chooses *Actions*, opens the run, and clicks the artifact name; the browser downloads a zip file that holds the PDF. A direct link, which also works without signing in because the repository is public, is https://git.ssw590-g1.tech/group1-startup/manual/actions/runs/2/artifacts/it-manual-pdf.

Downloaded and opened. The download of run #2 is a 4.0 MiB zip with one file, `IT-manual+build.2.a5f3446.pdf`. Opened in a PDF viewer, its title page reads `Version 1.0+build.2.a5f3446`, the same run number and commit as the run (Figure 4.7). An anonymous download (no login, no cookie) from the administrator's machine on October 20, 2026 at 03:16 UTC returned HTTP 200, 4,087,975 bytes, content type `application/zip`, and the same file name (Listing 16).

```
Real output. Run by Claude Code on Brahmi's go-ahead from Brahmi's Windows machine, with no cookie and no
↪  login.
Purpose: show that the artifact of run #2 can be downloaded without signing in (the repository is public).
Time: 2026-10-20 03:16 UTC

$ curl -sL -o art.zip -w "HTTP %{http_code} bytes %{size_download} type %{content_type}"
↪  https://git.ssw590-g1.tech/group1-startup/manual/actions/runs/2/artifacts/it-manual-pdf
HTTP 200 bytes 4087975 type application/zip

$ unzip -l art.zip
Archive:  /tmp/anon2/art.zip
  Length      Date    Time    Name
---------  ---------- -----   ----
  4228180  1980-00-00 00:00   IT-manual+build.2.a5f3446.pdf
---------                     -------
  4228180                     1 file
```

Listing 16: Anonymous download of the run #2 artifact, October 20, 2026, 03:16 UTC (real output): HTTP 200, a zip that contains the versioned PDF.

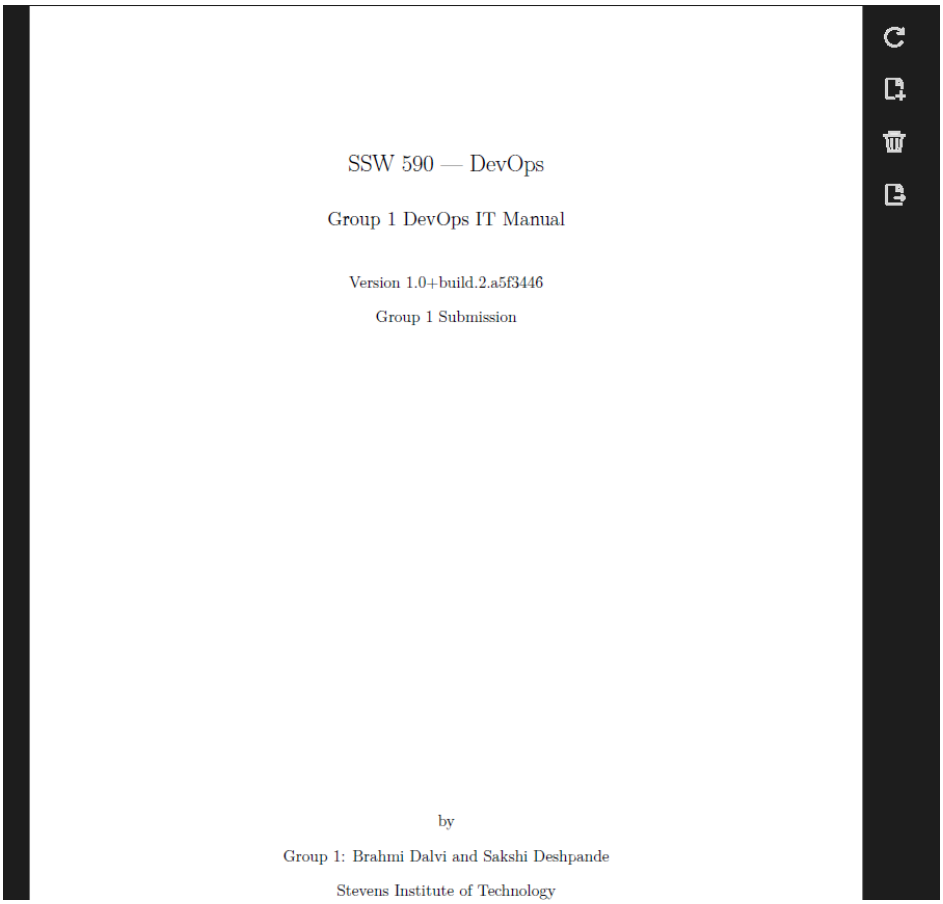


Figure 4.7: Title page of the PDF downloaded from the artifact of run #2 and opened in a viewer: `Version 1.0+build.2.a5f3446`.

Retention. No retention setting was configured by the team. The observed default is shown in the artifact's tooltip: the artifact of run #2 *expires at January 17, 2027, 22:01* (Figure 4.8), about 90 days after the run, and is 4.0 MiB. This is the observed value on this Gitea 1.27.3 installation, not a documented guarantee.

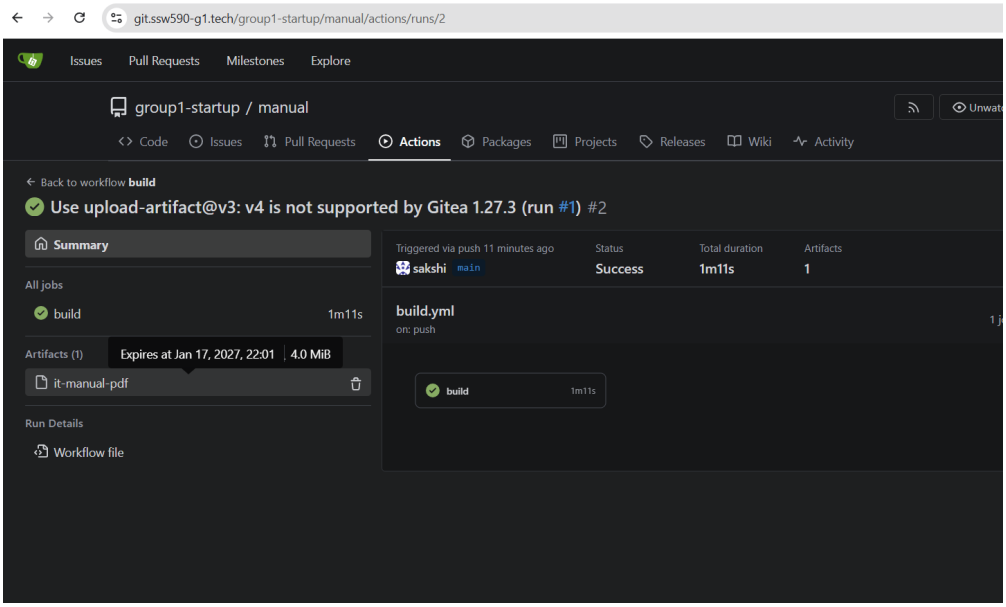


Figure 4.8: Run #2 summary with the tooltip of `it-manual-pdf`: "Expires at Jan 17, 2027, 22:01" and 4.0 MiB.

An artifact is not a backup. Artifacts expire and live on the same disk as Gitea; deleting the run or the repository removes them, and the server has no backup (the optional backup test of Chapter 2 was not attempted). The permanent copies are the LaTeX source in Git (which can rebuild any version from its commit) and the PDF that the team submits to the course. The artifact is a delivery path, not storage.

4.2.6 Change, failure, and repair evidence (10 points)

Owner: SD. **Status:** two successive changes, one controlled failure, and its repair, all built on the Gitea runner on October 20, 2026 **PASS**.

Each row is a separate push to `main`. The PDF identifier was read from the title page of the artifact downloaded from that run (Listing 18); times are UTC.

Table 4.6: Successive changes, a controlled failure, and its repair, October 20, 2026

Step	Change	Commit	Run / result	PDF identifier
Change 1	Row 3 text and evidence added	c59a784	#3 success, 03:10	1.0+build.3.c59a784
Change 2	Row 4 text and evidence added	34ae070	#4 success, 03:11	1.0+build.4.34ae070
Failure	One line \ControlledFailureForRowSix (an undefined command) added on purpose	dcdf778	#5 **failure**, 03:13	none: no PDF, no artifact
Repair	That line removed	a9636e4	#6 success, 03:13	1.0+build.6.a9636e4

Table 4.7: Controlled failure: expected and actual results

Check	Expected	Actual (run #5)
Compile step	fails at the bad line	failed: "! Undefined control sequence" at l.199 \ControlledFailureForRowSix; latexmk exit code 12
Later steps	not run	"Verify" and "Upload" skipped
Artifact	none	none (run page shows no artifact; download returns nothing)
Run status	failure	failure, in 14 seconds (Figure 4.9)
Gitea service	unaffected	unaffected: the failing commit changed only the LaTeX source, and the repair run #6 started 40 seconds later and succeeded

```
Gitea Actions run #5, read anonymously 2026-10-20 03:13:48 UTC
commit dcdf778  event push  runner group1-runner-1  label manual-builder
started 2026-10-20T03:13:02Z  finished 2026-10-20T03:13:16Z  conclusion failure
steps:
  Check out the triggering commit                        success  03:13:04 -> 03:13:06
  Record the toolchain                                   success  03:13:06 -> 03:13:07
  Compute the build identifier                           success  03:13:07 -> 03:13:07
  Compile the manual (fails on any LaTeX error)          failure  03:13:07 -> 03:13:14
  Verify the PDF exists and carries the build identifier skipped  03:13:15 -> 03:13:15
  Upload the PDF artifact                                skipped  03:13:15 -> 03:13:15
```

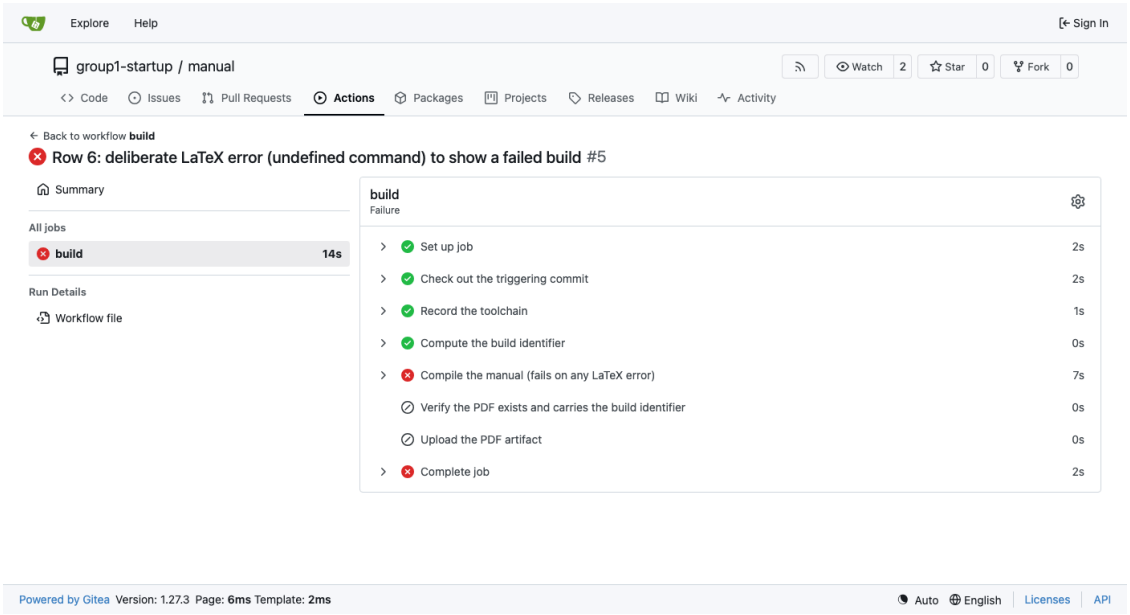


Figure 4.9: Run #5, October 20, 2026, seen signed out: the compile step failed, and the verify and upload steps were skipped.

```
Key lines from the job log (timestamps UTC):
  03:13:06 pdfTeX 3.141592653-2.6-1.40.29 (TeX Live 2026)
  03:13:07 Latexmk, John Collins, 9 March 2026. Version 4.88
  03:13:07 Pygments version 2.20.0, (c) 2006-present by Georg Brandl, Matthaus Chajdas and contributors.
  03:13:07 v24.21.0
  03:13:07 Build identifier: +build.5.dcdf778 (attempt 1)
  03:13:07 Latexmk: This is Latexmk, John Collins, 9 March 2026. Version 4.88.
  03:13:09 Build identifier: Version 1.0+build.5.dcdf778
  03:13:14 ! Undefined control sequence.
  03:13:14 l.199 \ControlledFailureForRowSix
  03:13:14 !  ==> Fatal error occurred, no output PDF file produced!
  03:13:14 Failure - Main Compile the manual (fails on any LaTeX error)
  03:13:15 exitcode '12': failure
  03:13:16 Job failed
```

Listing 17: Run #5: step results and the LaTeX error in the job log, October 20, 2026.

```
Runs of group1-startup/manual, read anonymously (no token) 2026-10-20 03:15:08 UTC
Artifact checked by downloading it-manual-pdf and reading the PDF title page.

run  commit   result   started (UTC)         artifact file                    title-page version
#1   1fb4e15  failure  2026-10-20 02:56:10   (none)                           -
#2   a5f3446  success  2026-10-20 02:59:52   IT-manual+build.2.a5f3446.pdf    Version 1.0+build.2.a5f3446
#3   c59a784  success  2026-10-20 03:10:16   IT-manual+build.3.c59a784.pdf    Version 1.0+build.3.c59a784
#4   34ae070  success  2026-10-20 03:11:45   IT-manual+build.4.34ae070.pdf    Version 1.0+build.4.34ae070
#5   dcdf778  failure  2026-10-20 03:13:02   (none)                           -
#6   a9636e4  success  2026-10-20 03:13:56   IT-manual+build.6.a9636e4.pdf    Version 1.0+build.6.a9636e4

Commit messages:
  1fb4e15  Initial import of the Group 1 IT manual with the Gitea Actions build
  a5f3446  Use upload-artifact@v3: v4 is not supported by Gitea 1.27.3 (run #1)
  c59a784  Row 3: record the first server builds (runs #1 and #2) with evidence 61-62
  34ae070  Row 4: trace run #2 to its PDF identifier (evidence 63)
```

```
dcdf778  Row 6: deliberate LaTeX error (undefined command) to show a failed build
a9636e4  Row 6: repair the deliberate LaTeX error from dcdf778
```

Listing 18: Runs #1–#6 with commit, result, artifact file name, and the version printed on the downloaded PDF's title page, read without an account, October 20, 2026.

4.2.7 Runner and secret boundary (5 points)

Owner: BD. **Status:** described from the observed configuration; the Docker-socket limits below were not tested **PASS**.

Who can edit the workflow. A workflow is code that the runner executes, so whoever can push to `group1-startup/manual` controls what runs. Two people can: BD (organization owner) and SD (collaborator with *Write*, Figure 4.10). The team `developers` also has the Actions unit, and registration on this Gitea is closed (`DISABLE_REGISTRATION`), so no outsider can create an account, fork the repository, or open a pull request. **Limitation:** `main` of this repository has no branch protection (unlike `twintrack`), so either member can push a workflow change that runs immediately without review; this is accepted for a two-person team and was not changed.

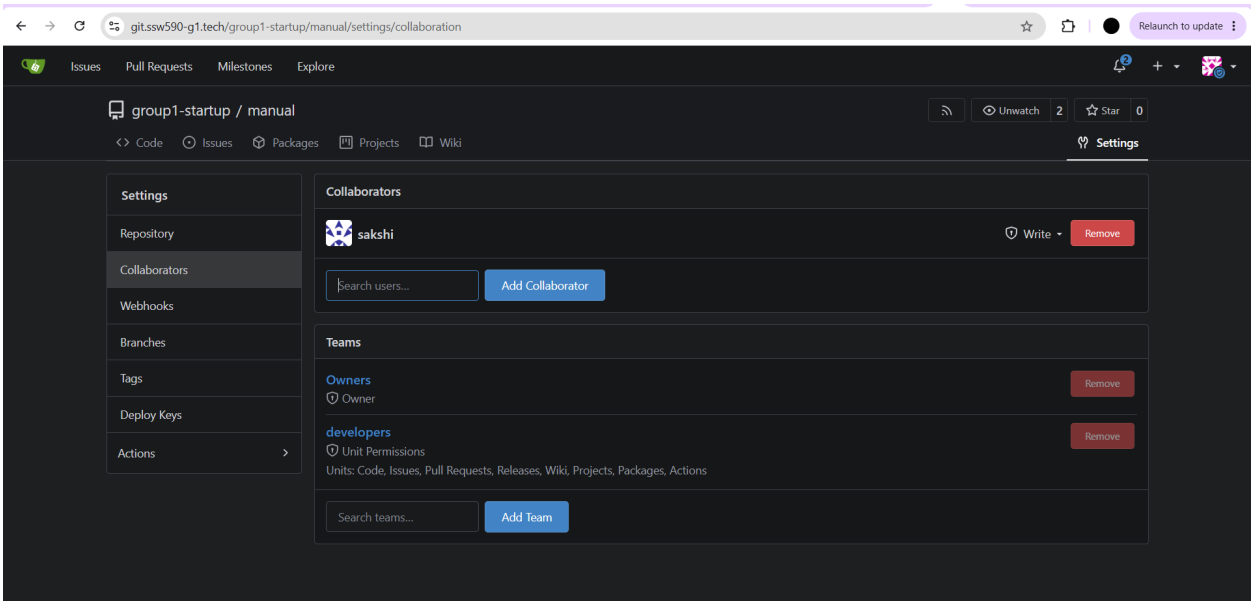


Figure 4.10: Settings, Collaborators of `group1-startup/manual`, October 19, 2026: `sakshi` with Write; teams `Owners` (Owner) and `developers` (unit permissions Code, Issues, Pull Requests, Releases, Wiki, Projects, Packages, Actions).

Where the secrets are.

- *Runner registration token* (hint only): shown in Gitea's Site Administration, Actions, Runners page; BD copies it into the clipboard and a script on BD's machine sends it over SSH to a root-only file on the server (mode 600). After a successful registration the file was emptied (0 bytes) and the runner was restarted without it; the registration itself is kept in the Docker volume. The token is not in the repository, the workflow, the logs, or this PDF. If it leaks, reset it on the same page.
- *Repository secrets:* none. The workflow uses no secret and no access token of its own.
- *Gitea access token* used once to create the repository: deleted right after use.

Docker privilege risk. The runner container mounts the host's Docker socket (Listing 13) so that it can create job containers. Anyone who controls the runner process can start any container on the host, which also holds Gitea's data volume; that is root-equivalent on this host, the same host that serves the team's Gitea [15]. The jobs themselves are not given the socket: the configuration sets `privileged: false` and no `valid_volumes`, which by design keeps a workflow from mounting host paths. *This was not tested*, so the manual does not claim it as verified. Compensating controls: only two trusted people can edit workflows, there are no outside accounts, no repository secrets exist to steal, and the runner publishes no port. A stronger design would use a separate runner host, which was not chosen because of cost.

Why untrusted workflow changes must not get privileged access or secrets. A pull request from a stranger could rewrite the workflow to run any command in the job, read every secret the job can see, and, with Docker access, attack the host and the other containers. Therefore this runner is registered only for this repository's trusted authors, holds no secrets, and must never be given privileged mode, host mounts, or secrets while outsiders can propose workflow changes.

Third-party code in the build. The workflow's two steps `actions/checkout` and `actions/upload-artifact` are fetched from github.com at the time of each run (the *Set up job* log, Figure 4.4, shows the clone), selected by a version tag that its owner can move. This is a supply-chain dependency that was accepted for simplicity; pinning each action to a commit hash is a possible improvement.

Non-secret configuration. The runner's Compose file and configuration are printed in Section 4.2.2 (Listings 13 and 14) and are kept in the repository under `config/runner/`. Hosts and Passwords tables: **updated** for the runner (Chapter 2).

4.2.8 Manual quality and controls (10 points)

Owner: SD. **Status:** final review of this chapter on October 20, 2026 **PASS**.

Table 4.8: Manual quality controls for this chapter, checked October 20, 2026

Control	Status	Where / how checked
Manual version increased	**PASS**	1.0 → 1.1, dated October 20, 2026; the build suffix is added automatically, the version itself only by a person
Dated history entry linking this chapter	**PASS**	Version 1.1 entry in Table 1
Hosts and Passwords tables	**PASS**	**Changed** (by BD): new Hosts row for the runner `gitea-runner` (same EC2 host, no published ports); new Passwords row for the runner registration token (hint only, not stored); SD's Gitea row now records her Write access to `group1-startup/manual`. Named in the history entry
Citations	**PASS**	Gitea Actions variables, runner labels, runner Docker installation, configuration (artifact retention), Let's Encrypt and Caddy sources in the Bibliography
Glossary and index terms	**PASS**	Gitea Actions, runner, artifact, job container in the Glossary; index entries for these and for the build identifier and controlled failure
Evidence inside the PDF, dated and sanitized	**PASS**	Evidence 50–58 (BD) and 60–66 (SD) as figures and listings. Scripted scan of every chapter source, printed configuration and command-output file for private keys, tokens, AWS keys and account numbers, passwords, e-mail addresses, and IPv4 addresses found only expected public or private addresses; every screenshot was inspected by eye
Links	**PASS**	Repository, workflow, runs, commits, and artifact links are public and open without an account
Failed-build troubleshooting procedure	**PASS**	Below; checked against the real failure of run #5
Build	**PASS**	This PDF was built by the workflow itself; its identifier is on the title page and in every footer

Failed-build troubleshooting

1. Open the failed run in Gitea (*Actions* tab) and expand the first red step.
2. Compile step failed: search the log for the first line starting with `!`; the following `l.<n>` gives the file line. Fix it and run `ci/local-build.sh` before pushing again.
3. "PDF is missing" or "build identifier not found": the compile ended early or

`-usepretex` was changed; compare with the workflow in Section 4.2.3.

4. Job never starts ("waiting for a runner"): the runner is offline or its label is not `manual-builder`; ask the runner owner (Section 4.2.2).
5. Checkout or upload fails: the runner cannot reach `https://git.ssw590-g1.tech`; check DNS and the Caddy container. An upload error naming "GHES" means an artifact action newer than Gitea supports (run #1).

Check against a real failure. For run #5, steps 1 and 2 led directly to the cause: the red step was "Compile the manual", and the log showed `! Undefined control sequence.` followed by `l.199 \ControlledFailureForRowSix` (Listing 17). Removing that line (commit `a9636e4`) gave the green run #6. Step 5 matches run #1.

Failed tests and corrective actions

Table 4.9: Failed tests and corrective actions in this chapter

When / who	What failed	Correction
October 19, 2026, BD	Runner registration, attempts 1–3: the token arrived truncated to one character (Section 4.2.2)	Read the token from the clipboard with a length check; attempt 4 registered; token file emptied afterwards
October 20, 2026, SD	Run #1: `upload-artifact@v4` rejected by Gitea 1.27.3 ("GHESNotSupportedError"; Section 4.2.3)	Switched to `@v3` (commit `a5f3446`); run #2 green
October 20, 2026, SD	The first screenshot tool run captured an empty page (the run page draws itself with JavaScript after loading)	Waited for rendering before the capture; no evidence used the empty image
October 20, 2026, SD	Evidence 63 (title page) rendered as a black box in the PDF of run #7: transparent background	Re-rendered on white (commit `1eaddcc`); run #8 checked
October 20, 2026, SD	Run #5: *controlled* failure, intended (Section 4.2.6)	Repaired by `a9636e4`; run #6 green

Bibliography

[1] D. Muresan, *DevOpsStartup: A Startup DevSecOps Assignment Manual, Version 1.18, Student Edition*, Stevens Institute of Technology, 2026, sSW 590 course manual (PDF, dosManual-V1.18.pdf); Chapters 1 and 2 govern this chapter.

[2] Docker, "Packet filtering and firewalls," https://docs.docker.com/engine/network/packet-filtering-firewalls/, cited from the course manual bibliography; re-verify before relying on it.

[3] Gitea, "Installation with docker," https://docs.gitea.com/installation/install-with-docker/, cited from the course manual bibliography; re-verify before relying on it.

[4] Caddy, "Automatic https," https://caddyserver.com/docs/automatic-https, cited from the course manual bibliography; re-verify before relying on it.

[5] GitHub Education, "Github student developer pack: Domain offers," https://education.github.com/pack?sort=az&tag=Domains, offer eligibility and terms must be checked at the time of use.

[6] Amazon Web Services, "New: AWS public IPv4 address charge and public IP insights," https://aws.amazon.com/blogs/aws/new-aws-public-ipv4-address-charge-public-ip-insights/, states a charge of 0.005 USD per public IPv4 address per hour from 2024-02-01. Taken from a search-result summary; confirm on the current pricing page. Accessed 2026-10-15.

[7] ——, "Migrate your Amazon EBS volumes from gp2 to gp3 and save up to 20% on costs," https://aws.amazon.com/blogs/storage/migrate-your-amazon-ebs-volumes-from-gp2-to-gp3-and-save-up-to-20-on-costs, source for the gp3 rate of 0.08 USD per GB-month in us-east-1, with third-party guides; the official pricing page was not read. Accessed 2026-10-15.

[8] Holori, "AWS t3.micro price, us-east-1," https://calculator.holori.com/aws/ec2/t3.micro/us-east-1, third-party calculator: 0.0104 USD per hour on demand. Not AWS's own page. Accessed 2026-10-15.

[9] Gitea, "Database preparation," https://docs.gitea.com/installation/database-prep/, cited from the course manual bibliography; accessed 2026-10-12.

[10] ——, "Configuration cheat sheet," https://docs.gitea.com/administration/config-cheat-sheet/, cited from the course manual bibliography; re-verify before relying on it.

[11] Name.com, "Domain safe (previously known as advanced security)," https://www.name.com/support/articles/360001351388-domain-safe-previously-known-as-advanced-security, support article, last updated 2026-06-09. Its body text could not be retrieved in full; the free-certificate statement comes from search-result summaries of it. Accessed 2026-10-15.

[12] Let's Encrypt, "Challenge types," https://letsencrypt.org/docs/challenge-types/, cited from the course manual bibliography; re-verify before relying on it.

[13] Name.com, "Renewing domain safe ssl certificates," https://www.name.com/support/articles/360020995114-renewing-domain-safe-ssl-certificates, support article. The 90-day renewal window is from a search-result summary of it. Accessed 2026-10-15.

[14] GitHub Community, "Namecheap domain, discussion 148485," https://github.com/orgs/community/discussions/148485, a student reports being asked to pay for the Namecheap SSL certificate after claiming the free domain; one report, not confirmed. Accessed 2026-10-15.

[15] Gitea, "Install gitea runner with docker," https://docs.gitea.com/runner/2/installation/docker/, listed as reference 14 of the course manual (Chapter 3); not separately opened by the authors.

[16] ——, "Gitea runner labels," https://docs.gitea.com/runner/labels/, listed as reference 13 of the course manual (Chapter 3); not separately opened by the authors.

[17] ——, "Gitea actions variables," https://docs.gitea.com/usage/actions/actions-variables/, accessed 2026-10-19.

[18] DigitalOcean, "How to configure firewall rules," https://docs.digitalocean.com/products/networking/firewalls/how-to/configure-rules/, cited from the course manual bibliography; re-verify before relying on it.

[19] Gitea, "Configuration cheat sheet: Actions," https://docs.gitea.com/administration/config-cheat-sheet/, aRTIFACT_RETENTION_DAYS defaults to 90; accessed 2026-10-19.

Appendix A

Document Authoring Guide

A.1 How to Use This Appendix

Use this appendix to maintain the template's file conventions, build process, and review rules. Update it when the document structure or toolchain changes.

A.2 Template Provenance

This template was developed by **Darian Muresan, Ph.D.**, in conjunction with **OpenAI Codex** and **Anthropic Claude Code**. Keep this attribution distinct from the assignment-specific instructor metadata.

A.3 File Naming and Responsibilities

Every chapter file shall use the `devops` prefix, a PascalCase chapter name, and the `.tex` suffix. Each such file shall contain one complete chapter, beginning with `\chapter{...}`, and all of its sections and subsections. Do not split one chapter into section-level `.tex` files. For example, all of Chapter 2 is contained in `devopsExampleAssignment.tex`. Do not add requirements or architecture-design chapters; this template records a DevOps learning activity and its operational evidence.

A.4 Synonyms and Controlled Terms

Use one preferred term for each important concept. If an assignment uses equivalent terms, record them here before release. For example, state whether "continuous delivery" refers to continuous delivery or continuous deployment (CD) generally or specifically excludes automatic production deployment. Add important terms to `devopsGlossary.tex` and index them where used.

A.5 Building the PDF

The template requires a LaTeX distribution with `pdflatex`, `bibtex`, `makeglossaries`, and `makeindex`. Install the tools using the platform instructions below, then run one of the following platform builds.

A.5.1 macOS and Linux Build

```
make -f makefileMacLinux pdf
```

A.5.2 Windows PowerShell Build

```
.\makefilePowerShell.ps1 -Action pdf
```

The completed build regenerates references, bibliography, glossary, and index. Review the log and visually inspect affected PDF pages. Do not edit generated auxiliary files or the PDF directly.

A.6 Install LaTeX Tools on macOS

A.6.1 Recommended Installation

Install the current full MacTeX distribution. It includes TeX Live, `pdflatex`, `bibtex`, `latexmk`, `makeglossaries`, `makeindex`, and useful applications such as TeXShop and TeX Live Utility. The full distribution is the least troublesome choice for this template because it includes the packages used by the glossary, table, listing, color, and hyperlink features.

Either download the signed installer from the MacTeX web site and complete the macOS installation dialog, or, if Homebrew is already installed, run:

```
brew install --cask mactex
```

Close and reopen Terminal after installation. If the TeX commands are not found, add the MacTeX binary directory to the current shell session:

```
export PATH="/Library/TeX/texbin:$PATH"
```

Do not place this command in a shell profile until it has been verified on the local computer.

A.6.2 Verification and Compilation

From the template directory, verify the tools:

```
command -v pdflatex bibtex latexmk makeglossaries makeindex
```

Each command should print a path. Then create the PDF:

```
make -f makefileMacLinux pdf
```

To remove generated intermediate files while keeping the PDF, run:

```
make -f makefileMacLinux clean
```

A.7 Install LaTeX Tools on Windows

A.7.1 Recommended Installation

Install the current Basic MiKTeX distribution using the official MiKTeX Windows installation guide. During installation, choose a per-user installation when appropriate and set missing-package installation to *Ask me first* or *Yes*. The latter is convenient for a student machine with Internet access; the former makes each package installation explicit.

Open MiKTeX Console after installation, apply available updates, and install the `latexmk` package if the verification command below cannot find it. On an organization-managed computer, obtain approval before installing software or changing execution policy.

A.7.2 Verification and Compilation

Open a new PowerShell window in the template directory and verify:

```
Get-Command pdflatex,bibtex,latexmk,makeglossaries,makeindex
```

Each command should display a command record. Build the PDF with:

```
.\makefilePowerShell.ps1 -Action pdf
```

If PowerShell blocks the script under a managed execution policy, ask the system administrator for the approved procedure. Do not bypass institutional security policy. To remove intermediate build files, run:

```
.\makefilePowerShell.ps1 -Action clean
```

A.8 Common Build Problems

Symptom	Resolution
Command not found	Close and reopen the terminal. On macOS, verify the TeX binary path; on Windows, verify MiKTeX completed installation and is on the user PATH.
Missing LaTeX package	Use MiKTeX Console or its approved on-the-fly package installation prompt on Windows. On macOS, install the full MacTeX distribution or use TeX Live Utility to add the missing package.
References or glossary incomplete	Run the supplied platform build instead of a single `pdflatex` command; it runs BibTeX, the glossary tool, and the index tool in the required order.
Script is blocked	Follow the institution's approved PowerShell policy or run the documented build commands with an approved administrator.

A.9 Authoring and Review Rules

Replace bracketed placeholders with assignment facts. Keep operational examples safe, bounded, and free of credentials. Link activities to evidence, retain history, and state unknowns or instructor decisions explicitly. Rebuild after structural, cross-reference, table, glossary, or index changes.

Appendix B

AI Agent Usage Guide

B.1 How to Use This Appendix

An AI agent that creates, reviews, or maintains an assignment shall read this file first. It shall identify the task in the routing table, read the minimum authoritative sources, and expand context only for a discovered dependency or inconsistency. This guide is maintained in LaTeX so it is visible, versioned, and reviewable with the assignment.

B.2 Governing Principles

The authored LaTeX files, versioned automation, and approved assignment materials are authoritative. The PDF is a review artifact and shall not be edited directly. AI may draft, explain, analyze, check consistency, and assist with builds. It shall not invent completion, evidence, access, costs, grades, approvals, command output, security posture, or operational results.

B.3 Chapter File Convention

Every assignment chapter shall be stored in exactly one file named with the `devops` prefix, a PascalCase chapter name, and the `.tex` suffix. For example, Chapter 2 is fully contained in `devopsExampleAssignment.tex`. A chapter file shall begin with its `\chapter{...}` command and contain all of that chapter's sections and subsections. Do not split one chapter across files, create section-only `.tex` files, or use `\input` or `\include` to assemble a chapter from smaller fragments.

Supporting non-chapter files may use a different name only when they are not chapter content: `prologue.tex`, `abstract.tex`, `devopsGlossary.tex`, and `devopsReferences.bib` are examples. Appendix chapters follow the same chapter-file convention.

B.4 AI Quick Start

1. Read this guide and identify the requested outcome.
2. Select the closest task below and load only its listed sources.
3. Inspect supplied facts and report consequential gaps or assumptions.
4. Make narrowly scoped changes; preserve stable IDs, glossary terms, and history.
5. Never request, print, store, or commit credentials, tokens, personal data, or production data.
6. Build the complete PDF, inspect the log, visually inspect affected pages, and record material AI use in Chapter C.

B.5 Task Routing

Table B.1: AI Task-Routing Table

Task	**Minimum authoritative sources**
Initialize an assignment	`devopsAssignment.tex`, `prologue.tex`, `devopsIntroduction.tex`, `devopsExampleAssignment.tex`, and the instructor's assignment facts.
Develop a workflow	`devopsExampleAssignment.tex`, source repository materials, and authorized environment constraints.
Define evidence	`devopsExampleAssignment.tex` and controlled execution outputs.
Review a retrospective	`devopsExampleAssignment.tex` and the submitted workflow or runbook.
Edit AI guidance	`devopsAIAgentGuide.tex`, `devopsAIPromptLog.tex`, `devopsDocumentGuide.tex`, and `devopsAssignment.tex`.
Compile or troubleshoot	`devopsAssignment.tex`, `devopsDocumentGuide.tex`, the affected chapter, and the build log.

B.6 DevOps-Specific Rules

1. Prefer versioned, repeatable automation over undocumented manual steps.
2. Treat deployment output, monitoring data, and screenshots as evidence only when they reflect an actual execution.
3. Design failure exercises to be isolated, reversible, and authorized.
4. Apply least privilege and never place secrets in LaTeX, source control, prompts, screenshots, or logs.

5. State expected cloud cost, cleanup action, and resource ownership for each remote-environment activity.

6. Identify whether a recommendation is a proposal, an instructor directive, or demonstrated evidence.

B.7 Example AI Requests

B.7.1 Create an Assignment Brief

```
Read devopsAIAgentGuide.tex and use the Initialize an Assignment context.
From the supplied SSW 590 objective and constraints, fill only supported
placeholders in the introduction and activity brief. Propose clear, observable
completion criteria, a safe cleanup action, and a rubric. Do not invent cloud
access, due dates, policy, costs, or approved tools. Update history, compile,
and report instructor decisions still needed.
```

B.7.2 Review a Student Workflow

```
Read devopsAIAgentGuide.tex and the workflow, evidence, and retrospective
chapters. Review the supplied repository and sanitized evidence for
reproducibility, least privilege, observability, rollback, and cleanup.
Separate verified facts from gaps and recommendations. Do not claim that an
unexecuted command passed, and do not request credentials or operate resources.
```

B.8 Human Decisions and Information Boundaries

The instructor retains authority over assignment scope, grading, due dates, approved tooling, access, budgets, security posture, exceptions, and policy. An AI agent shall surface alternatives and evidence, not represent a decision as approved. Record material prompts, assumptions, corrections, and unresolved issues in Chapter C.

B.9 Completion Checklist

- Scope, deliverables, environment, constraints, and cleanup are clear.
- Activity identifiers and evidence references are valid and unique.
- Examples contain no secrets, personal data, or unsafe production actions.
- New controlled terms are added to the glossary and indexed.
- The newest approved change is recorded in document history.
- The complete build succeeds with no unresolved references or missing files.

- A human reviews all decisions requiring instructional or operational authority.

Appendix C

AI Prompt Log

C.1 How to Use This Appendix

Record AI assistance that materially affects wording, automation, evidence interpretation, troubleshooting, or feedback: enough context for a reviewer to see the input, output, assumptions, and human corrections. Never record credentials or personal data.

C.2 Record: IT Manual Skeleton and Gitea Chapter Outline

Date: October 12, 2026; Tool and model: Claude Code (Claude Sonnet 5.5); Task: set up this separate IT manual and prepare the Gitea chapter before any infrastructure exists. Inputs: the course manual [1] (Version 1.18, Chapters 1 and 2 read in full; it is logged here as the governing source), and the conventions of the earlier `report/` document. Output used: the manual skeleton, the Gitea chapter outline with eight rubric-aligned subsections and an extra-credit subsection, the Hosts and Passwords tables, a draft cloud-agnostic Compose configuration under `gitea/`, and a work split between the two members. The assistant created no cloud resources, executed no deployment, and recorded no test results; every evidence item is **PENDING**. The first draft of the skeleton was derived from `report/` because the course template was not yet available; it was rebuilt from the Canvas template (version 1.3) after Brahmi Dalvi supplied it, which also required merging the per-section files into the single `devopsGitea.tex` chapter file the template's AI guide requires. Human decisions recorded: SQLite is the final database choice; Brahmi Dalvi owns the cloud deployment, DNS, HTTPS, and firewall work; the 20-point backup/restore extra credit is optional and unassigned (it was later not attempted). Human review and corrections: Brahmi Dalvi reviewed the skeleton, supplied the Canvas template to replace it, and set the decisions listed above. Each author's own review of the later drafts is recorded in that author's record below.

C.3 Record: SD Regions of the Gitea Chapter (Draft)

Date: October 12, 2026; Tool and model: Claude Code (Claude Opus 5.5); Task: carry out Phase A of SD's plan (`assignment-4/SAKSHI.md`) without any cloud resources: local rehearsal of the Gitea and Caddy stack, a persistence rehearsal, and drafts of the `giteaDatabase`, `giteaWorkflow`, `giteaArchitecture`, and `giteaManualQuality` regions. Inputs: the course manual [1] Chapters 1–2 and Appendix A, the team's `README.md` and `SAKSHI.md`, and the draft files in `gitea/`. Output used: the four region drafts, the architecture figure, the script `assignment-4/sd/persistence-test.sh`, a backlog draft, and glossary, bibliography, and history lines. The assistant ran the stack on SD's workstation at `https://localhost` with Caddy's local certificate authority, created a throwaway local administrator whose generated password was kept in a private file outside the repository and never printed, and ran the persistence script twice (the first run failed on a quoting bug, recorded in Table 3.19). Only the rehearsal's container and object IDs are quoted, inside a box marked as not graded; every VM result is **PENDING**. The assistant did not edit BD's regions, `gitea/`, or the root file, and created no cloud resources. Human review and corrections: SD read each draft in the built PDF and approved every push. She ran the server tests herself: the persistence test, with her password typed at a hidden prompt, and the row 6 setup script, with her own token. She took the Gitea screenshots and, with BD, chose a lighter evidence set for row 6 (one public API listing plus three screenshots instead of one screenshot per step). Later work in the same tool and model, October 15, 2026: the row 3 results and figures, the row 7 rewrite for the live AWS deployment (read-only commands run on the server with SD's key), the row 6 subsection, and the row 8 quality pass, including the secrets and address scan. Once during the session SD pasted a Gitea access token into the chat; the assistant did not use it and asked her to delete it in Gitea. The token is not recorded anywhere in this manual or the repository.

C.4 Record: BD Deployment, Evidence, and Drafted Regions

Date: October 12–15, 2026; Tool and model: Claude Code (Claude Sonnet 5.5); Task: carry out Brahmi Dalvi's part of the Gitea assignment: claim the Student Pack domain, create the AWS resources, deploy Gitea and Caddy, verify them, capture evidence, and draft the domain, deployment, HTTPS, and firewall subsections and the Hosts and Passwords tables. Inputs: the course manual [1], BD's screenshots, BD's answers and go-ahead before each step, and real command output from BD's computer and the server. Output used: the security group, key pair, `t3.micro` instance, Elastic IP, first-boot script `gitea/aws/ec2-user-data.sh`, the deployed Compose stack, the evidence files under `evidence/`, and the drafted text.

What the assistant did and did not do. It ran the AWS command line and SSH commands from BD's computer, each after BD's explicit approval, and saved the real output (Chapter 3). It never saw a password or private key: BD created the first Gitea administrator in her own terminal, the SSH private keys were written to files outside the repository

and never printed, and the temporary password for Sakshi's account was shared by BD outside the chat. The assistant verified Sakshi's SSH login from the server's log and read her persistence-test output against the pass conditions in `assignment-4/SAKSHI.md`; it did not verify her Gitea sign-in.

Human decisions the assistant did not make: use AWS (instructor approved), a new instance rather than reusing the earlier one, SQLite, declining the Namecheap authorization because it asked for read and write access to all repositories, declining the paid domain protection, the card handling, and which tests to run. Problems the assistant met and reported rather than hid: a Windows character encoding crash in the AWS tool that made a first check look like a wait, an access-denied on a public image parameter, PowerShell quoting that broke a remote script, a numbering clash in the evidence files, and layout errors in the first draft of the firewall figure. Unverified items are marked pending in the text, notably the certificate renewal test, the second-network firewall test, AWS prices, and the second certificate option. Human review and corrections: Brahmi Dalvi approved each paid or external step before the assistant ran it, supplied every screenshot, and performed the account-creation and token steps herself, so the assistant never handled a password or token. She made the decisions listed above, corrected her mailbox's time zone (which had made an order date look a day off), and chose to turn automatic renewal off and delete the stored card. Assistant errors found and fixed during the group's review were an evidence-numbering clash, a mangled formatting command in the firewall section, and two assumptions about where passwords are stored that nobody had stated.

C.5 Record: SD Rows of the Automated Manual Build Chapter

Date: October 19–20, 2026; Tool and model: Claude Code (Claude Opus 5.5); Task: SD's rows 3, 4, 6, and 8 of Chapter 4. Inputs: the course manual [1] Chapter 3, the team's split in the course repository (`assignment-5/`), Gitea's documentation for Actions variables and configuration [17, 19], and BD's runner. Output used: the standalone manual repository, the job image `ci/Dockerfile`, the workflow, `ci/local-build.sh`, the build identifier mechanism, the change, failure, and repair series (runs #3–#6), evidence 60–66, and rows 3, 4, 6, and 8. The assistant checked the Gitea documentation before relying on context names and artifact retention, and tested every command locally before pushing; it assumed `upload-artifact@v4` would work, which run #1 disproved, and switched to v3. Screenshots of public pages were taken by a headless browser without signing in; evidence was read from Gitea's public API without a token. SD made the first push herself, with her own credentials typed in her own terminal; later pushes used the credential stored on her computer, and no credential was given to the assistant. Human review and corrections: SD read the drafts and approved each push; SD and BD agreed the ownership split.

Glossary

artifact A file produced by a workflow run and stored with that run for download; kept for a limited retention period, so not a backup. 84

automation A repeatable, version-controlled procedure executed by software rather than undocumented manual steps. 84

container An isolated, runnable instance of a container image that packages an application with its dependencies. 84

container image An immutable, layered template built from a Dockerfile from which containers are started. 84

deployment The controlled delivery of a versioned change to an environment. 84

Docker Compose A tool that runs a multi-container application from a versioned YAML file. 84

evidence A retained record of an actual execution, observation, or review. 84

Gitea A self-hosted Git service with issues, pull requests, and organizations. 84

Gitea Actions Gitea's built-in continuous integration, compatible with GitHub Actions workflow files stored under `.gitea/workflows/`. 84

job container A temporary container that a runner starts from a fixed image for one workflow job and removes afterwards, so each build starts clean. 84

Kanban board A visual workflow board with columns such as To Do, In Progress, and Complete, used to make work and work-in-progress visible. 84

named volume Storage managed by Docker and mounted into a container; it outlives the container and is removed only explicitly. 84

observability The ability to infer system behavior from signals such as logs, metrics, traces, and health checks. 84

persistence Data surviving replacement of the container that uses it, because it lives on a mounted volume. 84

pipeline An automated sequence that builds, tests, packages, and/or deploys a change. 84

pull request A request to merge a branch into another, reviewed and approved before merging; in Gitea a description containing "Closes #N" closes issue N on merge. 84

rollback A controlled return to a known acceptable version or state after a failed or harmful change. 84

runbook A documented operational procedure for a repeatable task or incident response. 84

runner A program (here `act_runner`) that registers with Gitea, picks up workflow jobs, and runs them, typically in temporary job containers. 84

security group A stateful virtual firewall attached to an AWS instance that lists the inbound and outbound traffic it allows. 84

SQLite An embedded SQL database stored in a single file and accessed as a library, with no server process, network port, or credential. 84

synonyms Equivalent terms used in this assignment; record preferred terms to avoid ambiguity in instructions and evidence. 84

TeX Live A cross-platform distribution of TeX, LaTeX, and their packages; Overleaf compiles projects with it. 84

user data A script supplied when an EC2 instance is launched that runs once as root at first boot. 84

Acronyms

AI artificial intelligence. 84

AMI Amazon Machine Image. 84

AWS Amazon Web Services. 84

CD continuous delivery or continuous deployment. 73, 84

CI continuous integration. 84

DNS Domain Name System. 84

DORA DevOps Research and Assessment. 84

EC2 Amazon Elastic Compute Cloud. 84

IaC infrastructure as code. 84

SLO service-level objective. 84

SWOT strengths, weaknesses, opportunities, and threats. 84

TLS Transport Layer Security. 84

UML Unified Modeling Language. 84

Index

AI agent, 77
    chapter file convention, 77
    DevOps workflow, 77
AI prompt log, 81
Appendix
    AI Agent Usage Guide, 77
    Document Authoring Guide, 73
artifact, 61
automated manual build, 51
    quality controls, 68

build identifier, 61

controlled failure, 64

document conventions, 73
document history, v

Gitea, 8
    architecture and operation, 43
    database and persistence, 22
    firewall and external exposure, 33
    https and certificate renewal, 26
    manual quality and controls, 47
    optional extra credit: verified backup and restore, 50
    pinned gitea deployment, 14
    startup team workflow, 38
    student pack domain and dns, 9
Gitea Actions, 51

hosts table, 3

introduction, 1
IT manual, 1

job container, 57

LaTeX, 73
latexmk, 57

macOS, 74
MacTeX, 74
MiKTeX, 75
Muresan, Darian, iii

passwords table, 3
pull request, 38

runbook, 43
runner, 57

SQLite, 22
startup DevSecOps process, 8
synonyms, 73

template attribution, iii
TeX Live, 57
troubleshooting
    failed build, 69

Windows, 75